



UNIVERSITÀ DI TRENTO

Department of Information Engineering and Computer Science

Master's Degree in
Computer Science

FINAL DISSERTATION

TITLE
Sub-title (optional)

Supervisor
Roberto Doriguzzi Corin

Student
Alessia Pivotto

Academic year 2024/2025

Acknowledgements

...thanks to...

Contents

Abstract	4
1 Introduction	6
2 Background	8
2.1 Network Intrusion Detection Systems	8
2.1.1 Network Traffic Features and Representation	9
2.1.2 Signature-Based NIDS	10
2.1.3 Anomaly-Based and Machine-Learning NIDS	11
2.1.4 Machine Learning versus Deep Learning in NIDS	12
2.2 Adversarial Machine Learning in Cybersecurity	14
2.2.1 Threat Models	14
2.2.2 Evasion Attacks	15
2.2.3 Poisoning Attacks	17
2.2.4 Constraints in Network Traffic Manipulation	18
2.3 Traffic Manipulation for Evasion	20
2.3.1 Feature-Level Perturbation Techniques	20
2.3.2 Traffic-Level Manipulation Frameworks	21
2.4 Traffic Manipulation and Evasion Techniques	23
2.4.1 Fragmentation as a Primary Evasion Vector	23
2.4.2 Feature-Level Perturbation Techniques	25
2.4.3 Traffic-Level Manipulation Frameworks	25
2.5 Datasets and Adversarial Training for NIDS	27
2.5.1 Taxonomical Limitations of Existing Datasets	28
2.5.2 Adversarial Data Augmentation and Robustness Optimization	28
2.6 Research Tools and Frameworks	28
2.6.1 LUCID: Lightweight Deep Learning Solution for DDoS Detection	28
2.6.2 Traffic Manipulator: Black-Box Adversarial Traffic Generation	29
3 State of the Art	32
3.1 Adversarial Evasion Attacks Against ML-Based NIDS	32
3.1.1 Feature-Space Attack Methodologies	32
3.1.2 Protocol-Aware Problem-Space Attacks	32
3.1.3 Structural and Protocol-Level Manipulation	33
3.2 Defensive Approaches and Countermeasures	33
3.2.1 Adversarial Training and Data Augmentation	33
3.2.2 Input Preprocessing and Denoising	34
3.2.3 Architectural and Algorithmic Robustness	34
3.3 Traffic Manipulation and Evaluation Frameworks	34
3.3.1 The Fundamental Challenge: Feature Space to Problem Space Translation	34
3.3.2 Comprehensive Assessment and Defense Frameworks	35
3.3.3 Pattern-Based and Distribution-Aware Frameworks	36
3.3.4 Traffic-Space Manipulation and Optimization Frameworks	38
3.3.5 Timing-Based and Temporal Manipulation Frameworks	39
3.3.6 GAN-Based Defense and Hardening Frameworks	39
3.3.7 Federated and Distributed Framework Architectures	40

3.3.8	Comparative Framework Analysis	41
3.3.9	Framework Integration and Future Directions	41
3.4	Evaluation Practices and Dataset Limitations	42
3.4.1	Static Benchmark Dataset Challenges	42
3.4.2	Transferability and Generalization Issues	42
3.4.3	Real-World Validation Gaps	42
3.5	Research Gaps and Future Directions	42
3.5.1	Protocol-Aware Adversarial Generation	42
3.5.2	Fragmentation-Aware Robustness	43
3.5.3	Dynamic and Stateful Detection	43
3.5.4	Standardized Robustness Metrics	43
3.6	Critical Analysis and Synthesis	43
3.6.1	Realism versus Optimization Efficiency	43
3.6.2	Practical Deployability Considerations	44
3.6.3	Robustness versus Generalization	44
3.7	Conclusion	44
3.7.1	Framework Maturation and Specialization	44
3.7.2	Persistent Implementation Challenges	45
3.7.3	Critical Integration Gaps	45
3.7.4	Future Research Imperatives	45
3.7.5	Strategic Research Priorities	46
3.7.6	Implications for Network Security Practice	46
4	Methodology	47
4.1	Motivations	47
4.2	System Architecture	47
4.2.1	Phase 1: Feature-Space Optimization (GAN + PSO)	48
4.2.2	Phase 2: Stochastic Fragmentation	48
4.3	FRANTIC	49
4.3.1	Design Principles	49
4.3.2	Architecture	49
4.3.3	Algorithmic Formulation	49
4.4	Protocol Compliance Constraints	50
4.4.1	Stochastic Properties and Adversarial Implications	51
4.4.2	Impact on NIDS Evasion	52
4.5	Preprocessing Pipeline	52
5	Experimental Evaluation	54
5.1	Experimental Setup	54
5.2	Baseline Performance	54
5.3	Performance Under the Original Manipulator	55
5.4	Impact of Fragmentation-Enhanced Attacks	56
5.5	Robustness Analysis	58
5.6	Statistical Considerations	59
5.7	Ablation Study and Design Validation	60
5.8	Single Flow Evolution Analysis	61
5.8.1	Observed Transformation Patterns	61
5.8.2	Implications	62
5.9	Vulnerability Analysis: Feature Reset Test	62
5.9.1	Single-Feature Dependency Analysis	63
5.10	Strategic Insights	64

6	Applications of AEGIS	65
6.1	Dataset Augmentation and Penetration Testing	65
6.2	NIDS Robustness Evaluation	66
6.3	Adversarial Training	66
6.3.1	Training Strategy Comparison	67
6.4	Limitations	67
7	Conclusions and Future Work	71
7.1	Summary of Contributions	71
7.2	Implications for NIDS Research	72
7.2.1	The Critical Role of Traffic Reassembly	72
7.2.2	Rethinking Feature Engineering in ML-NIDS	72
7.2.3	Adversarial Robustness as a Standard Evaluation Criterion	72
7.3	Limitations	72
7.4	Future Work	73
7.4.1	Adversarial Training and Defensive Hardening	73
7.4.2	Extension to IPv6 and Advanced Transport-Layer Manipulation	73
7.4.3	Hybrid and Multi-Layer Detection Architectures	73
7.4.4	Explainability under Adversarial Conditions	73
7.4.5	Real-World Deployment Validation	73
	Bibliography	73
A	Complete tables	79
A.1	F1 score tables attack per attack	79
A.2	FNR tables attack per attack	80
B	Implementation of the Fragmentation Module	81
B.1	Orchestration Logic (<code>manipulator.py</code>)	81
B.2	Fragment Construction Logic (<code>rebuilder.py</code>)	82
C	Complete comparison tables	83
C.1	F1-score comparison	83
C.2	FNR comparison	84

Abstract

The increasing adoption of Machine Learning (ML) and Deep Learning (DL) techniques in Network Intrusion Detection Systems (NIDS) has significantly enhanced the capability to detect sophisticated cyber threats that traditional signature-based methods fail to identify. Unlike conventional detection approaches that rely on predefined rules and known attack signatures, ML-based NIDS learn complex patterns from network traffic data, enabling them to identify previously unseen attacks and zero-day exploits. However, this evolution has also exposed such systems to a new class of sophisticated threats known as adversarial machine learning attacks, where intelligent adversaries deliberately manipulate network traffic to evade detection while preserving the malicious functionality of their attacks.

Current adversarial attacks against NIDS predominantly operate in the feature space, manipulating statistical characteristics extracted from network traffic such as packet sizes, inter-arrival times, and flow durations. While these feature-based approaches have demonstrated effectiveness in controlled environments, they suffer from fundamental limitations in practical deployment scenarios. The irreversible and non-differentiable nature of feature extraction processes makes it extremely difficult to translate feature-space perturbations back into valid, replayable network traffic. This creates a significant gap between theoretical adversarial effectiveness and real-world implementability, where protocol constraints, inter-feature dependencies, and network infrastructure requirements must be satisfied.

Furthermore, existing adversarial frameworks focus primarily on manipulating high-level traffic statistics while largely ignoring structural vulnerabilities in packet processing pipelines. Packet fragmentation, a legitimate networking mechanism defined in RFC 791 for handling packets that exceed Maximum Transmission Unit (MTU) constraints, represents a particularly underexplored attack vector. By fragmenting packets into smaller, protocol-compliant segments, attackers can significantly alter traffic characteristics, disrupt feature extraction algorithms, and exploit weaknesses in packet reassembly logic implemented by many NIDS solutions.

This thesis addresses these critical limitations by developing AEGIS (Adversarial Evaluation and Generation framework for Intrusion detection Systems), a comprehensive framework that integrates FRANTIC (FRAGmeNTation adversarial Component), an advanced packet fragmentation module, with an existing GAN-PSO-based traffic manipulation framework. Unlike purely feature-space approaches, FRANTIC operates at the IPv4 layer to perform controlled, RFC-compliant packet fragmentation that preserves attack semantics while introducing structural perturbations that challenge contemporary NIDS architectures.

The proposed methodology employs a two-phase adversarial generation pipeline that combines the optimization power of existing feature-space manipulation with novel structural transformations. The first phase utilizes a Generative Adversarial Network (GAN) trained on benign traffic distributions combined with Particle Swarm Optimization (PSO) to identify optimal feature perturbations that minimize detection confidence. The second phase applies stochastic IPv4 fragmentation with configurable parameters including fragmentation probability (ρ), minimum fragment size (64 bytes), and maximum fragment size (500 bytes), enabling systematic exploration of structural vulnerability space.

FRANTIC implements four core design principles: strict RFC compliance ensuring all fragments conform to IPv4 standards, configurability through adjustable parameters for systematic experimentation, semantic transparency maintaining invisibility to transport and application layers, and stochastic randomization preventing deterministic evasion signatures. The fragmentation algorithm operates probabilistically on eligible packets, randomly sampling fragment sizes within bounded intervals while ensuring proper 8-byte alignment and correct flag handling required by IPv4 specifications.

Comprehensive experimental evaluation was conducted using the CICIDS2019 dataset across three representative ML-based NIDS architectures: Random Forest (RF), Multi-Layer Perceptron (MLP), and LUCID Convolutional Neural Network. The evaluation methodology compared detection performance across three distinct scenarios: baseline unmodified traffic, traffic manipulated using the

original GAN-PSO framework, and traffic enhanced with FRANTIC fragmentation.

The experimental results demonstrate dramatic performance degradation across all evaluated models when subjected to fragmentation-enhanced adversarial traffic. Random Forest models experienced the most severe impact, with F1-scores collapsing from 0.9999 (baseline) to 0.0306 (fragmented), representing a 96.9% performance degradation. False Negative Rates (FNR) increased dramatically from 0.0001 to 0.9837, indicating that nearly 98% of malicious traffic successfully evaded detection. Multi-Layer Perceptron networks showed similar vulnerability with F1-scores degrading from 0.9987 to 0.1826 (81.7% degradation) and FNR increasing from 0.0014 to 0.8922. Even the temporal-aware LUCID architecture, designed to capture sequential dependencies in packet flows, experienced severe degradation with F1-scores declining from 0.9757 to 0.1695 (82.6% degradation) and FNR rising from 0.0466 to 0.9005.

Attack-specific analysis revealed differential vulnerability patterns across malicious traffic categories. Volumetric attacks such as WebDDoS, NTP amplification, and SYN floods achieved near-complete evasion under fragmentation, with FNR values approaching 1.0 across all models. Protocol-constrained attacks like LDAP and PortMap showed greater resilience but still experienced significant degradation. These findings indicate that fragmentation-based evasion exploits fundamental assumptions in feature extraction pipelines rather than model-specific weaknesses.

Comparative analysis between the original manipulator and the fragmentation-enhanced version confirmed that structural perturbations provide substantially greater evasion effectiveness than feature-space manipulation alone. While the original framework achieved moderate success with FNR increases of 10-20% for most attack types, the addition of fragmentation consistently amplified evasion success across all attack categories and model architectures. This demonstrates that fragmentation operates along a different vulnerability axis than traditional feature manipulation, enabling compound evasion effects that are more difficult to defend against.

The robustness analysis revealed that architectural complexity does not inherently confer adversarial robustness. Traditional machine learning approaches (RF, MLP) and deep learning architectures (LUCID) all exhibited similar vulnerability patterns under fragmentation, suggesting that the effectiveness of this evasion technique stems from exploiting fundamental gaps in traffic representation rather than specific modeling limitations. The results highlight a critical discrepancy between packet-level network reality and the feature-level abstractions used by learning-based detectors.

Beyond adversarial evaluation, this research contributes significant practical applications including dataset augmentation for training more robust NIDS models, penetration testing methodologies for evaluating deployed security systems, and adversarial training frameworks for improving detection resilience. The AEGIS framework enables systematic generation of realistic adversarial traffic that can be directly used for red team exercises, security assessment, and development of next-generation detection capabilities.

The implications of this work extend beyond specific technical contributions to fundamental questions about NIDS architecture and evaluation methodology. The findings demonstrate that high accuracy on clean benchmark datasets provides a false sense of security when systems face realistic adversarial conditions. The research establishes packet fragmentation as a critical vulnerability class that transcends individual model architectures and highlights the necessity for packet reassembly mechanisms integrated before feature extraction in production NIDS deployments.

Future research directions include extending the framework to support IPv6 fragmentation mechanisms, developing real-time adversarial defense systems capable of detecting and mitigating fragmentation-based evasion, investigating hybrid detection architectures that combine signature-based and anomaly-based approaches, and applying explainable AI techniques to understand the precise mechanisms by which fragmentation circumvents detection algorithms.

This thesis provides the cybersecurity community with both offensive capabilities for realistic security testing and defensive insights for developing more robust intrusion detection systems. By bridging the gap between theoretical adversarial machine learning and practical network security, this work contributes to the critical goal of building detection systems capable of operating effectively in hostile, adversarial environments where attackers actively adapt their strategies to circumvent defensive mechanisms.

1 Introduction

Traditional cyber threat detection approaches, such as those based on signatures, rules, and heuristics, have been in use for a long time to protect networks and information systems. These solutions operate using pre-defined rules and criteria defined in the form of signatures for known threats like byte sequences and traffic thresholds. These solutions can be effectively used for known threats; however, they are less effective in dealing with unknown threats.

The major drawback in conventional detection systems is their lack of ability to generalize attack patterns defined explicitly. The polymorphic code, zero-day attacks, and emerging threats pose no difficulty in bypassing such predefined signature-based detection systems. Furthermore, in conventional detection systems, the absence of contextual intelligence often gives rise to situations such as identifying bursts of legitimate traffic as denial-of-service attacks and overlooking two-phase attack plans acted upon over time periods in networks. These detection systems require continuous human intervention, thereby reducing scalability in modern networks.

To address this, learning algorithms from the Machine Learning (ML) family have been increasingly incorporated into the design of Network Intrusion Detection Systems (NIDS). Traditional methods for detecting intrusive actions are largely specific to signature definition. By contrast, learning algorithms developed into NIDS can learn models for detecting intrusive actions from data, and they learn to recognize complicated patterns and behavior anomalies without being limited to signature definition. Historically available data, for instance, can be utilized by ML algorithms to identify patterns for both normal and intrusive actions.

That being said, ML-based NIDS further facilitate multi-dimensional analysis of network traffic by taking into account temporal, statistical, and behavioral features. For instance, sophisticated approaches like deep learning are capable of automatically learning hierarchical features, thereby lessening manual intervention for feature engineering. Nevertheless, to present these benefits, ML-based NIDS also bring forth new issues, for instance, reliance on high-quality training data, the possibility of overfitting, high computational complexity, and lack of interpretability.

Moreover, using ML in the domain of network security introduces fresh security issues as well. Adversarial attacks against the ML-based NIDS are possible; for instance, attackers can attempt to deceive the system with malicious traffic as well as with malicious data used for the training process, keeping the malicious functionality intact.

Most of the adversarial approaches presently lying in the NIDS domain are focused on modifying features extracted from traffic flows, for instance, packet sizes, inter-arrival times, or protocol-level attributes. While effective, these feature-based attacks completely ignore a wide class of evasion techniques based on the manipulation of packet structure. Of these, packet fragmentation represents a very strong and often underestimated evasion vector since it allows an attacker to change traffic characteristics, hinder feature extraction, and exploit weaknesses in the packet reassembly logic of some NIDS implementations.

The lack of standardized adversarial data at a traffic level causes a considerable difference between experimentation and real-world deployment. Feature-space perturbations, while more effective at a lab level, can hardly be backtracked to replayable traffic because of constraints on protocols, inter-feature relationships, and the one-way process involved in feature extractors. This can cause NIDS models to possess an exaggerated sense of resilience when trained or even tested on adversarial data alone, while still being susceptible to methods that deal directly in terms of packet and flow.

This challenge leads to the incentive of developing efficient traffic manipulation techniques capable of producing adversarial traces that are useful in adversarial learning as well as penetration testing. More specifically, a fragmenting approach presents a controllable and legitimate means of manipulating traffic features without affecting the corresponding malicious service provided by the attack traffic: Instead of manipulating traffic features independently, traffic fragmentation allows a different sort of

exploration of the traffic’s feature space by exploiting the traffic’s structural features as seen from an external perspective, which in particular are vulnerable in an ML/DL-powered NIDS concerning flow reconstruction, feature extraction, and temporal modeling.

In this thesis, these challenges are remedied by integrating an existing traffic manipulation toolkit with FRANTIC (FRAGmeNTation adversarial Component), an additional packet fragmentation module for handling this job. The proposed final framework, called **AEGIS** (Adversarial Evaluation and Generation framework for Intrusion Detection Systems), would enable the creation of adversarial examples for network traffic at the packet level of granularity, involving the crafting of subtle manipulations across multiple derived characteristics of the network traffic. This would improve evasion capability while maintaining corresponding attack semantics in order to create more realistic versions of adversarial samples that can be directly used for adversarial training or penetration testing.

This thesis investigates the impact of fragmentation-based adversarial transformations on NIDS robustness through the following research questions:

- **RQ1:** How does packet fragmentation affect the detection performance of machine-learning-based NIDS?
- **RQ2:** Does combining feature-level manipulation with fragmentation provide stronger evasion capabilities compared to feature manipulation alone?
- **RQ3:** Can fragmentation-augmented adversarial traffic be used to construct realistic datasets that support adversarial training, benchmarking, and penetration testing?

The remainder of this thesis is structured as follows: **Chapter 2** provides the background information and the required basic knowledge that is essential in this thesis. **Chapter 3** provides the state of the art in the field, discussing the available traffic manipulation and evasion frameworks for NIDS, with special emphasis on the available packet fragmentation methodologies and their use in evasion techniques, as well as the limitations identified in the state of the art. **Chapter 4** provides the proposed methodology, including the available manipulator, the identified limitations, and the proposed packet fragmentation module. **Chapter 5** presents the experimental evaluation of the proposed methodology and the analysis of the effect of the identified evasion technique on NIDS. **Chapter 6** provides the potential applications of the proposed manipulator, including data augmentation, adversarial testing, and adversarial training. Finally, **Chapter 7** concludes the thesis and provides potential future work.

2 Background

2.1 Network Intrusion Detection Systems

Network Intrusion Detection Systems (NIDS) are a fundamental component of modern cybersecurity infrastructures, designed to monitor and analyze network traffic in real time to identify malicious activities, anomalous behaviors, or policy violations. Positioned at strategic points within a network, NIDS provide a global view of all incoming and outgoing communications, enabling the detection of threats that may compromise the confidentiality, integrity, and availability (CIA) of systems and data [4, 30]. Unlike Host-based Intrusion Detection Systems (HIDS), which monitor individual endpoints, NIDS operate at the network level, offering centralized visibility across multiple devices and communication flows [35].

The primary role of a NIDS is to transform raw network traffic into actionable security intelligence by generating alerts, logs, and forensic data that can be further analyzed by security operators or automated response mechanisms. As network environments grow in scale and complexity, NIDS are increasingly integrated with emerging paradigms such as Software-Defined Networking (SDN) and cloud-native security architectures, where centralized control planes facilitate network-wide monitoring and policy enforcement [10, 18].

From an architectural perspective, a standard NIDS is typically composed of four core functional components: (i) *event generators*, responsible for collecting raw traffic data; (ii) *event analyzers*, which process and analyze this data to identify potential intrusions; (iii) *response units*, which execute predefined or adaptive countermeasures; and (iv) *event databases*, which store historical data for auditing, forensic analysis, and trend discovery [15]. These components collectively form a detection pipeline that converts high-volume packet streams into structured representations suitable for automated analysis.

Detection Pipeline and Operational Framework

To operationalize detection, NIDS process network telemetry through a structured pipeline that typically consists of monitoring, feature extraction, classification, and response stages [50]. During the monitoring phase, raw packet streams are captured from network interfaces, often amounting to massive volumes of data. To enable efficient analysis, packets are aggregated into higher-level abstractions, commonly referred to as *flows*, defined by shared properties such as source and destination IP addresses, ports, and protocol identifiers [27, 25].

Detection mechanisms employed by NIDS are traditionally categorized into two principal paradigms: signature-based detection and anomaly-based detection [8].

Feature extraction represents a critical and irreversible transformation in the NIDS pipeline, converting raw traffic into numerical feature vectors suitable for machine learning models. Depending on the system design, features may be extracted at different granularities [24, 25].

Packet-based NIDS operate on individual packets, capturing attributes such as frame length, TCP flags, and inter-arrival times. Flow-based NIDS compute statistical aggregates over entire connections, including metrics such as packet counts, byte volumes, and average transmission rates, yielding more stable behavioral representations. More advanced approaches further aggregate multiple flows into communication channels to capture long-term behavioral intent between endpoints [26, 50, 16].

Before classification, extracted features typically undergo preprocessing steps such as normalization, encoding, and feature reduction to improve computational efficiency and detection accuracy. Techniques such as min-max scaling, one-hot encoding, and dimensionality reduction methods (e.g., Principal Component Analysis or Genetic Algorithms) are commonly employed to eliminate redundant or irrelevant attributes [19, 7].

Modern NIDS increasingly rely on machine learning (ML) and deep learning (DL) models as inference engines to handle high-dimensional traffic data and complex attack patterns. Architectures such as Multilayer Perceptrons (MLP), Convolutional Neural Networks (CNN), Recurrent Neural Networks (RNN), and Long Short-Term Memory (LSTM) networks are used to capture spatial and temporal dependencies in network traffic [27, 59]. Unsupervised models, including deep autoencoders, perform anomaly detection by computing reconstruction errors, with traffic exceeding predefined thresholds classified as malicious [44].

Following classification, the response phase determines appropriate mitigation actions, ranging from passive alerting and logging to active intervention through Intrusion Prevention Systems (IPS). Automated responses may include blocking malicious IP addresses, terminating suspicious connections, or redirecting traffic to honeypots [15, 13]. Recent research further emphasizes explainable NIDS (X-IDS), which incorporate human-in-the-loop mechanisms and interpretability techniques such as SHAP and LIME to translate model outputs into actionable insights for security analysts [40].

2.1.1 Network Traffic Features and Representation

The representation of network traffic constitutes a fundamental abstraction step in the design of NIDS, necessitated by the immense volume and unstructured nature of raw packet telemetry, which can reach terabytes per day in operational networks. To enable effective analysis, raw packet streams must be transformed into structured feature representations suitable for ML and DL models, a process that is inherently irreversible and non-differentiable [28, 27]. The effectiveness of a NIDS is therefore tightly coupled to the choice of traffic features, which must capture both static protocol semantics and dynamic temporal behaviors while preserving the syntactic validity of network communication [59].

Granularity of Traffic Representation

Network traffic representation is commonly classified according to its level of granularity, ranging from packet-level analysis to flow-level and higher-order aggregated abstractions [26, 27].

Packet-level representation analyzes individual packets upon arrival, providing fine-grained visibility into real-time network states. Features at this level are typically extracted from packet headers and include attributes such as packet length, header sizes, time-to-live (TTL), transport-layer flags, sequence numbers, and immediate timestamps [26]. Some packet-based NIDS further employ sliding temporal windows that group a fixed number of consecutive packets to provide short-term contextual snapshots of traffic behavior. While packet-level analysis enables rapid anomaly detection, the sheer volume of packets often limits the feasibility of supervised learning and motivates the use of unsupervised or outlier-based detection approaches [27].

Flow-level representation constitutes the most widely adopted paradigm in contemporary NIDS. In this setting, packets are aggregated into bidirectional flows defined by a common 5-tuple consisting of source and destination IP addresses, source and destination ports, and the transport protocol [25]. Flow-based features reduce noise and heterogeneity by summarizing communication behavior over time, enabling the extraction of stable statistical patterns that are more amenable to classification. To support online detection, flows are frequently segmented into time-based sub-flows, allowing malicious activity to be identified before connection termination [19]. This representation also facilitates spatial encodings, such as flow matrices, which are particularly suitable for convolutional neural networks (CNNs) that learn localized feature interactions.

Beyond individual flows, higher-order aggregation strategies have been proposed to capture broader behavioral contexts. Channel-level representations, for example, aggregate all multi-flow traffic exchanged between a given source and destination IP pair [16]. Such abstractions are effective in detecting distributed or low-and-slow attacks, command-and-control (C&C) communications, and transient multi-node behaviors that may appear benign when individual flows are analyzed in isolation [61].

Taxonomy of Network Traffic Features

Features extracted for NIDS analysis are commonly organized into three complementary dimensions: temporal, statistical (spatial), and protocol-based features [24].

Temporal features characterize the timing dynamics of network communication and are critical for detecting repetition-based and rate-based attacks, such as Distributed Denial of Service (DDoS). Typical temporal metrics include inter-arrival times (IAT) between packets or flows, flow duration, jitter, and burstiness statistics [19, 27]. These features are particularly sensitive to both benign network variability and intentional obfuscation, as attackers may reshape traffic timing to evade detection without altering payload content [24].

Statistical features provide quantitative summaries of traffic volume and distribution, capturing the intensity and structure of communication. Common examples include total bytes and packets transmitted, packets per second, mean and variance of packet sizes, and ratios between forward and backward traffic [26]. Entropy-based features are also employed to detect sudden changes in the randomness of header fields, such as destination IP distributions, which often accompany volumetric or scanning attacks [19]. Feature selection studies have identified attributes such as *Fwd IAT Total* and *Bwd Packet Length Min* as highly discriminative for DDoS classification [36].

Protocol-based features capture the functional semantics of network communication across the TCP/IP stack. These include TCP flag statistics (e.g., SYN, ACK, FIN, RST), window size dynamics, sequence number behavior, and fragmentation indicators [52, 25]. In encrypted environments where payload inspection is infeasible, NIDS increasingly rely on protocol metadata, such as TLS handshake attributes, cipher suite lists, protocol versions, and certificate properties, to infer malicious intent [16]. Preserving semantic and syntactic validity in these features is essential, as arbitrary perturbations may violate protocol constraints and render traffic invalid or easily detectable [50].

Structural and Graph-Based Representations

Recent NIDS frameworks have moved beyond traditional tabular feature representations toward structural and graph-based abstractions. Graph Neural Network (GNN)-based NIDS model network entities and their interactions as graphs, where nodes represent hosts, flows, or protocol states, and edges encode communication relationships [48, 21]. In such systems, static network attributes, such as known vulnerabilities encoded in attack graphs, are combined with dynamic traffic measurements to identify the specific malicious actions responsible for observed anomalies.

Graph-based representations have also demonstrated robustness against certain obfuscation techniques. For instance, in IoT malware detection, executables may be represented as Control Flow Graphs (CFGs), where nodes correspond to basic blocks and edges represent execution paths, allowing the model to learn structural properties that persist despite syntactic transformations [21]. Feature learning in these settings often exploits node-level metrics, such as degree centrality or connectivity patterns, to capture interdependencies that are difficult to obscure through simple traffic manipulation [48].

2.1.2 Signature-Based NIDS

Signature-based NIDS constitute one of the earliest and most widely deployed paradigms for network threat detection. These systems identify malicious activity by performing deterministic pattern-matching operations between observed network traffic and a predefined repository of known threat signatures [48, 6]. The primary objective of signature-based intrusion detection systems (SIDS) is to safeguard the CIA of networked systems by generating alerts whenever traffic exhibits characteristics associated with previously documented attacks.

In this context, a *signature* is typically defined as a unique sequence of bytes, protocol fields, or traffic attributes that unambiguously characterize a specific malicious behavior. Signatures are derived from indicators of compromise extracted during the analysis of known attacks and are subsequently encoded into detection rules or heuristics [28, 58]. During the measurement phase, the system inspects packet payloads and headers, extracting relevant byte patterns or attributes that are formalized into signatures used for classification. The creation of these signatures often relies on expert knowledge and manual rule engineering performed by security analysts. Widely adopted open-source SIDS platforms, such as Snort and Suricata, maintain extensive and continuously updated rule sets to detect a broad spectrum of attacks, including port scanning, buffer overflows, and denial-of-service (DoS) activities [6, 16].

The operational core of SIDS consists of a classification phase in which extracted traffic patterns are compared against a signature database using string-matching or rule-evaluation algorithms. Alerts are triggered only when traffic satisfies rigid conditions explicitly defined within the rule set, resulting in highly deterministic detection behavior [47, 8]. However, maintaining the effectiveness of SIDS requires continuous manual updates, as new signatures must be generated and distributed for each newly discovered attack. Even minor modifications to malware code or changes in transport protocols can invalidate existing signatures, rendering them ineffective until updated rules are deployed [5].

A major strength of SIDS lies in their high precision and extremely low false-positive rates when detecting known attacks. This reliability makes SIDS particularly attractive in operational environments where false alarms are costly or disruptive [6].

Limitations and Evasion Strategies

Despite their strengths, signature-based NIDS suffer from fundamental limitations due to their reliance on static pattern-matching logic. Most notably, they are inherently incapable of detecting zero-day attacks or previously unseen threats for which no signature exists. Additionally, the increasing prevalence of encrypted network traffic significantly limits the effectiveness of SIDS, as encrypted payloads appear as random ciphertext, preventing the extraction of meaningful semantic signatures. The manual effort required to continuously derive and deploy new signatures further constrains their scalability in dynamic threat environments [26, 16, 19].

Adversaries exploit these limitations through a variety of evasion techniques. Common strategies include byte-level obfuscation and character substitution, which alter payload representations without affecting malicious intent, thereby bypassing exact signature matches. Encryption and tunneling mechanisms, such as TLS-based encapsulation, conceal malicious content from inspection engines, rendering signature databases ineffective [32, 50].

Packet fragmentation represents another powerful evasion strategy, whereby malicious payloads are divided across multiple packets that may be processed independently by the SIDS, preventing the reconstruction of a coherent signature. More advanced techniques include polymorphic and metamorphic malware, which dynamically modify code structure across executions, as well as adversarial manipulation strategies that deliberately exploit known detection rules to evade classification [53, 7]. Collectively, these evasion methods highlight the intrinsic fragility of signature-based detection in adversarial settings.

2.1.3 Anomaly-Based and Machine-Learning NIDS

Anomaly-based NIDS represent a paradigm shift from deterministic signature matching toward behavioral modeling, where malicious activity is identified by measuring deviations from an established profile of normal network behavior [10]. Unlike signature-based systems, which rely on predefined knowledge of known exploits, anomaly-based approaches are inherently capable of detecting previously unseen or zero-day attacks by identifying statistically or semantically abnormal traffic patterns [27]. This capability has driven a widespread transition toward ML and DL techniques, which can automatically extract meaningful representations from the high-dimensional and high-volume traffic characteristic of modern networks [48].

Contemporary anomaly-based NIDS (A-NIDS) typically operate under a semi-supervised or unsupervised learning paradigm, often referred to as a "zero-positive" setting, in which models are trained exclusively on benign traffic to capture the distribution of legitimate network behavior. Raw packets are first transformed into structured feature vectors through a feature extraction process that emphasizes statistical and protocol-level attributes, such as packet sizes, inter-arrival times, flow durations, and TCP flags, particularly to accommodate encrypted traffic where payload inspection is infeasible [23, 27]. The learned representation serves as a baseline against which future observations are evaluated.

Deep Learning architectures, including Deep Neural Networks (DNNs), Recurrent Neural Networks (RNNs), and Graph Neural Networks (GNNs), further enhance detection capabilities by autonomously discovering hierarchical and temporal relationships within traffic without requiring expert-driven feature design [61].

Several detection strategies are employed depending on the model architecture:

- **Reconstruction-based Detection:** Autoencoders (AEs) learn a compressed latent representation of benign traffic through an encoding–decoding process. Malicious traffic, being poorly represented by the learned benign manifold, results in high reconstruction error, which is used as an anomaly score [44].
- **Prediction-based Detection:** Sequence models such as Long Short-Term Memory (LSTM) networks predict future packets or events based on historical context. Deviations are detected when observed traffic exhibits low likelihood under the predicted distribution [27].
- **Graph-based Analysis:** Emerging approaches model hosts and flows as graphs, identifying anomalies through structural or topological deviations from learned benign communication patterns [61].

Limitations

Decision Thresholds and False Positives Anomaly detection decisions are made by comparing an anomaly score to a predefined threshold, often derived using statistical decision theory such as the Neyman–Pearson framework [14]. This introduces an inherent trade-off: higher thresholds reduce false positives but increase false negatives, while lower thresholds improve sensitivity at the cost of excessive alarms. High false-positive rates remain a central limitation of A-NIDS, as legitimate but rare behaviors may be incorrectly flagged as malicious [35]. This issue is exacerbated by the intrinsic complexity of network traffic distributions, which makes learning an accurate and stable decision boundary exceedingly difficult [11].

Concept Drift and Adaptability Network traffic is fundamentally non-stationary, exhibiting periodic patterns and long-term behavioral shifts driven by evolving applications and user activity [35]. To remain effective, A-NIDS must adapt to this phenomenon, known as concept drift, through periodic retraining or online learning mechanisms [50]. While adaptability improves long-term detection performance, it also introduces a critical vulnerability to adversarial manipulation.

The adaptive nature of ML-based A-NIDS exposes them to causative or poisoning attacks, in which adversaries deliberately contaminate the training data to corrupt the learned notion of normality. A notable example is the “Boiling Frog” attack, where small amounts of malicious traffic are gradually injected over extended periods, slowly shifting the decision boundary until large-scale attacks are misclassified as benign [43]. This susceptibility underscores a central research challenge: while A-NIDS excel at detecting unknown threats, their real-world deployment critically depends on robustness against adversarial evasion and manipulation [9].

2.1.4 Machine Learning versus Deep Learning in NIDS

The evolution of NIDS has been characterized by a progressive shift from traditional, shallow ML paradigms toward more expressive DL architectures. Conventional ML-based NIDS commonly rely on algorithms such as Support Vector Machines (SVM), Decision Trees (DT), Random Forests (RF), Naive Bayes (NB), and K-Nearest Neighbors (KNN) to classify network traffic into benign or malicious categories [5, 10]. In contrast, DL-based NIDS employ multi-layer neural architectures, including Multilayer Perceptrons (MLP), Convolutional Neural Networks (CNN), Recurrent Neural Networks (RNN), Long Short-Term Memory (LSTM) networks, and more recently Graph Neural Networks (GNN), to model complex, non-linear network behaviors at scale [7].

Feature Engineering and Data Representation

A fundamental distinction between ML- and DL-based NIDS lies in their approach to feature engineering and data representation. Traditional ML models are heavily dependent on expert-driven feature extraction and selection processes, requiring domain knowledge to manually craft, rank, and reduce features derived from raw traffic. Common techniques include statistical ranking methods such as Information Gain, Chi-Squared tests, Recursive Feature Elimination, and dimensionality reduction

approaches like Principal Component Analysis [40, 25, 51]. While effective in controlled settings, this manual process is labor-intensive and susceptible to cognitive bias, potentially overlooking subtle yet discriminative traffic patterns [20].

Deep learning models mitigate this dependency through representation learning, autonomously extracting hierarchical and non-linear features directly from packet sequences or aggregated traffic statistics. This capability enables DL-based NIDS to capture intricate temporal, spatial, and relational patterns within large-scale network data that may be difficult to encode manually [19, 4]. However, this autonomy can also result in the learning of spurious correlations or the neglect of structured domain constraints, which may negatively impact robustness and generalization [22].

Model Complexity and Computational Requirements

From a computational perspective, traditional ML models are characterized by relatively simple structures and a limited number of learnable parameters. This simplicity translates into low training overhead, fast inference times, and suitability for deployment in resource-constrained environments such as edge devices and Internet-of-Things (IoT) networks [37]. Their performance is primarily governed by the quality of the engineered feature set rather than model depth.

Conversely, DL-based NIDS consist of deep, nested non-linear architectures with a large number of trainable weights and biases. While this complexity allows DL models to achieve state-of-the-art detection accuracy on high-dimensional and large-scale datasets, it imposes high computational costs, often requiring specialized hardware such as GPUs or TPUs and extended training times [19, 4]. These requirements may limit their practicality in latency-sensitive or resource-limited operational settings.

Interpretability and Trustworthiness

Interpretability is a critical requirement for operational NIDS, as security analysts must understand and trust automated decisions. Traditional ML-based NIDS offer a clear advantage in this regard, as many of their models, such as decision trees and rule-based classifiers, are inherently interpretable and function as "white-box" systems [40]. Such transparency enables Cyber Security Operation Center (CSOC) analysts to trace detection decisions back to specific traffic attributes, facilitating troubleshooting and informed response actions [51].

In contrast, DL models are often regarded as "black boxes" due to the opacity of their internal decision-making processes [59]. The difficulty of mapping outputs back to meaningful input features creates a semantic gap between detection scores and actionable insights, potentially undermining operator trust [54]. Although post-hoc explainability techniques, such as SHAP, activation analysis, or GNN-based explanations, have been proposed, they remain approximations and do not fully resolve the inherent lack of transparency of deep models [48].

Robustness and Adversarial Vulnerability

Robustness represents a pervasive challenge for both ML- and DL-based NIDS, albeit in different forms [27]. Traditional ML models often struggle to generalize to previously unseen or zero-day attacks, as they rely on static feature distributions derived from historical data [56]. However, their simpler decision boundaries and lower model capacity can confer a degree of resistance to certain adversarial perturbations, particularly when attacks fail to transfer across model classes [58].

Deep learning models exhibit stronger generalization capabilities but are highly vulnerable to adversarial evasion attacks [59]. Carefully crafted, human-imperceptible perturbations, such as subtle modifications to packet sizes, timing information, or flow statistics, can cause DL-based NIDS to misclassify malicious traffic as benign with high confidence [45]. Moreover, both white-box and black-box decision-based attacks have demonstrated effectiveness across a wide range of learning-based detectors, indicating that high detection accuracy does not necessarily imply adversarial robustness [46].

2.2 Adversarial Machine Learning in Cybersecurity

The increasing adoption of ML and DL techniques in cybersecurity has significantly enhanced the detection capabilities of NIDS. However, this progress has also exposed such systems to a new class of threats collectively referred to as *Adversarial Machine Learning* (AML). AML studies how intelligent adversaries can intentionally manipulate data or learning processes to subvert the behavior of machine learning models, causing misclassification, degradation of performance, or leakage of sensitive information [29].

Unlike traditional threat models, which assume stationary data distributions and passive adversaries, AML explicitly considers attackers who actively probe, manipulate, and adapt to the deployed detection mechanisms [32]. These adversaries may operate under varying degrees of knowledge and capability, targeting different stages of the machine learning pipeline, from training to inference.

2.2.1 Threat Models

The definition of threat models in AML for NIDS requires a multidimensional characterization of an adversary’s knowledge, capabilities, and strategic objectives. Unlike traditional security assumptions that treat attackers as passive or statistically independent from the defense mechanism, AML explicitly considers adaptive adversaries who actively manipulate inputs to induce classification errors. As a result, threat modeling becomes a fundamental prerequisite for evaluating the robustness of ML-based NIDS and for designing realistic adversarial defenses.

Attacker Knowledge Assumptions

The amount of information available to the attacker about the target NIDS strongly influences the effectiveness of an adversarial attack. Accordingly, adversarial settings are commonly categorized into three levels of transparency [10].

White-box attacks assume complete knowledge of the detection system, including the model architecture, learned parameters, loss function, and training data. This level of access enables the attacker to compute exact gradients and generate highly optimized adversarial perturbations, often representing a worst-case scenario for robustness evaluation [58, 3]. Although unrealistic for most real-world deployments, white-box settings are widely used as a benchmark to assess the lower bound of a model’s security [10].

Gray-box attacks represent an intermediate scenario in which the adversary possesses partial information, such as the feature extraction process, the learning algorithm, or the general data distribution, but lacks access to the trained parameters. In this setting, attackers typically rely on surrogate models trained on similar data to approximate the decision boundaries of the target NIDS. Due to the phenomenon of adversarial transferability, perturbations crafted against the surrogate often remain effective against the deployed system [59, 32, 51].

Black-box attacks assume no internal knowledge of the NIDS and restrict the attacker to observing the system’s outputs, such as class labels or confidence scores, in response to crafted inputs. These attacks are usually decision-based or score-based and require iterative probing of the system to infer exploitable vulnerabilities [58, 59]. Black-box threat models are generally considered the most realistic for operational NIDS deployed in production environments [10].

Attacker Capabilities and Operational Phase

Adversarial attacks are further classified based on the stage of the machine learning pipeline at which the adversary intervenes [29]. This distinction is commonly expressed through the dichotomy between causative and exploratory attacks.

Causative attacks, also known as poisoning attacks, occur during the training or retraining phase of the model. In this scenario, the attacker injects carefully crafted malicious samples into the training dataset to corrupt the learned decision function [43]. A representative strategy is the *Boiling Frog* attack, where small perturbations are gradually introduced over multiple retraining cycles, subtly shifting the decision boundary until malicious traffic is classified as benign [35].

Exploratory attacks, or evasion attacks, target the inference phase, where the model parameters remain fixed [29]. Here, the adversary manipulates test-time inputs to bypass the deployed NIDS without altering the training data. In the network domain, this typically involves modifying traffic features, such as packet timing or flow statistics, so that malicious activity is misclassified as legitimate [10].

From an operational perspective, attacker capability is also constrained by the *manipulation space*. *Feature-space attacks* operate directly on the numerical feature vectors used by the classifier, enabling efficient gradient-based optimization but often lacking realism due to the non-invertible nature of feature extraction [24]. In contrast, *problem-space attacks* manipulate raw network traffic (e.g., packets or flows) before feature extraction, requiring the adversary to satisfy strict protocol and semantic constraints while preserving malicious functionality [31].

Adversarial Goals and Security Violations

The strategic objectives of an adversary are typically mapped to classical security violations [60]. *Integrity attacks* are the most prevalent in NIDS and aim to induce false negatives by causing malicious traffic to be classified as benign [29]. *Availability attacks* seek to degrade the overall utility of the detection system by generating excessive false positives or overwhelming the classifier, effectively leading to denial-of-service conditions [15]. *Privacy attacks* focus on extracting sensitive information about the model or its training data through systematic probing [39].

Additionally, adversarial goals can be categorized by their specificity. *Targeted attacks* aim to cause misclassification of specific high-value traffic instances, whereas *untargeted attacks* seek to maximize the overall error rate or reduce the confidence of the model across broad traffic classes [7].

Domain-Specific Constraints in Network Traffic

Adversarial manipulation in the network domain is subject to strict structural and semantic constraints that distinguish NIDS from other AML application domains, such as computer vision. Perturbations must preserve protocol validity, respect feature value ranges, and maintain logical consistency across correlated attributes (e.g., transport-layer flags and protocol identifiers) [25, 38]. Moreover, successful adversarial samples must retain their malicious functionality, ensuring that the attack objective, such as service disruption or unauthorized access, is still achieved after manipulation [33].

As a consequence, time-based features, including inter-arrival times and flow-level statistics, are frequently targeted in adversarial NIDS attacks, as they offer greater flexibility for manipulation while remaining semantically valid [59]. These constraints significantly increase the complexity of problem-space attacks but are essential for producing realistic and deployable adversarial traffic.

2.2.2 Evasion Attacks

Evasion attacks constitute a fundamental inference-phase adversarial strategy against NIDS, in which an attacker aims to bypass a deployed detector without modifying its learned parameters. Unlike poisoning attacks, which corrupt the training process, evasion attacks exploit the sensitivity of ML and DL models to carefully crafted distribution shifts, inducing the misclassification of malicious traffic as benign at test time [29, 5]. This paradigm represents a significant evolution from traditional signature obfuscation techniques toward systematic manipulation of the statistical and temporal representations learned by modern NIDS [2].

Adversarial Intervention Space

Evasion attacks can be categorized according to the space in which adversarial perturbations are applied: feature space or problem space (also referred to as traffic space) [30].

Feature-space attacks operate directly on the numerical feature vectors consumed by the classifier. These methods, often adapted from computer vision, leverage gradient-based optimization to identify minimal perturbations that cross the decision boundary [38, 59]. However, in the NIDS domain, feature extraction is typically non-differentiable and irreversible, rendering feature-space perturbations

difficult or impossible to translate back into valid network traffic [27]. As a result, such attacks are frequently criticized for their limited physical realizability.

Problem-space attacks, by contrast, directly manipulate raw network traffic (e.g., packets or PCAP traces) before feature extraction [50]. The attacker modifies observable packet- or flow-level characteristics so that, once processed by the NIDS feature extractor, the resulting representation lies within the benign region of the model’s decision space. Due to their end-to-end feasibility and independence from internal model knowledge, problem-space attacks represent the most realistic threat model for externally deployed NIDS [24].

Evasion Strategies and Traffic Manipulation Techniques

To achieve evasion, attackers employ a range of heuristic and algorithmic manipulation strategies targeting features known to be influential in classification decisions:

- *Temporal Reshaping*: Inter-arrival times (IAT), jitter, and flow duration are modified by introducing controlled delays between packets, allowing attackers to evade detectors sensitive to temporal regularities [34].
- *Volumetric and Statistical Mimicry*: Attackers inject dummy or “chaff” packets to skew aggregate statistics such as average packet size, packet rate, or total byte count, while preserving the malicious payload [1].
- *Header Manipulation*: Non-functional header fields, including Time-To-Live (TTL), TCP window size, or rarely used flags, are altered to exploit learned model biases while maintaining protocol validity [22, 37].
- *Packet Fragmentation*: Malicious payloads are split across multiple fragments to disrupt flow reconstruction or dilute discriminative features used by the detector [17].

These techniques are often combined to shift the statistical footprint of malicious traffic toward regions densely populated by benign samples, a strategy commonly referred to as *mimicry* [33].

Domain Constraints and Semantic Preservation

A defining challenge of evasion attacks in the network domain is the necessity to satisfy strict syntactic, semantic, and functional constraints that are absent in unstructured domains such as image analysis [25]. Perturbations must preserve protocol compliance, ensuring that packet headers, checksums, and field ranges remain valid; otherwise, malformed traffic may be dropped by intermediate network devices before reaching the NIDS or the intended target [53].

Beyond syntactic validity, attackers must ensure *malicious functionality preservation*, meaning that the modified traffic must still accomplish its original offensive objective [33]. For example, in Denial-of-Service (DoS) attacks, excessive temporal perturbations may reduce the effective transmission rate below the threshold required to exhaust victim resources [24]. Similarly, command-and-control (C&C) communications or injection-based exploits must maintain payload semantics to remain executable on the target system [16, 52].

The Inverse Feature-Mapping Problem

The practicality of evasion attacks is fundamentally constrained by the *inverse feature-mapping problem*. While an attacker may compute an optimal adversarial perturbation in feature space, reconstructing a valid network packet sequence that yields those exact feature values is often computationally intractable due to the one-way nature of feature extraction [24]. This gap between theoretical optimality and operational feasibility limits the effectiveness of purely gradient-based methods in real-world NIDS deployments [17].

Consequently, recent research has increasingly focused on blind perturbation strategies, reinforcement learning-based traffic manipulation, and heuristic-driven approaches that do not rely on exact

inverse mappings or internal model knowledge [9]. These methods, while less precise in theory, offer higher deployability and pose a more credible threat to modern ML-based NIDS infrastructures [50].

2.2.3 Poisoning Attacks

Poisoning attacks, also referred to as *causative attacks*, constitute an adversarial strategy aimed at compromising the learning phase of an ML model by intentionally manipulating the training dataset [29]. In the context of NIDS, such attacks exploit a fundamental assumption underlying most learning algorithms, namely, that training and operational data are drawn from a stationary and trustworthy distribution [27]. By violating this assumption, an adversary can bias the model-construction process, leading to long-term degradation of detection performance.

The primary objectives of poisoning attacks are typically mapped to integrity and availability violations. Integrity-oriented poisoning aims to increase the false negative rate, allowing malicious traffic to bypass the detector undetected, whereas availability-oriented poisoning seeks to overwhelm the system with false positives, effectively rendering the NIDS unusable due to alert fatigue or resource exhaustion [11, 30].

Mechanisms of Training Data Manipulation

Poisoning attacks are commonly categorized according to the adversary’s manipulation strategy on the training set. In *data insertion* attacks, the adversary injects carefully crafted samples into the training dataset, whereas in *data alteration* attacks, existing samples or their labels are modified to corrupt the learning process.

A classical poisoning technique is the *correlated outlier attack*, where spurious features are introduced into malicious training samples to induce the learner to associate those features with benign behavior [29]. Similarly, *red herring attacks* introduce auxiliary features alongside malicious payloads; once the model learns these features as part of a detection rule, the adversary removes them in subsequent attacks, thereby evading detection [29].

In anomaly-based NIDS employing statistical learning techniques such as Principal Component Analysis (PCA), poisoning is often realized through *variance injection*. Here, adversaries introduce carefully engineered “chaff” traffic to inflate variance along specific dimensions, effectively rotating the learned normal subspace toward a malicious direction [43]. As a consequence, subsequent high-volume attacks, such as Denial-of-Service (DoS) traffic, appear statistically consistent with the poisoned notion of normality [43].

More advanced poisoning strategies include *label-flipping attacks*, in which malicious samples are deliberately mislabeled as benign (or vice versa), and *backdoor attacks*, where hidden triggers are embedded during training to force the model to misclassify specific traffic patterns at inference time [2]. In backdoor scenarios, the classifier behaves normally on clean inputs but consistently fails when the attacker-defined trigger condition is satisfied.

Adaptive and Incremental Poisoning

A particularly insidious variant of poisoning is the *Boiling Frog* strategy, which is designed to evade detection in systems that rely on periodic or continuous retraining [43]. In this attack, the adversary initially introduces an imperceptible amount of poisoned data, gradually increasing its presence over multiple retraining cycles. This incremental manipulation acclimates the detector to anomalous patterns, progressively shifting the decision boundary until large-scale malicious activity can be executed without triggering alarms.

Such attacks are especially relevant for self-adaptive or semi-supervised NIDS deployed in non-stationary environments, where models are retrained to accommodate concept drift [35]. In these settings, false negatives produced by the current detector may be erroneously incorporated into future training sets as benign samples, thereby reinforcing and institutionalizing the model’s failure [27].

Practical Limitations and Operational Constraints

Despite their theoretical impact, poisoning attacks face substantial practical barriers. Their success typically requires prolonged access to the training pipeline or influence over data collection mechanisms, capabilities that are more consistent with insider threats or compromised infrastructure than with external attackers [20]. Moreover, poisoning must remain stealthy; anomalous deviations in the training data distribution risk triggering manual inspection or automated data sanitization mechanisms [43].

The feasibility of poisoning in NIDS is further constrained by the *inverse feature-mapping problem*. Because feature extraction from raw network traffic is non-differentiable and irreversible, it is often computationally infeasible to reconstruct valid, replayable network packets that yield attacker-specified feature values [24]. This limitation significantly reduces the practicality of feature-space poisoning attacks in real-world network environments [39].

Defensive Countermeasures

To mitigate poisoning threats, several defensive strategies have been proposed. Data sanitization techniques, such as the Reject On Negative Impact (RONI) defense, evaluate the influence of individual training samples and discard those that disproportionately degrade model performance [29]. Robust statistical estimators, such as Median Absolute Deviation (MAD) or Laplace-based thresholds, can further reduce sensitivity to adversarial outliers by avoiding reliance on Gaussian assumptions [43]. Additionally, controlled retraining procedures, trusted data sources, and human-in-the-loop validation remain critical safeguards against large-scale poisoning attempts.

In summary, while poisoning attacks pose a severe threat to learning-based NIDS, particularly in adaptive and distributed training settings, their operational complexity and access requirements render them less prevalent than evasion attacks in real-world deployments. Nevertheless, they remain a critical concern for insider threat models and emerging paradigms such as federated and self-learning intrusion detection systems [2].

2.2.4 Constraints in Network Traffic Manipulation

The architectural design of NIDS imposes stringent constraints on adversarial traffic manipulation that fundamentally distinguish the network domain from unstructured domains such as computer vision [24]. While adversarial examples in image classification can be generated through independent and continuous pixel-level perturbations, network traffic is governed by a rigid hierarchy of protocol rules, semantic dependencies, and physical limitations that severely restrict the space of realizable modifications [52, 33]. As a consequence, successful evasion strategies must operate in the *problem-space*, ensuring that perturbed traffic remains syntactically valid, semantically coherent, temporally consistent, and functionally malicious [38].

Protocol Compliance and Syntactic Constraints

The most fundamental constraint in network traffic manipulation is strict protocol compliance [1]. Perturbed traffic must adhere to the rules of the TCP/IP protocol stack to ensure that packets are processable by intermediate routers and capable of reaching the intended destination [53]. Any violation, such as incorrect checksums, malformed headers, or inconsistent sequence and acknowledgment numbers, results in packets being discarded before they can be analyzed by the NIDS, thereby nullifying the attack [50].

At the feature level, syntactic constraints prohibit the use of invalid value ranges and incoherent encodings. Attributes such as packet size, flow duration, and byte counts must remain non-negative, while fields like Time-To-Live (TTL) are bounded by fixed protocol limits (e.g., 8-bit values capped at 255) [38]. Furthermore, many NIDS features are discrete or binary (e.g., TCP flags), and adversarial perturbations that produce continuous or fractional values cannot be reconstructed into valid packets [17]. Similarly, categorical features encoded via one-hot representations must preserve mutual exclusivity, as a packet cannot simultaneously belong to conflicting protocols such as TCP and UDP [31].

Semantic Preservation and Feature Interdependencies

Beyond syntactic validity, adversarial traffic must preserve semantic coherence across correlated features [20]. Network telemetry exhibits strong interdependencies; modifying one feature often necessitates corresponding adjustments in others to maintain a logically consistent flow profile [27]. For example, an increase in the total number of transmitted bytes must be accompanied by changes in packet count or payload size, while specifying an application-layer service such as SSH for a UDP-based flow constitutes a semantic violation [52].

This challenge is exacerbated by the *inverse feature-mapping problem*, whereby the transformation from raw packets to feature vectors is non-differentiable and irreversible [24]. Although an adversary may identify an optimal perturbation in feature space, reconstructing a set of replayable packets that yields exactly those values is often computationally intractable [17]. Moreover, problem-space manipulations frequently induce *side-effect features*, where altering a target attribute inadvertently affects correlated features, potentially exposing the attack to detection [50].

Temporal Consistency and Physical Limitations

Temporal dynamics constitute a critical and tightly constrained dimension of network behavior [25]. Adversarial traffic must preserve causal ordering and timing semantics; for instance, acknowledgment packets cannot precede the data they acknowledge, and excessive delays may trigger connection timeouts or protocol retransmissions [20]. While attackers may reshape traffic by increasing inter-arrival times (IAT) to mimic benign distributions, they are constrained by physical transmission limits and protocol-level timeout mechanisms [34].

Consequently, most practical evasion strategies focus on adding delays or injecting chaff traffic to increase temporal variance rather than arbitrarily decreasing IATs [24]. Perturbed temporal features must also remain statistically consistent, ensuring coherence between minimum, maximum, mean, and variance values within a flow [59]. Violations of these temporal relationships often result in traffic patterns that are easily distinguishable from legitimate behavior.

Malicious Functionality Preservation

A defining requirement of any realistic evasion attack is the preservation of malicious functionality [31]. Perturbations that alter payload contents risk disabling the attack itself, such as corrupting an SQL injection string or invalidating a buffer overflow exploit [52]. Similarly, in volumetric attacks like Denial-of-Service (DoS), adversaries must ensure that traffic shaping does not reduce the transmission rate below the threshold required to exhaust the victim’s resources [24].

In operational settings, attackers are often further constrained by limited control over legitimate background traffic and may only be capable of injecting additional packets along an ingress link [43]. To remain stealthy, perturbations are frequently bounded by distortion metrics, such as limiting the Mean Absolute Percentage Error (MAPE) of modified features to within 20%, to avoid detection by statistical anomaly detectors or human operators [59].

Infeasibility of Arbitrary Perturbations

Arbitrary perturbations are fundamentally infeasible in the network domain due to the structured, incomplete, and irregular nature of network traffic feature spaces [27]. Many combinations of feature values simply cannot be realized by any valid packet sequence, leaving large regions of the mathematical feature space unoccupied by real traffic [55]. As a result, naïvely adding Gaussian noise to feature vectors, while effective in image-based adversarial learning, produces samples that are syntactically invalid, semantically incoherent, and operationally meaningless in live networks.

In summary, the effectiveness of adversarial manipulation against NIDS is bounded not by optimization capability alone, but by the necessity to satisfy strict protocol, semantic, temporal, and functional constraints. Failure to respect these domain-specific requirements yields adversarial examples that may succeed in controlled laboratory evaluations but cannot be instantiated or transmitted in real-world network infrastructures [38].

2.3 Traffic Manipulation for Evasion

Adversarial evasion attacks against NIDS aim to bypass detection mechanisms by manipulating malicious traffic at inference time, without altering the parameters of the deployed model. Unlike poisoning attacks, which corrupt the learning process during training, evasion strategies operate exclusively on the input space, crafting traffic that preserves its malicious intent while appearing benign to the classifier [29, 26].

In the network domain, traffic manipulation for evasion must be conducted in the *problem-space*, ensuring that all modifications remain syntactically valid, semantically coherent, temporally consistent, and operationally deployable. This requirement fundamentally distinguishes NIDS evasion from adversarial attacks in unstructured domains such as image classification, where arbitrary perturbations can be applied directly to raw inputs [50, 33].

Existing evasion strategies can be broadly categorized according to the abstraction level at which manipulation is performed. Feature-level approaches directly modify extracted traffic features to induce misclassification, while problem-space techniques operate on raw packets and flows, implicitly influencing the resulting feature representation [53]. This section focuses on feature-level perturbation techniques, which constitute the theoretical foundation upon which many traffic manipulation frameworks, including the manipulator extended in this thesis, are built.

2.3.1 Feature-Level Perturbation Techniques

Feature-level perturbations (FLA) represent a class of adversarial evasion attacks in which malicious network traffic is modified at the level of extracted features to induce misclassification by machine learning-based NIDS. These attacks exploit the sensitivity of both traditional and DL models to small, carefully crafted perturbations that shift malicious samples across learned decision boundaries into regions associated with benign behavior [58, 59]. Since model parameters remain unchanged, FLA are particularly attractive to adversaries operating under realistic constraints where retraining or direct system compromise is infeasible [29].

Taxonomy of Feature-Level Perturbation Strategies

Feature-level evasion techniques are commonly categorized based on the degree of adversarial knowledge and their reliance on model internals [8].

In white-box settings, where the adversary has full access to the target model’s architecture and parameters, perturbations are computed using the gradient of the loss function with respect to the input features. The Fast Gradient Sign Method (FGSM) applies a single-step update in the direction of maximal loss increase, while iterative variants such as the Basic Iterative Method (BIM) and Projected Gradient Descent (PGD) refine the perturbation over multiple steps to improve evasion success [46]. More selective approaches, such as the Jacobian-based Saliency Map Attack (JSMA), identify and perturb only the most influential features, minimizing the overall distortion introduced into the sample. Optimization-based attacks like the Carlini and Wagner (C&W) method further refine this process by explicitly minimizing perturbation magnitude while enforcing misclassification constraints [10].

In more realistic black-box or gray-box scenarios, adversaries lack access to gradients and internal parameters, relying instead on model outputs or binary decisions. Decision-based attacks such as Boundary Attack and HopSkipJumpAttack iteratively refine adversarial samples using only classification feedback [59]. Generative approaches employ models such as Generative Adversarial Networks (GANs), including IDSGAN and WGAN-based frameworks, to learn the manifold of benign traffic and project malicious samples onto it [52]. Alternatively, heuristic optimization techniques such as Genetic Algorithms (GA) and Particle Swarm Optimization (PSO) search the feature space for configurations that minimize anomaly scores while respecting domain constraints [31].

Functional Preservation and Feature Selection

A defining challenge of feature-level perturbation in NIDS is the requirement to preserve malicious functionality while altering the feature representation [38]. To achieve this, attackers conceptually

partition the feature space into *functional* and *mutable* attributes [53].

Functional features, such as protocol identifiers, destination addresses, and exploit-critical flags, must remain unchanged to ensure the attack reaches the target and executes correctly [31]. Consequently, adversarial manipulation is primarily focused on mutable dimensions, including inter-arrival times (IAT), packet sizes (e.g., via payload padding), TCP window sizes, and flow-level statistical aggregates [24]. These features exert a significant influence on NIDS decision-making while allowing controlled modification without violating protocol semantics.

A common strategy is *mimicry*, wherein malicious features are shifted toward the statistical distribution of benign traffic, effectively camouflaging the attack within densely populated regions of the feature space [33]. Advanced frameworks explicitly incorporate domain constraints during optimization to ensure that perturbations remain protocol-compliant and semantically coherent [52].

Limitations and Deployability Challenges

Despite their effectiveness in controlled evaluations, feature-level perturbations face substantial obstacles in real-world deployment [17]. The most significant challenge is the *inverse feature-mapping problem*: the extraction of NIDS features from raw packets is non-differentiable and irreversible, making it difficult to reconstruct a valid packet sequence that precisely matches an optimized adversarial feature vector [24].

Additionally, network features exhibit strong interdependencies; modifying one attribute often necessitates coordinated changes in others to maintain semantic consistency [20]. Naïve perturbations that ignore these relationships produce infeasible traffic patterns that are either discarded by the network stack or trivially detected by the NIDS [50].

From an operational perspective, many black-box attacks assume repeated querying of the target system to refine perturbations. In practice, NIDS rarely expose confidence scores or labels to external entities, and excessive probing can itself trigger defensive mechanisms or blacklist the attacker [34]. Moreover, the computational overhead of iterative optimization or generative training often precludes the use of feature-level attacks in real-time evasion scenarios [7].

In summary, feature-level perturbation techniques provide critical insights into the vulnerabilities of learning-based NIDS but are constrained by realism, deployability, and problem-space feasibility. These limitations motivate the development of traffic manipulation frameworks that operate directly on network packets while implicitly shaping the resulting feature representation, an approach explored in the subsequent sections of this thesis.

2.3.2 Traffic-Level Manipulation Frameworks

The demonstrated vulnerability of ML-NIDS to adversarial manipulation has motivated a transition from abstract feature-space attacks toward traffic-level (problem-space) evasion frameworks that are physically realizable in operational networks [50]. Traffic-level manipulation operates directly on raw network packets and flows, such as those contained in PCAP traces or generated in live traffic streams, so that the features extracted by the NIDS fall within regions associated with benign behavior [34].

Unlike feature-level perturbations, which directly modify numerical vectors, traffic-level frameworks must contend with the *inverse feature-mapping problem*: the transformation from raw packets to structured features is non-differentiable, lossy, and irreversible [17]. As a result, effective evasion requires indirect control over the feature representation through carefully constrained packet- and flow-level transformations.

Traffic-level evasion frameworks employ a range of algorithmic strategies to alter observable traffic characteristics while preserving protocol validity and malicious functionality.

Recent approaches formulate traffic manipulation as a sequential decision-making problem. The NetMasquerade framework models traffic modification as a Finite Horizon Markov Decision Process, leveraging a pre-trained Traffic-BERT encoder to capture benign traffic semantics and guide a reinforcement learning agent in selecting packet-level modifications [34]. Perturbations are applied to controllable attributes such as packet sizes and inter-packet delays (IPD), allowing the attacker to reshape traffic distributions without violating protocol constraints. Similarly, ProGen employs a traffic-space Generative Adversarial Network (GAN), based on DoppelGANger, to project malicious

flows into the benign manifold within a Basic Feature Sequence (BFS) representation [52].

Other frameworks rely on heuristic or rule-based transformations that target directly controllable flow statistics. Approaches such as Adv-Bot manipulate flow duration, packet counts, and byte volumes while enforcing syntactic and semantic constraints through projection operators [17]. Sequence-aware models, including TANTRA, utilize recurrent architectures such as Long Short-Term Memory (LSTM) networks to reshape packet arrival patterns so that attack traffic statistically mimics benign temporal behavior [24].

Practical implementations frequently adopt non-payload-based obfuscation techniques that avoid modifying exploit content. These include packet fragmentation and splitting, injection of dummy or “chaff” packets, and manipulation of packet ordering or spacing to distort aggregate statistics such as mean packet size or packets-per-second rates [50]. Some early approaches also exploit discrepancies between NIDS parsing and host stack behavior by injecting packets with invalid sequence numbers or checksums that are ignored by the victim but processed by the detector, thereby polluting the NIDS state [26].

Constraints and Realism in Adversarial Environments

The feasibility of traffic-level manipulation is governed by a hierarchy of domain-specific constraints that are largely absent in unstructured domains such as computer vision [50, 52].

All adversarial packets must conform to the TCP/IP protocol stack to ensure they are forwarded by intermediate devices and processed by the destination host [53]. Header fields must maintain valid checksums, sequence numbers, and field ranges, while categorical attributes such as protocol identifiers must remain logically consistent [17].

Beyond syntactic validity, traffic-level evasion must preserve the malicious intent of the original attack [27]. Arbitrary payload modifications risk rendering exploits non-executable, while excessive timing perturbations may induce connection timeouts or reduce attack throughput below effective thresholds, particularly in Denial-of-Service (DoS) scenarios [24, 34].

Traffic-level modifications often induce *side-effect features*, whereby changes to one observable attribute inadvertently affect correlated features [50]. For example, injecting chaff packets increases not only packet counts but also total bytes and temporal variance, potentially reintroducing detectable anomalies [50]. Managing these collateral effects is a central challenge in problem-space evasion.

Deployment Challenges and Operational Feasibility

Despite their realism, traffic-level frameworks face significant obstacles in operational deployment.

Most real-world NIDS operate as “invisible” systems, providing no confidence scores or labels to external entities [50]. Consequently, attackers are constrained to hard-label black-box settings, relying on query-efficient strategies or the transferability of adversarial samples crafted against surrogate models [34]. Excessive probing may itself trigger defensive mechanisms or source blacklisting.

The non-stationary nature of network traffic further complicates evasion. Periodic retraining or online learning can shift decision boundaries over time, rendering previously effective adversarial flows obsolete and forcing attackers to continually adapt their strategies [9].

Traffic-level evasion must often be executed under strict timing constraints. Frameworks requiring extensive optimization or long convergence times are impractical for high-throughput or real-time attack scenarios [34].

Evaluation Methodologies

Evaluating traffic-level evasion requires metrics that extend beyond conventional classification performance. Standard measures such as accuracy and F1-score fail to capture the security implications of undetected malicious traffic [24]. Accordingly, prior work proposes metrics such as Malicious Traffic Evasion Rate (MER), Detection Evasion Rate (DER), and Probability Decline Rate (PDR) to quantify both evasion effectiveness and collateral detectability [24].

A comprehensive evaluation must simultaneously assess *validity* (replayability and protocol correctness), *evasiveness* (ability to bypass detection), and *maliciousness* (preservation of attack functionality) [27]. Additional analyses, including ablation studies and cross-model transferability testing, are commonly employed to evaluate robustness across architectures and threat models [17].

2.4 Traffic Manipulation and Evasion Techniques

Traffic manipulation for evasion represents a critical class of adversarial attacks targeting NIDS by modifying malicious network traffic to bypass detection while preserving attack semantics. Unlike poisoning attacks that compromise training data, evasion strategies operate exclusively at inference time, crafting traffic that appears benign to classifiers while maintaining malicious intent [29, 26].

2.4.1 Fragmentation as a Primary Evasion Vector

Packet fragmentation constitutes one of the most effective and widely studied traffic-level evasion techniques, exploiting fundamental discrepancies between traffic interpretation mechanisms of NIDS and end-host protocol stacks. Rather than altering payload semantics, fragmentation operates at the measurement and reconstruction phase by subdividing malicious payloads into multiple protocol data units (PDUs), creating a semantic gap between the traffic representation observed by the NIDS and that reconstructed by the destination host [15, 50].

Fragmentation Mechanisms and Protocol Context

IPv4 Fragmentation Fundamentals: IP fragmentation was introduced in IPv4 as a network-layer mechanism to enable transmission of packets exceeding the Maximum Transmission Unit (MTU) of intermediate network links. When a packet exceeds the MTU of the next hop, routers can dynamically fragment it into multiple smaller units, each encapsulated within its own IPv4 header, with reassembly performed exclusively at the destination host.

The IPv4 header contains three essential fragmentation fields: (i) **Identification** - a 16-bit value shared by all fragments from the same original packet; (ii) **Flags** - including the Don't Fragment (DF) and More Fragments (MF) indicators; and (iii) **Fragment Offset** - specifying fragment position within the original payload in 8-byte blocks. Each fragment carries only a portion of the original payload while duplicating the IPv4 header, with reassembly occurring through fragment buffering until all offsets are received or reassembly timeout expires.

IPv6 Fragmentation Evolution: Unlike IPv4, IPv6 prohibits fragmentation by intermediate routers, requiring fragmentation to be performed exclusively by the source host using dedicated Fragment Extension Headers. This architectural change was designed to simplify router processing and improve forwarding efficiency. However, since most fragmentation-based evasion techniques target IPv4 deployments, which remain prevalent in operational networks, IPv6's modified approach has not eliminated the fundamental vulnerability [12].

Impact on NIDS Processing and Feature Extraction

Packet fragmentation fundamentally disrupts the visibility of malicious activity during critical phases of NIDS operation, particularly feature extraction and flow reconstruction.

Packet-Level Distortions: Fragmentation directly alters low-level packet attributes including frame size, segment length, and TCP length fields, while simultaneously duplicating header information across multiple fragments. These modifications distort features such as packet length distributions, header-to-payload ratios, and protocol-specific heuristics used by NIDS classifiers [1].

Flow-Level Statistical Manipulation: At the flow level, fragmentation increases packet counts associated with single logical transmissions while preserving 5-tuple flow identifiers. This manipulation significantly alters aggregate statistics including packet count, average packet size, bytes per second, flow duration, and packets-per-second metrics, even though underlying communication semantics remain unchanged [16]. In flow-based and sequence-based NIDS, these distortions affect statistical aggregates such as mean packet size, variance, and temporal dispersion commonly used to distinguish benign from malicious behavior.

Signature Dissolution: A critical consequence of fragmentation is signature dissolution, where coherent patterns required for positive detection become distributed across multiple packets. Due to performance constraints, many NIDS architectures perform limited or no stateful reassembly, instead analyzing packets or sub-flows in isolation [37]. When malicious content is fragmented, detection systems may fail to observe complete attack signatures within their analysis windows, particularly impacting machine learning-based detectors that rely on fixed-length feature windows or partial flow observations [1].

Advanced Fragmentation Attack Strategies

Temporal Obfuscation Integration: Sophisticated attackers combine fragmentation with temporal manipulation strategies, introducing artificial delays between fragments to manipulate inter-arrival times (IAT) and reshape traffic’s statistical distribution. This technique shifts malicious flows toward the denser manifold of benign traffic, rendering both statistical anomaly detectors and deep learning models ineffective [34, 27].

Reassembly Policy Exploitation: Adversaries exploit divergent reassembly policies implemented by different operating systems for handling duplicated, overlapping, or out-of-order fragments. NIDS sensors, often optimized for performance rather than protocol completeness, fail to replicate these diverse reassembly behaviors faithfully, resulting in detectors observing partial or incoherent attack representations [37, 12].

Network-Layer Evasion Combinations: Fragmentation can be combined with additional manipulation strategies including packet reordering, intentional transmission delays, and invalid sequence number injection to increase ambiguity in temporal features such as inter-arrival times and sequence coherence. These multi-vector approaches are particularly effective against machine learning-based NIDS that depend on statistical regularities extracted from packet sequences [24, 20].

Empirical Evidence and Performance Impact

Extensive experimental evaluation has documented the significant impact of fragmentation-based evasion on contemporary NIDS systems across multiple detection paradigms.

Classical Classifier Vulnerability: Systematic fragmentation substantially degrades performance of traditional classifiers including Support Vector Machines (SVMs) and Decision Trees by disrupting feature coherence and sequence consistency. These models, which rely heavily on engineered statistical features, prove particularly vulnerable when their underlying feature assumptions are violated through fragmentation [44].

Deep Learning Model Susceptibility: Even state-of-the-art deep learning-based detectors demonstrate significant vulnerability to fragmentation attacks. Evaluations of the LUCID framework revealed that fragmenting HTTP GET flood traffic causes malicious flows to become statistically indistinguishable from benign ones, increasing False Negative Rates (FNR) to values as high as 61.5% [1]. These findings indicate that sequence-aware architectures remain susceptible when temporal and structural assumptions underlying feature extraction are systematically violated.

Defensive Countermeasures and Operational Trade-offs

Stateful Reassembly Requirements: Mitigating fragmentation-based evasion typically requires implementing comprehensive IP defragmentation and stateful TCP reassembly prior to feature extraction. While such defenses can substantially restore detection capability—recovering up to 28% of lost recall in empirical studies—they impose significant computational overhead [37].

Performance and Scalability Constraints: Operational evaluations report CPU utilization increases of approximately 12% when robust reassembly mechanisms are enabled, highlighting a fundamental trade-off between detection robustness and high-throughput performance requirements. This overhead is particularly problematic in high-speed network environments where NIDS must process traffic at line rate without introducing latency bottlenecks.

Persistent Vulnerability Assessment: The computational trade-off between detection accuracy and operational efficiency makes IP fragmentation a persistent and practical evasion vector,

particularly against ML-based detectors that operate on partial observations or rely on aggregated statistics rather than full payload reconstruction. Consequently, fragmentation remains an attractive technique for adversaries seeking low-cost, high-impact evasion strategies against contemporary NIDS deployments.

2.4.2 Feature-Level Perturbation Techniques

Feature-level perturbations (FLA) represent a class of adversarial evasion attacks in which malicious network traffic is modified at the level of extracted features to induce misclassification by machine learning-based NIDS. These attacks exploit the sensitivity of both traditional and deep learning models to small, carefully crafted perturbations that shift malicious samples across learned decision boundaries into regions associated with benign behavior [58, 59].

Taxonomy of Feature-Level Perturbation Strategies

Feature-level evasion techniques are commonly categorized based on the degree of adversarial knowledge and their reliance on model internals [8]. In white-box settings, perturbations are computed using gradients of the loss function with respect to input features, employing methods such as the Fast Gradient Sign Method (FGSM) and iterative variants including Basic Iterative Method (BIM) and Projected Gradient Descent (PGD) [46].

In realistic black-box scenarios, adversaries rely on model outputs or binary decisions rather than internal parameters. Decision-based attacks such as Boundary Attack iteratively refine adversarial samples using only classification feedback, while generative approaches employ Generative Adversarial Networks (GANs) to learn benign traffic manifolds and project malicious samples onto them [52]. Heuristic optimization techniques including Genetic Algorithms (GA) and Particle Swarm Optimization (PSO) search feature spaces for configurations that minimize anomaly scores while respecting domain constraints [31].

Functional Preservation and Operational Constraints

A defining challenge of feature-level perturbation in NIDS is the requirement to preserve malicious functionality while altering the feature representation [38]. Attackers must conceptually partition the feature space into *functional* and *mutable* attributes, where functional features such as protocol identifiers and destination addresses must remain unchanged to ensure attack success [53].

Adversarial manipulation primarily targets mutable dimensions including inter-arrival times, packet sizes, TCP window sizes, and flow-level statistical aggregates that influence NIDS decision-making while allowing controlled modification without violating protocol semantics [24]. Common strategies include *mimicry*, wherein malicious features are shifted toward statistical distributions of benign traffic, effectively camouflaging attacks within densely populated regions of the feature space [33].

Limitations and Deployability Challenges

Despite effectiveness in controlled evaluations, feature-level perturbations face substantial obstacles in real-world deployment [17]. The most significant challenge is the *inverse feature-mapping problem*: feature extraction from raw packets is non-differentiable and irreversible, making it difficult to reconstruct valid packet sequences that precisely match optimized adversarial feature vectors [24].

Network features exhibit strong interdependencies, where modifying one attribute often necessitates coordinated changes in others to maintain semantic consistency [20]. Naïve perturbations that ignore these relationships produce infeasible traffic patterns that are either discarded by network stacks or trivially detected by NIDS [50]. Moreover, many black-box attacks assume repeated querying of target systems, which is impractical for operational NIDS that rarely expose confidence scores and may trigger defensive mechanisms through excessive probing [34].

2.4.3 Traffic-Level Manipulation Frameworks

The demonstrated vulnerability of ML-NIDS to adversarial manipulation has motivated development of traffic-level (problem-space) evasion frameworks that operate directly on raw network packets and

flows rather than abstract feature representations [50]. These frameworks must address the *inverse feature-mapping problem* by achieving indirect control over feature representations through carefully constrained packet- and flow-level transformations.

Recent approaches formulate traffic manipulation as sequential decision-making problems. Net-Masquerade models traffic modification as a Finite Horizon Markov Decision Process, leveraging pre-trained Traffic-BERT encoders to capture benign traffic semantics and guide reinforcement learning agents in selecting packet-level modifications [34]. ProGen employs traffic-space Generative Adversarial Networks based on DoppelGANger to project malicious flows into benign manifolds within Basic Feature Sequence representations [52].

Practical implementations frequently adopt non-payload-based obfuscation techniques including packet fragmentation and splitting, injection of dummy packets, and manipulation of packet ordering or spacing to distort aggregate statistics while avoiding modification of exploit content [50]. Some approaches exploit discrepancies between NIDS parsing and host stack behavior by injecting packets with invalid sequence numbers that are ignored by victims but processed by detectors, thereby polluting NIDS state [26].

Constraints and Realism in Adversarial Environments

Traffic-level manipulation is governed by domain-specific constraints largely absent in unstructured domains such as computer vision [50]. All adversarial packets must conform to TCP/IP protocol stacks to ensure forwarding by intermediate devices and processing by destination hosts. Traffic-level modifications often induce *side-effect features*, where changes to one observable attribute inadvertently affect correlated features, potentially reintroducing detectable anomalies [50].

Operational deployment faces additional challenges including the invisible nature of most real-world NIDS that provide no confidence scores to external entities, constraining attackers to hard-label black-box settings [50]. The non-stationary nature of network traffic further complicates evasion, as periodic retraining can shift decision boundaries over time, forcing continual adaptation of attack strategies [9].

The effectiveness of NIDS critically depends on the accurate reassembly of network traffic, which constitutes a prerequisite for stateful analysis and the reconstruction of original communication semantics from fragmented or segmented protocol data units (PDUs) [15]. In contemporary detection pipelines, this process is typically implemented through flow exporters and analyzers, such as Argus or Zeek, which aggregate individual packets into bidirectional flows based on a canonical five-tuple composed of source and destination IP addresses, source and destination ports, and the transport protocol [17]. For online detection, packets are often collected within predefined temporal windows to form sub-flows, enabling early classification before the termination of a connection [19].

Challenges in Flow and Fragment Reassembly

Despite its central role, flow reassembly in NIDS is inherently fragile due to a persistent semantic gap between the reconstruction policies implemented at the sensor and those enforced by the destination host’s operating system [15]. This gap is exacerbated by both natural network behavior and adversarial manipulation strategies.

Packet Fragmentation and Segmentation. Adversaries exploit reassembly inconsistencies by fragmenting malicious payloads into multiple packets such that the interpretation of the traffic at the NIDS differs from that at the destination host [12]. Different operating systems apply divergent policies for handling duplicated, overlapping, or out-of-order fragments, which NIDS sensors, often designed for performance rather than completeness, fail to replicate faithfully [37]. As a result, the detector may observe only partial or incoherent representations of the attack.

Packet Ordering and Network Noise. Even in the absence of active manipulation, packet reordering and stochastic transmission delays are common in real networks due to multiplexing and congestion [24]. These phenomena complicate causal interpretation, leading to anomalous sequences such as acknowledgment packets arriving before the data they reference [20]. Such disorder undermines the assumptions of detection models that rely on temporally consistent packet sequences.

Temporal Reshaping and Timing Attacks. Attackers further exploit temporal dependencies by intentionally reshaping inter-arrival times (IAT) or injecting packet delays to evade detectors sensitive to timing-based features [34]. While moderate timing perturbations may successfully bypass detection, excessive delays risk triggering transport-layer timeouts, potentially disrupting the malicious activity before reassembly is completed [50].

Implications for Feature Extraction and Detection Accuracy

Fragmentation and temporal inconsistencies have profound implications for both feature extraction and classification accuracy in ML-based NIDS.

Signature Dissolution: When malicious payloads are distributed across multiple fragments, the coherent patterns required for positive detection are dissolved into independent observations [37]. Unless full stateful reassembly is performed, the detector fails to observe a complete attack signature within its analysis window, leading to systematic false negatives [1].

Volumetric and Statistical Distortion: Fragmentation directly alters low-level attributes such as TCP `Len`, decreasing individual packet sizes while inflating aggregate metrics including packet count, flow duration, and packets-per-second ratios [17]. These distortions propagate to higher-level statistical features, such as mean packet size and variance, used to model benign behavior in flow-based detectors [16].

Impact on Detection Performance: Empirical evaluations of DL-based frameworks, including LUCID, demonstrate that fragmented malicious traffic often becomes statistically indistinguishable from benign flows, increasing the False Negative Rate (FNR) to values as high as 61.5% under realistic attack scenarios [1]. This highlights a structural vulnerability in sequence-based and windowed detection architectures when their reassembly assumptions are violated.

Computational Trade-offs and Operational Constraints

While incorporating IP defragmentation and stateful TCP reassembly can substantially restore detection performance, such defenses impose a non-negligible computational overhead [37]. Experimental studies report increases of approximately 12% in CPU utilization when robust reassembly mechanisms are enabled, presenting a critical trade-off between security resilience and throughput requirements in high-speed network environments [37]. Consequently, many operational NIDS adopt partial or heuristic reassembly strategies, leaving them persistently vulnerable to fragmentation-based evasion techniques.

2.5 Datasets and Adversarial Training for NIDS

The empirical evaluation of NIDS, as well as the assessment of their robustness against adversarial manipulation, is fundamentally dependent on the availability of high-fidelity and statistically representative benchmark datasets [27]. Historically, the NSL-KDD dataset has served as a foundational reference for intrusion detection research, having been introduced as a refined version of the KDD Cup '99 dataset to mitigate issues related to data redundancy and class imbalance [7]. Despite its continued use due to its simplicity, NSL-KDD is increasingly regarded as obsolete, as it fails to capture the protocol diversity, traffic volume, and attack complexity characteristic of modern network environments [38].

Contemporary research has therefore shifted toward more realistic benchmarks such as CICIDS-2017 and its successor CSE-CIC-IDS2018, which provide up-to-date attack taxonomies and a rich set of flow- and packet-level features extracted from common protocols including HTTP, HTTPS, FTP, and SSH [7]. The UNSW-NB15 dataset further improves realism by incorporating nine distinct attack categories and traffic captured from a hybrid real-synthetic testbed [3]. For domain-specific scenarios, BoT-IoT and MedBIoT focus on the unique behavioral patterns of Internet of Things (IoT) infrastructures, while Kitsune enables the evaluation of unsupervised anomaly detection through packet-based feature sequences [31]. More recent datasets such as HIKARI21 address the growing prevalence of encrypted traffic by incorporating application-layer attacks encapsulated within HTTPS sessions [51].

2.5.1 Taxonomical Limitations of Existing Datasets

Despite their widespread adoption, existing NIDS datasets exhibit fundamental limitations that hinder the generalizability of learned detection models. A primary concern is the lack of realism: many benchmarks are generated in controlled laboratory environments using automated traffic-generation tools with fixed configurations, resulting in limited structural variability and artificially inflated detection performance [22]. Privacy constraints further exacerbate this issue, as the sensitivity of real enterprise traffic prevents the release of unmasked payloads, forcing researchers to rely on simulated datasets that fail to capture naturally occurring behavioral diversity [27].

Another critical limitation is the insufficient coverage of adversarial behaviors. Standard datasets typically lack samples explicitly designed to evade machine learning-based detectors, compelling researchers to synthesize adversarial examples using techniques such as the Fast Gradient Sign Method (FGSM) or Jacobian-based Saliency Map Attacks (JSMA) [44]. However, these feature-space perturbations often neglect the physical and protocol-level constraints of real network traffic, limiting their practical relevance [26].

Finally, most benchmarks are static and implicitly assume data stationarity, whereas real-world network traffic is inherently non-stationary and subject to continuous concept drift [9]. As a result, models trained on fixed datasets tend to degrade rapidly when deployed in operational environments, exposing a critical gap between laboratory evaluation and real-world performance.

2.5.2 Adversarial Data Augmentation and Robustness Optimization

To counter evasion strategies, adversarial training (AT) has emerged as a primary defensive paradigm for improving the robustness of NIDS [13]. This approach augments the training distribution with adversarially perturbed samples, forcing the model to learn more resilient decision boundaries [42]. Frameworks such as GADoT employ GANs to synthesize “fake-benign” attack traffic, reducing undetected DDoS flows from over 60% to approximately 1.8% in controlled evaluations [1].

More advanced defense architectures integrate multi-stage strategies, combining Gaussian Data Augmentation during training with inference-time Feature Squeezing to suppress adversarial noise [42]. However, adversarial training is susceptible to the “blind-spot” problem, whereby models remain vulnerable to attack families not represented in the augmented dataset [50]. Additionally, retraining on adversarial examples often induces covariate shift, degrading accuracy on clean data and necessitating larger model capacities to maintain baseline performance [57].

Emerging self-supervised approaches, such as BERT-ps models pre-trained on terabyte-scale raw traffic, demonstrate improved robustness to network noise and distributional shifts compared to models trained from scratch or those relying solely on handcrafted augmentation [55]. Finally, the integration of explainable NIDS (X-IDS) and human-in-the-loop analysis is increasingly recognized as essential for bridging the semantic gap between numerical detection scores and actionable security decisions, thereby enhancing trust in automated defense mechanisms [54].

2.6 Research Tools and Frameworks

This section provides an overview of the two main tools utilized in this research: LUCID, a lightweight deep learning-based NIDS for DDoS attack detection, and Traffic Manipulator, a black-box traffic mutation framework for generating adversarial network traffic. These tools represent the foundation for the experimental evaluation and extension work presented in subsequent chapters.

2.6.1 LUCID: Lightweight Deep Learning Solution for DDoS Detection

LUCID (Lightweight, Usable CNN in DDoS Detection) is a practical deep learning framework specifically designed for DDoS attack detection in resource-constrained online environments [19]. The system leverages Convolutional Neural Networks (CNNs) to learn behavioral patterns of both DDoS and benign traffic flows while maintaining low processing overhead and rapid attack detection capabilities.

Architecture and Design Principles

LUCID employs a dataset-agnostic preprocessing mechanism that generates traffic observations consistent with those collected in existing online systems, where detection algorithms must process segments

of traffic flows captured within predefined time windows. This design choice addresses a fundamental challenge in operational NIDS deployment: the need to perform classification on partial flow observations rather than complete connection traces.

The core CNN architecture consists of multiple convolutional layers followed by dense classification layers, optimized for both detection accuracy and computational efficiency. The model operates on traffic flow representations encoded as fixed-size matrices, where each matrix element corresponds to specific packet-level or flow-level features extracted from network telemetry.

Key design principles of LUCID include:

- **Lightweight Processing:** Minimal computational overhead suitable for deployment in resource-constrained environments
- **Real-time Classification:** Rapid inference capabilities enabling online detection with low latency
- **Flow-based Analysis:** Processing of bidirectional traffic flows rather than individual packets
- **Time-windowed Operation:** Classification based on traffic segments collected over configurable time windows

Feature Extraction and Traffic Representation

LUCID’s preprocessing pipeline transforms raw packet captures (PCAP files) into structured feature matrices suitable for CNN processing. Traffic flows are defined by the standard 5-tuple (source IP, source port, destination IP, destination port, protocol) and are processed bidirectionally. The feature extraction process captures both spatial and temporal characteristics of network communications.

The framework supports configurable parameters including:

- **Packets per Flow:** Maximum number of packets aggregated within a single flow sample
- **Time Window:** Length of the temporal window for flow segmentation (in seconds)
- **Traffic Type:** Selective processing of benign, malicious, or mixed traffic

This flexibility enables adaptation to different network environments and traffic characteristics while preserving the semantic structure of communication patterns essential for effective DDoS detection.

Experimental Validation and Performance

LUCID has been extensively evaluated on standard benchmark datasets including CIC-IDS2017, CSE-CIC-IDS2018, and CIC-DDoS2019, demonstrating competitive detection accuracy while maintaining significantly lower computational requirements compared to traditional deep learning approaches. The framework’s effectiveness has been validated across various DDoS attack types, including volumetric, protocol, and application-layer attacks.

The practical deployment characteristics of LUCID make it particularly suitable for evaluating adversarial robustness, as its lightweight nature enables rapid experimentation with different defensive strategies and attack scenarios.

2.6.2 Traffic Manipulator: Black-Box Adversarial Traffic Generation

Traffic Manipulator is a black-box traffic mutation framework designed to generate adversarial network traffic that evades learning-based Network Intrusion Detection Systems (ML-NIDS) while preserving the original malicious functionality. The framework leverages Particle Swarm Optimization (PSO) to efficiently explore the high-dimensional, discrete traffic space and identify effective, functionality-preserving perturbations.

Proposed by Han et al. [24], Traffic Manipulator is among the first frameworks to generate *realistic, functionality-preserving adversarial traffic* capable of bypassing machine-learning-based NIDS.

Unlike feature-space attacks, where crafted perturbations cannot be reliably translated back into valid packet-level representations, the manipulator operates directly in *traffic space*. It modifies raw packets while maintaining protocol compliance and respecting the operational constraints of the underlying malicious activity.

Design Philosophy

The tool follows a two-stage process combining a **Generative Adversarial Network (GAN)** with **Particle Swarm Optimization (PSO)**:

1. **Adversarial Sample Search via GAN.** A GAN is trained on the target dataset to learn approximate decision boundaries of the ML-NIDS. The generator produces candidate samples near the benign/malicious boundary, guiding the subsequent optimization toward regions likely to induce misclassification. Due to the non-differentiability of NIDS feature extraction pipelines, GAN outputs are not used directly as traffic, but rather as *boundary indicators* to initialize PSO.
2. **Traffic Mutation via PSO.** A customized PSO is used to iteratively mutate malicious traffic samples. Each particle encodes a sequence of manipulation operators, and fitness is evaluated by querying the target NIDS. The PSO converges toward mutation patterns that reduce the model’s malicious confidence while keeping functionality intact.

Optimization-Based Traffic Mutation

The core innovation of Traffic Manipulator lies in its formulation of traffic evasion as a constrained optimization problem. Given original malicious network traffic and a target benign feature set, the framework iteratively searches for traffic mutations that minimize the distance between extracted features and the target distribution while maintaining attack functionality.

The PSO-based search mechanism operates through the following key components:

- **Particle Representation:** Each particle represents a potential traffic mutation strategy
- **Fitness Evaluation:** Objective function measuring similarity to target benign features
- **Constraint Satisfaction:** Preservation of protocol validity and attack semantics
- **Neighborhood Search:** Local optimization within particle swarms for efficient exploration

Manipulation Operators

The manipulator employs domain-specific operators that selectively modify traffic attributes deemed safe to perturb, including:

- adjustments to packet inter-arrival times,
- controlled modifications of transport-layer payload sizes,
- packet reordering and fine-grained timing perturbations,
- insertion or modification of protocol-layer options when semantically permissible.

These operators are carefully designed to ensure that the generated adversarial traffic remains functionality-preserving, protocol-compliant, and semantically valid, while increasing the likelihood of being classified as benign by the target ML-NIDS.

End-to-End Workflow

Given an initial malicious network flow, the framework initializes mutation candidates using GAN-informed latent representations and iteratively refines them through Particle Swarm Optimization (PSO). At each iteration, candidate mutations are evaluated against the target detection model, and the optimization process continues until the traffic is misclassified as benign or a predefined iteration budget is exhausted.

Through this structured optimization strategy, the manipulator enables effective gray-box, and, in certain configurations, fully black-box, evasion. It is widely regarded as a strong baseline for generating realistic, functionality-preserving adversarial network traffic.

Model-Agnostic and Generic Framework

A distinguishing characteristic of Traffic Manipulator is its model-agnostic design, which enables evaluation against diverse NIDS architectures without requiring knowledge of internal model parameters or training data. This black-box approach more accurately reflects real-world attack scenarios where adversaries lack access to target system internals.

The framework has been validated against multiple state-of-the-art NIDS implementations, including Kitsune, demonstrating consistent evasion effectiveness across different detection paradigms. The generic architecture facilitates rapid adaptation to new target systems and traffic types, making it a valuable tool for systematic adversarial robustness evaluation.

Practical Implementation and Deployment

Traffic Manipulator provides a comprehensive implementation including feature extractors, traffic rebuilders, and evaluation metrics necessary for end-to-end adversarial traffic generation. The framework supports standard network trace formats and integrates with existing traffic analysis tools, enabling seamless incorporation into security research workflows.

Configurable parameters allow fine-tuning of the optimization process to balance between evasion effectiveness and computational efficiency, supporting both comprehensive security evaluations and rapid prototyping of defensive mechanisms.

Overall, the choice of traffic representation and feature taxonomy plays a decisive role not only in detection accuracy but also in the robustness of NIDS against evasion and adversarial manipulation. Understanding these feature dimensions is therefore essential for evaluating the effectiveness and limitations of traffic manipulation strategies, as explored in subsequent chapters.

3 State of the Art

The proliferation of deep learning-based Network Intrusion Detection Systems (NIDS) has introduced unprecedented capabilities for detecting sophisticated cyber threats, yet it has simultaneously revealed critical vulnerabilities to adversarial manipulation. This chapter provides a comprehensive examination of the current state of the art in adversarial machine learning applied to network intrusion detection, encompassing attack methodologies, defensive mechanisms, and evaluation frameworks. The analysis focuses particularly on the fundamental challenges of bridging theoretical adversarial examples with practical network constraints, highlighting the evolution from feature-space manipulations to protocol-aware traffic generation.

3.1 Adversarial Evasion Attacks Against ML-Based NIDS

The literature identifies a critical vulnerability in deep learning-based Network Intrusion Detection Systems, where malicious actors employ evasion attacks at inference time to induce misclassification without altering the model’s parameters [?, ?]. These attacks leverage small, calculated perturbations to malicious traffic, shifting them across the learned decision boundary such that they are perceived as benign [?, ?]. The research community has approached this challenge from multiple angles, each with distinct advantages and limitations.

3.1.1 Feature-Space Attack Methodologies

The majority of current research focuses on Feature-Level Attacks (FLA) that directly perturb numerical characteristics of input traffic vectors [?]. This approach treats network data as abstract numerical tensors, applying well-established adversarial techniques from computer vision domains. Common algorithms include the Fast Gradient Sign Method (FGSM), Jacobian-based Saliency Map Attack (JSMA), DeepFool, and Carlini & Wagner (CW) attacks [?, ?]. These methods typically assume that attackers can manipulate high-level statistical features such as flow duration, total bytes, or packet count distributions [?].

The optimization strategies employed in feature-space attacks predominantly utilize gradient-based methods to identify directions of maximal classification loss [?]. Advanced iterations incorporate multi-step saddle-point (min-max) formulations to discover worst-case perturbations within defined ℓ_p norm constraints [?, ?]. The mathematical elegance of these approaches has made them attractive for theoretical analysis, as they provide clean optimization objectives and measurable perturbation bounds.

However, feature-space attacks suffer from fundamental realism limitations. The transformation from raw network packets to feature vectors is neither invertible nor differentiable, making it impossible to reconstruct valid network traffic from perturbed feature representations [?]. Authors explicitly acknowledge that feature-level attacks often result in unrealistic adversarial features because they ignore domain-specific constraints [?, ?]. Perturbations frequently produce invalid value ranges, such as negative packet lengths or incoherent binary flags, rendering the resulting traffic physically impossible to implement on actual networks [?].

3.1.2 Protocol-Aware Problem-Space Attacks

Recognizing the limitations of feature-space manipulation, a distinct research direction has emerged focusing on problem-space attacks that manipulate the physical manifestation of network traffic while respecting protocol constraints [?, ?]. This approach represents a fundamental shift from abstract mathematical optimization toward practical implementability [27, 50].

Protocol-aware frameworks allow for legitimate transformations that maintain network communication integrity [?]. These include splitting malicious packets into multiple fragments, altering inter-arrival times between packets, modifying non-critical header fields, padding payloads, and injecting

crafted stealthy packets [?, ?]. The search strategies employed range from heuristic optimization algorithms like Particle Swarm Optimization (PSO) to reinforcement learning agents that learn arbitrary perturbations within defined constraint sets [?, ?].

Protocol compliance is enforced through careful restriction of perturbations to features that do not break communication integrity. Safe mutation operators ensure that original packet ordering is preserved, critical protocol fields remain valid, and the resulting traffic adheres to TCP/IP stack requirements. Functionality verification mechanisms ensure that perturbed samples maintain their malicious intent when executed against target systems.

The evolution toward problem-space attacks represents a crucial maturation of adversarial NIDS research, acknowledging that theoretical mathematical success must translate to practical network implementability. However, this approach introduces new complexities in terms of optimization efficiency and constraint satisfaction.

3.1.3 Structural and Protocol-Level Manipulation

The literature distinguishes structural manipulation from feature-space perturbation by emphasizing alterations to packet sequence, timing, or composition that are functionally implementable on live networks. This category encompasses several sophisticated techniques that exploit the inherent complexity of network protocol stacks.

Packet fragmentation emerges as a particularly important structural manipulation technique [?]. This approach is explicitly identified as a legitimate transformation that bypasses NIDS by splitting malicious packets into multiple fragments [?]. Fragmentation subverts detection by ensuring that individual packets, when processed by feature extractors, do not contain sufficient salient characteristics to exceed classification thresholds [?]. The technique exploits the over-generalization problem common in anomaly-based NIDS, where systems trained on full observations fail to recognize fragmented malicious traffic [?].

Protocol field manipulation represents another significant attack vector, where adversaries alter IP flags, TCP sequence numbers, or other header fields in ways that are processed by NIDS but may be ignored or handled differently by target operating systems [?]. This creates potential for semantic gaps between detection and actual system behavior.

Timing alteration through Inter-Arrival Time (IAT) manipulation constitutes a core structural strategy [?]. Research indicates that attackers can achieve success rates exceeding 35% by manipulating only time-based features, demonstrating the vulnerability of temporal-based detection mechanisms [?]. Traffic reshaping techniques further extend this concept by aligning malicious traffic’s statistical distribution with benign patterns, effectively disguising attacks within normal operational bounds [?].

3.2 Defensive Approaches and Countermeasures

The adversarial threat against ML-based NIDS has prompted extensive research into defensive mechanisms. These approaches span proactive training-time hardening, reactive detection mechanisms, and architectural modifications designed to enhance robustness against sophisticated evasion attempts.

3.2.1 Adversarial Training and Data Augmentation

Adversarial Training (AT) represents the dominant proactive defense strategy, involving augmentation of training datasets with adversarial examples to reinforce model decision boundaries [?]. This approach aims to create more robust classifiers by exposing them to potential attack patterns during the learning phase. Advanced variants include Ensemble Adversarial Training, which utilizes samples transferred from multiple disparate models to harden detectors against a broader range of attack types [?].

The implementation of adversarial training in network intrusion detection contexts requires careful consideration of domain-specific constraints. Unlike computer vision applications where adversarial examples can be generated through direct pixel manipulation, network traffic adversarial training must respect protocol semantics and maintain functional validity. This constraint significantly complicates the generation of diverse, realistic adversarial training samples.

Effectiveness evaluations reveal that while adversarial training increases robustness against known attack patterns, it often incurs a natural accuracy trade-off where performance on clean data degrades.

Furthermore, adversarially trained models may exhibit brittleness against adaptive attacks where adversaries gain knowledge of the specific defense mechanisms employed.

3.2.2 Input Preprocessing and Denoising

Input preprocessing and denoising methods attempt to purify traffic before it reaches classification stages. These approaches leverage the hypothesis that adversarial perturbations represent noise-like patterns that can be filtered without affecting legitimate traffic characteristics [?]. Techniques include utilizing Denoising Autoencoders (DAE) to map perturbed data back onto learned manifolds of clean traffic [?].

Traffic normalization and reassembly approaches, such as Reconstruction from Partial Observation (RePO), leverage denoising autoencoders to detect attacks even when traffic is presented as disjointed fragments [?]. RePO specifically addresses the over-generalization problem by training models to reconstruct data from randomly masked partial observations, forcing learning of latent semantics rather than superficial statistical patterns [26]. This approach demonstrates significant robustness improvements in adversarial settings, though it incurs substantial computational overhead during inference [26].

The effectiveness of preprocessing-based defenses varies considerably depending on the sophistication of attack methods. While effective against basic perturbation strategies, these defenses can often be circumvented by adaptive attacks that account for the specific preprocessing mechanisms employed.

3.2.3 Architectural and Algorithmic Robustness

Beyond data-centric defenses, research has explored architectural modifications and algorithmic approaches to enhance intrinsic robustness. Robust feature engineering involves identifying and omitting features that are easily manipulated by attackers, such as packet size and inter-arrival times, to create more stable though potentially less accurate models [?].

Graph-based approaches represent a novel direction in robust NIDS design, modeling network traffic as dynamic graphs where nodes represent network entities and edges capture communication patterns [?]. This representation naturally captures structural relationships that are more difficult for adversaries to manipulate without disrupting fundamental communication patterns [?].

Ensemble methods combine multiple diverse models with the hypothesis that adversarial examples crafted for one model are less likely to fool an ensemble of heterogeneous classifiers. The effectiveness of this approach depends critically on the diversity of ensemble components and their relative susceptibility to common attack patterns.

3.3 Traffic Manipulation and Evaluation Frameworks

The evolution of adversarial NIDS research has necessitated development of sophisticated frameworks that address the fundamental challenge of translating theoretical adversarial machine learning concepts into realistic network attack scenarios [27]. These frameworks represent the core contribution of contemporary research, attempting to bridge the gap between mathematical optimization in feature spaces and the practical constraints of network protocol implementation [38, 51]. This section provides a comprehensive analysis of the major frameworks that have emerged to address different aspects of adversarial traffic generation, evaluation, and defense.

3.3.1 The Fundamental Challenge: Feature Space to Problem Space Translation

Before examining specific frameworks, it is essential to understand the core problem they attempt to solve. Traditional adversarial machine learning operates in continuous feature spaces where perturbations can be applied through gradient-based optimization [24]. However, network traffic exists in a highly constrained, discrete domain where modifications must respect protocol semantics, maintain functional integrity, and remain implementable on actual network infrastructure [27, 50].

The irreversibility problem represents the most significant challenge: while image pixels can be directly modified to create adversarial examples, network features are derived through complex, non-invertible transformations from raw packet data [24]. This fundamental asymmetry has driven the development of specialized frameworks that operate either by constraining feature-space modifications to realistic ranges or by directly manipulating traffic in the problem space [50].

3.3.2 Comprehensive Assessment and Defense Frameworks

TIKI-TAKA: Time-Based Robustness Assessment

TIKI-TAKA represents one of the most comprehensive frameworks for assessing NIDS robustness while incorporating advanced defensive mechanisms [59]. The framework employs a sophisticated multi-component architecture designed to evaluate vulnerability to temporal-based adversarial attacks while simultaneously implementing countermeasures against such attacks [59].

Mechanism and Architecture: The core of TIKI-TAKA lies in its systematic approach to black-box adversarial evaluation. The framework implements five distinct decision-based attack strategies: Natural Evolution Strategy (NES), Boundary Attack, HopSkipJumpAttack, Pointwise Attack, and OPT-ATTACK. These algorithms operate under strict query limitations, reflecting realistic constraints where attackers cannot perform unlimited probing of target systems. The framework restricts perturbations to a carefully selected subset of 22 temporal features, including Inter-Arrival Times (IAT), flow duration statistics, and active/idle period characteristics, while preserving protocol-critical fields such as TCP flags and port numbers.

The temporal focus of TIKI-TAKA reflects a deep understanding of NIDS vulnerability patterns. Many deep learning-based detection systems rely heavily on temporal correlations and statistical patterns derived from packet timing. By systematically manipulating these characteristics while preserving semantic content, TIKI-TAKA can effectively evaluate the robustness of temporal-based detection mechanisms.

Defense Integration: TIKI-TAKA’s defensive component implements a tripartite strategy combining model voting ensembling, Ensemble Adversarial Training (EAT), and reactive query detection. The ensembling approach leverages diversity across multiple model architectures to reduce the likelihood that adversarial examples transfer effectively across all ensemble components. The EAT component proactively hardens individual models by training them on adversarial examples generated from multiple source models, improving robustness against transferable attacks.

The query detection mechanism represents a particularly innovative component, employing Deep Similarity Encoders (DSE) to identify and blacklist sources performing repeated, highly similar adversarial probing. The system maintains interaction buffers that track query patterns and applies similarity thresholds to detect systematic boundary exploration attempts. Upon detecting suspicious query patterns, the system can implement IP-based blacklisting or introduce additional verification requirements.

Limitations and Challenges: Despite its sophistication, TIKI-TAKA faces several significant limitations. The introduction of adversarial training typically results in natural accuracy degradation on clean traffic, creating a fundamental trade-off between robustness and baseline performance. The query detection mechanism, while innovative, relies on similarity thresholds that sophisticated adversaries might circumvent by alternating between adversarial queries and benign traffic to mask their exploration patterns.

The framework’s focus on temporal features, while addressing a critical vulnerability class, may miss other attack vectors that exploit spatial or semantic relationships in network traffic. Additionally, the computational overhead introduced by ensemble methods and repeated similarity calculations may limit deployability in high-throughput network environments.

RePO: Reconstruction from Partial Observation

RePO addresses the fundamental over-generalization problem in anomaly-based NIDS through an innovative architectural approach based on denoising autoencoders [26]. This framework represents a shift from traditional signature-based or statistical detection toward learned representations that are inherently robust to partial observations and adversarial perturbations [26].

Architectural Innovation: The core innovation of RePO lies in its training methodology, which forces models to learn robust latent representations by reconstructing traffic from randomly masked partial observations. During training, a masking function with hyperparameter (typically 0.75) randomly hides portions of input feature vectors, compelling the model to learn the underlying semantic structure of normal traffic rather than superficial statistical patterns. This approach directly addresses the over-generalization problem where standard autoencoders learn to reconstruct both benign and malicious traffic with equal facility.

The masking process serves dual purposes: it prevents overfitting to specific feature combinations and creates inherent robustness against adversarial perturbations that might fool models trained on complete observations. The resulting learned representations capture essential characteristics of normal traffic that are preserved even when partial information is available.

RePO+ Enhancement: The enhanced version, RePO+, introduces a sophisticated inference mechanism that replicates each test sample 100 times with different random masks, creating a non-deterministic evaluation process. The resulting reconstruction errors are aggregated, typically through statistical measures such as mean and variance, to compute a final anomaly score. This stochastic approach significantly complicates adversarial example generation, as attackers must craft perturbations that fool the detector across multiple random mask configurations.

The non-deterministic nature of RePO+ represents a fundamental shift in adversarial robustness strategy. Traditional adversarial defenses attempt to harden models against specific attack patterns, while RePO+ creates a moving target where the effective decision boundary changes with each evaluation. This approach prevents adversaries from crafting universal adversarial examples that consistently evade detection.

Computational Complexity and Deployment Challenges: The primary limitation of RePO and RePO+ lies in computational complexity. The requirement to perform 100 parallel forward passes for each test sample during inference creates substantial computational overhead that may be prohibitive for real-time detection in high-throughput network environments. The framework's effectiveness depends critically on the quality of learned representations, which may require extensive training on representative datasets that capture the full diversity of normal network behavior.

Furthermore, the masking strategy, while effective for robustness, may inadvertently mask critical features needed for detecting sophisticated attacks that closely mimic normal traffic patterns. The balance between masking aggressiveness and detection sensitivity requires careful tuning for specific deployment environments.

3.3.3 Pattern-Based and Distribution-Aware Frameworks

A2PM: Adaptive Perturbation Pattern Method

A2PM represents a sophisticated approach to generating realistic adversarial examples for both robustness evaluation and training data augmentation [38]. The framework addresses the critical challenge of maintaining domain constraints while achieving effective adversarial perturbation through pattern-based learning and adaptation [38].

Pattern Learning Mechanism: A2PM operates by learning statistical patterns characteristic of different traffic classes through systematic analysis of feature distributions and correlations. The framework records valid value intervals for feature subsets and learns inter-feature dependencies that must be preserved during perturbation. This pattern-based approach ensures that generated adversarial examples respect the natural statistical relationships present in real network traffic.

The learning process involves comprehensive statistical analysis of training data to identify class-specific characteristics, correlational structures, and permissible value ranges. The resulting patterns serve as constraints during adversarial generation, ensuring that perturbations remain within realistic bounds while achieving classification evasion.

Gray-Box Optimization: A2PM employs a gray-box perturbation strategy that leverages partial knowledge about target models while respecting domain constraints. The framework iteratively optimizes perturbations to maximize evasion probability while ensuring compliance with learned patterns and statistical constraints. This approach balances the optimization power of gradient-based methods with the realistic constraints necessary for implementable attacks.

The optimization process incorporates multiple objectives including evasion effectiveness, constraint satisfaction, and minimal perturbation magnitude. The framework employs multi-objective optimization techniques to navigate the complex trade-offs between these competing requirements.

Application Scope and Limitations: A2PM demonstrates particular effectiveness for Enterprise networks and established attack categories where sufficient training data exists to learn robust statistical patterns. The framework excels in scenarios involving structured attacks with well-defined characteristics, such as traditional DDoS patterns or established malware communication behaviors.

However, the framework’s effectiveness degrades when facing high class imbalance or novel attack patterns not well represented in training data. The pattern-based approach inherently assumes that future attacks will exhibit statistical characteristics similar to historical examples, potentially missing zero-day attacks or sophisticated evasion techniques specifically designed to break learned patterns. Additionally, performance on modern encrypted traffic or complex application-layer attacks may be limited by the difficulty of extracting meaningful patterns from limited observational data.

ProGen: Projection-Based Distribution Mapping

ProGen introduces a revolutionary approach to adversarial traffic generation through distribution-level manipulation rather than instance-level perturbation [?]. This framework represents a paradigm shift from traditional adversarial example generation toward comprehensive distribution mimicry [?].

Distribution-Projection Paradigm: ProGen employs specialized traffic-space Generative Adversarial Networks, specifically the DoppelGANger architecture, to learn complex mappings between malicious and benign traffic distributions. Rather than adding noise to individual instances to cross decision boundaries, ProGen learns to project entire malicious traffic distributions onto the manifold of benign traffic. This approach addresses fundamental limitations in traditional adversarial generation by ensuring that generated traffic exhibits the full statistical complexity of legitimate network communications.

The framework utilizes Basic Feature Sequence (BFS) representations that bridge numerical computation requirements with traffic semantics. BFS representations maintain sufficient detail for optimization while avoiding feature conflicts that could violate protocol constraints. The representation includes temporal, statistical, and structural characteristics that preserve the essential nature of network communications.

GAN-Based Traffic Generation: The DoppelGANger component of ProGen specifically addresses the challenges of generating realistic sequential data with complex dependencies. Traditional GANs often struggle with the temporal and correlational complexity of network traffic, but DoppelGANger incorporates specialized architectures for handling sequential dependencies and multi-variate correlations characteristic of network flows.

The generation process involves training on large corpora of benign traffic to learn the underlying statistical distributions and temporal patterns. The trained generator can then transform malicious traffic samples by mapping them onto learned benign manifolds while preserving essential attack functionality.

Validation and Real-World Deployment Challenges: While ProGen demonstrates impressive results in controlled evaluation environments, significant challenges remain for real-world deployment. The framework has primarily been evaluated on static datasets using offline metrics, with limited validation in live network environments where additional complexities such as network jitter, routing variations, and infrastructure diversity may affect performance.

The GAN training process requires substantial computational resources and extensive datasets representative of target network environments. The learned distributions may not transfer effectively across different network contexts, limiting the framework’s generalizability. Additionally, the complexity of the generation process may introduce latencies incompatible with real-time attack scenarios.

3.3.4 Traffic-Space Manipulation and Optimization Frameworks

TrafficManipulator: Bi-Level Optimization Architecture

TrafficManipulator represents one of the most sophisticated attempts to combine the optimization power of machine learning with the practical constraints of network traffic generation [51]. The framework employs a bi-level optimization strategy that separates feature-space exploration from traffic-space implementation [51].

Two-Stage Optimization Process: The first stage of TrafficManipulator employs GANs to explore adversarial feature spaces in a model-agnostic manner. This stage operates in the abstract feature domain where optimization can leverage gradient-based methods for efficient exploration of the adversarial landscape. The GAN component learns to identify feature modifications that maximize evasion probability against target detection models.

The second stage employs Particle Swarm Optimization (PSO) combined with heuristic packet crafting to translate abstract feature modifications into concrete traffic mutations. This stage operates under strict safety constraints that ensure resulting traffic maintains protocol validity and functional integrity. Permitted mutations include inter-arrival time modification, payload padding, packet replication, and header field adjustment within protocol-specified ranges.

Safe Mutation Operators: TrafficManipulator implements a comprehensive set of safe mutation operators designed to bridge the gap between feature-space optimization results and implementable traffic modifications. These operators ensure that mutations preserve original packet ordering, maintain protocol state consistency, and respect semantic constraints inherent in network communications.

The safety framework includes validation mechanisms that verify protocol compliance, functional preservation, and implementability constraints. Each proposed mutation undergoes multi-level verification including syntactic correctness, semantic consistency, and functional validation through simulation or controlled testing.

Meta-Information Representation: A critical innovation in TrafficManipulator is its meta-information vector representation that provides invertible mappings between traffic instances and numerical feature spaces. This representation enables gradient-based optimization while maintaining sufficient detail to reconstruct valid traffic instances that implement the optimized modifications.

The meta-information vectors capture essential traffic characteristics including timing patterns, size distributions, protocol semantics, and structural relationships while providing computational interfaces suitable for numerical optimization algorithms.

Limitations and Deployment Constraints: TrafficManipulator operates as an offline attack framework, requiring traffic replay mechanisms to achieve actual attack implementation. This limitation restricts its applicability to scenarios where traffic can be captured, processed, modified, and replayed without immediate detection. Real-time attack scenarios require different approaches that can modify traffic streams in flight.

The framework’s effectiveness against deep payload inspection systems remains limited unless combined with complementary techniques such as encryption or payload obfuscation. Additionally, the PSO optimization process can require substantial computational resources and time, potentially limiting scalability for large-scale attack scenarios.

3.3.5 Timing-Based and Temporal Manipulation Frameworks

TANTRA: Temporal Adversarial Network Traffic Reshaping

TANTRA addresses the specific vulnerability of temporal-based detection systems through sophisticated timing manipulation techniques [24]. The framework represents a focused approach to adversarial evasion that exploits the heavy reliance of many modern NIDS on temporal patterns and statistical timing characteristics [24].

LSTM-Based Temporal Modeling: TANTRA employs Long Short-Term Memory (LSTM) networks to learn complex temporal patterns characteristic of benign network communications. The LSTM architecture captures long-range dependencies and subtle timing correlations that simpler statistical approaches might miss. The learned temporal models serve as generators for realistic timing patterns that can be applied to malicious traffic for evasion purposes.

The temporal modeling process involves training on extensive corpora of benign traffic to capture the full diversity of legitimate timing patterns across different applications, protocols, and network conditions. The resulting models can generate timing sequences that exhibit the statistical and structural characteristics of normal communications while accommodating the functional requirements of ongoing attacks.

Traffic Stream Reshaping: The core attack mechanism involves real-time or near-real-time modification of packet arrival times to match learned benign timing distributions. TANTRA can adjust inter-arrival times, introduce or remove bursts, and modify timing correlations to align malicious traffic with normal patterns. This process requires sophisticated scheduling algorithms that can implement timing modifications while preserving attack functionality and avoiding excessive delays that might trigger timeout mechanisms.

The reshaping process must balance multiple competing requirements including evasion effectiveness, attack preservation, and implementation feasibility under realistic network conditions. The framework incorporates optimization algorithms that explore the space of possible timing modifications to identify configurations that maximize evasion probability while satisfying operational constraints.

Real-Time Implementation Challenges: TANTRA faces significant challenges in real-time deployment scenarios where timing modifications must be implemented during active network communications. The framework requires precise control over packet transmission timing, which may be difficult to achieve in complex network environments with variable latency and routing conditions.

The effectiveness of temporal reshaping depends critically on the target detection system's sensitivity to timing patterns and the accuracy of learned benign models. Detection systems that employ multiple feature types or robust temporal analysis may be less susceptible to timing-only attacks. Additionally, excessive timing modifications may weaken attack effectiveness by introducing delays that affect exploit timing or create detectable anomalies in application-layer behavior.

3.3.6 GAN-Based Defense and Hardening Frameworks

GADoT: GAN-based Adversarial Training for DDoS Detection

GADoT represents a sophisticated defensive framework that employs generative adversarial techniques to enhance the robustness of DDoS detection systems [27]. The framework addresses the challenge of creating effective adversarial training data while avoiding the feedback loops that can weaken adversarial training effectiveness [27].

WGAN-GP Architecture for Traffic Generation: GADoT employs Wasserstein GANs with Gradient Penalty (WGAN-GP) specifically trained on benign traffic to generate "fake-benign" samples for adversarial training purposes. The WGAN-GP architecture provides improved training stability compared to traditional GANs and generates more realistic synthetic traffic samples. The framework trains exclusively on benign traffic to create generators that can produce realistic normal traffic patterns without exposure to malicious examples.

The generated fake-benign samples serve multiple purposes in the adversarial training process. They can replace specific features in malicious samples to create hybrid instances that retain attack characteristics while exhibiting benign statistical properties in selected dimensions. Alternatively, they can be injected as dummy packets into attack flows to skew aggregate statistics and potentially evade detection.

Decoupled Generation and Training: A critical architectural decision in GADoT involves decoupling the generation process from the target detection model. Unlike frameworks where the same model used for detection also influences adversarial generation, GADoT employs independent generators that prevent feedback effects that could weaken adversarial training effectiveness. This separation ensures that generated adversarial examples maintain their challenging characteristics throughout the training process.

The decoupled architecture also enables transfer learning where generators trained on one dataset can provide adversarial examples for training detection models on different datasets, potentially improving generalization and robustness across diverse deployment environments.

Multi-Feature Perturbation Strategy: GADoT implements sophisticated perturbation strategies that operate across multiple feature dimensions simultaneously. Rather than focusing on isolated feature modifications, the framework can coordinate changes across temporal, statistical, and structural characteristics to create comprehensive evasion attempts that challenge detection systems across multiple vulnerability dimensions.

The framework incorporates domain knowledge about DDoS attack characteristics and network protocols to ensure that perturbations maintain attack effectiveness while achieving evasion goals. This includes preserving the essential volumetric or protocol characteristics that enable DDoS attacks while modifying observable features used for detection.

Scope Limitations and Computational Requirements: GADoT's current scope focuses primarily on volumetric DDoS attacks such as SYN floods and HTTP GET floods, with limited adaptation to more sophisticated attack patterns such as low-rate DDoS attacks or application-layer attacks with complex behavioral characteristics. Extending the framework to broader attack taxonomies requires additional research and development effort.

The GAN-based generation process demands substantial computational resources for training and may require specialized hardware for efficient operation. The training process can be time-consuming and may require extensive datasets representative of target deployment environments to achieve effective generation capability.

3.3.7 Federated and Distributed Framework Architectures

FedNIDS: Federated Learning for Collaborative Detection

FedNIDS addresses the emerging challenges of distributed network security in environments where multiple organizations or network segments must collaborate for threat detection while maintaining data privacy and autonomy [50]. The framework represents an important evolution toward realistic deployment scenarios for modern network security systems [50].

Two-Stage Federated Architecture: FedNIDS implements a sophisticated two-stage federated learning architecture designed to handle the heterogeneous and non-independently distributed nature of real-world network data. The first stage involves collaborative training of a global Deep Neural Network (DNN) across distributed clients using FedProx optimization to address data distribution heterogeneity. The FedProx algorithm incorporates proximal terms that prevent client updates from diverging excessively from global models, improving convergence stability in heterogeneous environments.

The second stage implements rapid adaptation mechanisms where the global model can be fine-tuned using local data from specific clients that encounter novel threats or adversarial examples. This

approach enables rapid threat intelligence dissemination without requiring centralized data sharing or extensive retraining procedures.

Privacy-Preserving Threat Intelligence Sharing: A critical component of FedNIDS involves mechanisms for sharing threat intelligence across federated participants while preserving data privacy and organizational autonomy. The framework enables clients to share model updates and limited threat indicators without exposing sensitive network data or internal infrastructure characteristics.

The privacy preservation mechanisms include differential privacy techniques, secure aggregation protocols, and selective sharing mechanisms that allow organizations to control the level of information sharing based on organizational policies and security requirements. These mechanisms enable collaborative learning while maintaining compliance with data protection regulations and organizational security policies.

Adversarial Considerations in Federated Environments: Federated deployment scenarios introduce unique adversarial challenges including model poisoning attacks where malicious participants attempt to corrupt the global model through carefully crafted local updates. FedNIDS incorporates robustness mechanisms including Byzantine-resilient aggregation algorithms and anomaly detection techniques to identify and mitigate potential poisoning attempts.

The distributed nature of federated systems also creates opportunities for adversaries to exploit inconsistencies between different client models or to craft attacks that evade detection at individual clients while remaining detectable at the global level. The framework addresses these challenges through ensemble approaches and consistency checking mechanisms.

Scalability and Deployment Challenges: FedNIDS faces significant challenges in real-world deployment including communication overhead, synchronization requirements, and heterogeneity management across diverse network environments. The framework requires robust communication protocols that can handle intermittent connectivity, varying computational capabilities, and diverse data characteristics across federated participants.

The effectiveness of federated learning depends critically on having sufficient diversity and quality in participant datasets. Environments with limited participation or poor data quality may achieve suboptimal performance compared to centralized alternatives.

3.3.8 Comparative Framework Analysis

The diversity of frameworks reviewed reveals fundamental differences in approach, scope, and limitations that reflect the complex challenges of adversarial machine learning in network security contexts. Table 3.1 provides a systematic comparison of key characteristics across major frameworks.

Table 3.1: Comprehensive Framework Comparison

Framework	Primary Focus	Approach	Deployment	Key Limitation
TIKI-TAKA	Temporal robustness	Query-based + DSE	Real-time	Query detection gaming
RePO/RePO+	Over-generalization	Masked autoencoders	High compute	Inference overhead
A2PM	Pattern preservation	Statistical learning	Offline	Novel attack blindness
ProGen	Distribution mimicry	Traffic-space GAN	Offline	Transfer limitations
TrafficManipulator	Bi-level optimization	GAN + PSO	Offline	Real-time constraints
TANTRA	Temporal evasion	LSTM reshaping	Near real-time	Timing dependency
GADoT	Adversarial training	WGAN-GP defense	Training-time	DDoS scope limitation
FedNIDS	Distributed learning	Federated approach	Distributed	Communication overhead

3.3.9 Framework Integration and Future Directions

The analysis reveals that current frameworks address different aspects of the adversarial NIDS challenge but lack comprehensive integration. Most frameworks focus on specific vulnerability classes or attack vectors, creating opportunities for sophisticated adversaries to exploit gaps between different defensive approaches.

Future framework development must address integration challenges including unified evaluation metrics, cross-framework compatibility, and comprehensive threat modeling that accounts for multi-vector attacks. The development of standardized interfaces and evaluation protocols could enable framework composition and layered defense strategies.

Additionally, the field requires frameworks that can operate effectively in dynamic environments where threat landscapes evolve rapidly. Current frameworks often assume static threat models and may struggle to adapt to novel attack patterns or changing network characteristics in operational deployments.

3.4 Evaluation Practices and Dataset Limitations

Current evaluation practices in adversarial NIDS research reveal several systematic limitations that impact the validity and generalizability of research findings. These limitations span dataset characteristics, evaluation metrics, and experimental design choices.

3.4.1 Static Benchmark Dataset Challenges

Public datasets commonly used for NIDS evaluation, such as NSL-KDD and CICIDS2017, suffer from aging and limited representativeness of modern network traffic patterns. These datasets often capture network conditions from controlled laboratory environments that fail to reflect the complexity and diversity of contemporary operational networks. High accuracy rates reported on these datasets may represent artifacts of overfitting to specific, low-complexity network conditions rather than genuine detection capability.

The static nature of benchmark datasets presents particular challenges for adversarial evaluation. Real network environments exhibit dynamic characteristics, evolving attack patterns, and diverse operational contexts that static datasets cannot capture. This limitation undermines the external validity of adversarial robustness assessments performed exclusively on benchmark data.

3.4.2 Transferability and Generalization Issues

Adversarial examples often exhibit limited transferability between different model architectures and training datasets. Attacks generated for deep learning models frequently fail to fool traditional machine learning approaches like Random Forests, and vice versa. This limitation complicates practical deployment of both attacks and defenses, as operational environments typically employ diverse detection technologies.

The transferability problem extends beyond model architecture to include differences in feature extraction, preprocessing pipelines, and deployment configurations. Adversarial examples crafted for one operational context may fail entirely when applied to slightly different network environments or NIDS implementations.

3.4.3 Real-World Validation Gaps

Most adversarial NIDS evaluations occur purely at the data level, lacking end-to-end validation through actual network attack scenarios. The failure to design comprehensive attack scenarios that span from traffic generation through network transmission to detection means that the practical threat posed by many mathematically successful adversarial examples remains unproven.

This validation gap represents a fundamental limitation in current research methodologies. Theoretical demonstrations of adversarial effectiveness do not necessarily translate to practical attack capabilities, particularly given the complex interactions between network infrastructure, protocol implementations, and detection systems in operational environments.

3.5 Research Gaps and Future Directions

Analysis of the current literature reveals several significant research gaps that represent opportunities for advancing the field of adversarial machine learning in network security contexts.

3.5.1 Protocol-Aware Adversarial Generation

A critical gap exists in the integration of formal network protocol rules into adversarial generation algorithms. Current approaches often treat protocol compliance as post-hoc constraints rather than

fundamental optimization considerations. Future research must develop methodologies that incorporate protocol semantics directly into adversarial optimization objectives, ensuring that gradient-based search procedures naturally produce valid traffic patterns.

The challenge extends beyond basic protocol compliance to encompass complex protocol interactions, state management, and temporal dependencies. Advanced protocols exhibit intricate behavioral patterns that must be preserved for both functionality and stealthiness, requiring sophisticated modeling approaches that current adversarial generation methods lack.

3.5.2 Fragmentation-Aware Robustness

Most anomaly-based NIDS suffer from over-generalization when confronted with fragmented or partial traffic observations. This vulnerability represents a significant research gap, as real networks routinely experience packet fragmentation due to Maximum Transmission Unit (MTU) limitations and routing considerations. Current detection systems typically assume complete flow visibility, making them vulnerable to attacks that exploit fragmentation-based obfuscation.

Research into fragmentation-aware robustness must address both defensive hardening and attack generation perspectives. Defensive approaches require development of detection algorithms that can reliably identify threats from partial observations, while attack methodologies must incorporate fragmentation as a systematic evasion strategy rather than a simple perturbation technique.

3.5.3 Dynamic and Stateful Detection

Existing NIDS architectures generally treat network interactions as independent events, lacking consideration for long-term behavioral patterns or adversarial probing attempts. This limitation creates opportunities for sophisticated adversaries to perform gradual reconnaissance and boundary exploration that individual detection attempts might miss.

Future research must develop stateful detection mechanisms capable of identifying and responding to complex attack campaigns that unfold over extended timeframes. These approaches require maintaining and analyzing historical interaction patterns while efficiently managing computational and storage overhead in high-throughput network environments.

3.5.4 Standardized Robustness Metrics

The absence of standardized metrics for adversarial robustness assessment hampers comparative evaluation and reproducible research. Current studies employ diverse evaluation criteria, experimental setups, and performance measures that limit the ability to systematically compare different approaches or track field-wide progress.

Development of standardized robustness metrics must account for the multi-dimensional nature of adversarial threats in network security contexts, including attack sophistication, implementation realism, detection evasion rates, and operational impact. These metrics should enable meaningful comparison across different detection approaches while capturing the practical considerations relevant to operational deployment.

3.6 Critical Analysis and Synthesis

The current state of adversarial machine learning research in network intrusion detection reveals a fundamental tension between mathematical rigor and practical implementability. While the field has achieved significant theoretical advances in understanding and generating adversarial examples, substantial gaps remain in translating these insights into operationally relevant security improvements.

3.6.1 Realism versus Optimization Efficiency

A central challenge emerges from the inherent trade-off between optimization efficiency and realistic constraint satisfaction. Feature-space attacks achieve high mathematical success rates and efficient optimization convergence but produce traffic patterns that violate basic protocol rules and cannot be implemented on real networks. Conversely, protocol-aware approaches generate implementable traffic but sacrifice optimization efficiency and often achieve lower evasion rates against sophisticated detection systems.

This trade-off reflects deeper conceptual challenges in adapting adversarial machine learning method-

ologies, originally developed for continuous domains like computer vision, to discrete, highly constrained domains like network protocols. Future research must develop novel optimization approaches that can efficiently navigate discrete constraint spaces while maintaining the theoretical guarantees and performance characteristics of gradient-based methods.

3.6.2 Practical Deployability Considerations

Many proposed defense mechanisms suffer from computational complexity or operational requirements that limit their practical deployability in high-throughput network environments. Sophisticated denoising approaches, ensemble methods, and adversarial training techniques often require computational resources that exceed typical operational constraints or introduce latency that disrupts real-time detection requirements.

The deployment challenge extends beyond computational considerations to include integration complexity, maintenance overhead, and adaptation to evolving threat landscapes. Practical NIDS deployments require defense mechanisms that can operate effectively with minimal configuration, adapt to changing network conditions, and maintain performance across diverse operational environments.

Research must prioritize development of defense approaches that balance robustness improvements with operational practicality. This requires careful consideration of deployment contexts, resource constraints, and integration challenges from the initial design phases rather than treating these as secondary optimization criteria.

3.6.3 Robustness versus Generalization

Adversarial training and other robustness-enhancing techniques often improve resistance to known attack patterns while potentially reducing generalization capability against novel or unforeseen threats. This fundamental tension reflects the broader challenge of balancing specificity and adaptability in machine learning security applications.

The challenge is particularly acute in network security contexts where threat landscapes evolve rapidly and attack methodologies continually advance. Defense systems must maintain effectiveness against both current adversarial techniques and future attack innovations, requiring research approaches that emphasize general robustness principles over specific attack countermeasures.

Future research directions must address this tension through development of theoretical frameworks that predict robustness transfer across different threat models and practical validation methodologies that assess defense effectiveness against evolving attack landscapes.

3.7 Conclusion

The state of the art in adversarial machine learning for network intrusion detection reveals a rapidly evolving field characterized by sophisticated framework development that addresses the fundamental challenges of translating theoretical adversarial concepts into practical network security applications. The comprehensive analysis of contemporary frameworks demonstrates significant progress in bridging the gap between feature-space mathematical optimization and protocol-aware traffic manipulation, yet substantial challenges remain in achieving robust, deployable solutions for operational environments.

3.7.1 Framework Maturation and Specialization

The evolution from general-purpose adversarial libraries to specialized network frameworks represents a crucial maturation of the field. The detailed examination of frameworks such as TIKI-TAKA, RePO, A2PM, ProGen, TrafficManipulator, TANTRA, GADoT, and FedNIDS reveals distinct approaches to different aspects of the adversarial NIDS challenge. Each framework addresses specific vulnerability classes or operational constraints, reflecting the recognition that no single approach can comprehensively address the full spectrum of adversarial threats against network intrusion detection systems.

TIKI-TAKA’s temporal focus and query detection mechanisms demonstrate sophisticated understanding of real-world attack scenarios, while RePO’s architectural innovation addresses fundamental over-generalization problems through masked autoencoder approaches. The pattern-based learning of A2PM and the distribution projection paradigm of ProGen represent complementary approaches to maintaining realism in adversarial generation. TrafficManipulator’s bi-level optimization and TANTRA’s timing-based reshaping illustrate the complexity required to bridge optimization ef-

iciency with practical implementability.

The defensive frameworks, particularly GADoT’s decoupled adversarial training and FedNIDS’s privacy-preserving collaborative learning, demonstrate important advances in creating robust detection systems that can operate effectively in distributed, heterogeneous environments while maintaining practical deployment constraints.

3.7.2 Persistent Implementation Challenges

Despite significant theoretical advances, the framework analysis reveals persistent challenges in translating sophisticated adversarial techniques into operationally viable security improvements. The fundamental tension between optimization efficiency and realistic constraint satisfaction appears across multiple frameworks, suggesting that this represents a core challenge requiring novel theoretical developments rather than incremental engineering improvements.

Computational complexity emerges as a critical limiting factor across frameworks. RePO’s 100-fold inference overhead, GADoT’s GAN training requirements, and TrafficManipulator’s PSO optimization demands illustrate the computational costs associated with sophisticated adversarial robustness. These requirements often exceed practical deployment constraints in high-throughput network environments where real-time detection is essential.

The scope limitations evident in many frameworks reflect the difficulty of developing comprehensive solutions that address the full spectrum of adversarial threats. Most frameworks excel within specific domains—temporal attacks, DDoS detection, federated scenarios—but struggle with broader applicability. This specialization, while enabling deep technical solutions, creates vulnerabilities where sophisticated adversaries might exploit gaps between framework capabilities.

3.7.3 Critical Integration Gaps

The framework analysis reveals significant integration gaps that represent both current limitations and future research opportunities. Most frameworks operate in isolation, addressing specific vulnerability classes without consideration for interactions with other defensive mechanisms or attack vectors. The absence of standardized interfaces, evaluation protocols, and interoperability mechanisms limits the development of comprehensive, layered defense strategies.

Cross-framework compatibility represents a particularly critical gap. Organizations deploying network security systems typically employ multiple detection technologies, yet current frameworks lack mechanisms for coordinated operation or information sharing between different adversarial defense approaches. This fragmentation creates opportunities for sophisticated attacks that exploit inconsistencies between different defensive systems.

The evaluation gap Between offline framework testing and operational deployment validation represents another critical limitation. While frameworks demonstrate impressive capabilities under controlled laboratory conditions, limited validation in live network environments undermines confidence in their operational effectiveness.

3.7.4 Future Research Imperatives

The comprehensive framework analysis identifies several critical research directions that must be addressed to advance the field toward practical deployment readiness. Protocol-aware adversarial generation remains a fundamental challenge requiring integration of formal network protocol specifications with optimization algorithms. Current approaches treat protocol compliance as post-hoc constraints rather than fundamental optimization considerations, limiting their effectiveness and realism.

Standardization emerges as a critical need across multiple dimensions. The development of standard evaluation metrics, benchmark datasets, and interoperability protocols would enable systematic comparison of framework capabilities and facilitate integration of complementary approaches. The absence of these standards currently hampers reproducible research and practical deployment assessment.

Real-time adaptability represents another crucial research direction. Current frameworks typically assume static threat models and deployment environments, yet operational networks exhibit dynamic characteristics requiring adaptive defensive mechanisms. Future frameworks must incorporate learning and adaptation capabilities that can respond to evolving threat landscapes while maintaining

performance under operational constraints.

The development of unified theoretical frameworks that can predict and explain adversarial robustness across different attack vectors and defensive mechanisms represents a fundamental research challenge. Current approaches often rely on empirical evaluation without strong theoretical foundations for predicting performance in novel scenarios or transfer across deployment environments.

3.7.5 Strategic Research Priorities

Based on the comprehensive framework analysis, several strategic research priorities emerge for advancing the field toward operational relevance. First, the development of hybrid frameworks that combine the strengths of different approaches while mitigating individual limitations requires sustained attention. The integration of temporal manipulation capabilities from TANTRA with the robustness mechanisms of RePO, for example, could create more comprehensive defensive systems.

Second, computational efficiency must become a primary design consideration rather than a secondary optimization target. Future frameworks must achieve adversarial robustness within computational budgets compatible with high-throughput network environments. This requires fundamental algorithmic innovations rather than incremental improvements to existing approaches.

Third, comprehensive evaluation methodologies that span from mathematical robustness to operational effectiveness must be developed. Current evaluation practices focus primarily on mathematical metrics without adequate consideration of practical deployment challenges, integration complexity, or adaptation to evolving threat landscapes.

Fourth, the development of framework composition architectures that enable coordinated operation of multiple specialized systems represents a critical research opportunity. Rather than pursuing single comprehensive solutions, the field might benefit from modular approaches that enable tailored composition of capabilities based on specific deployment requirements.

3.7.6 Implications for Network Security Practice

The framework analysis has important implications for network security practitioners and researchers. The sophistication of contemporary adversarial techniques demands corresponding sophistication in defensive frameworks, yet practical deployment constraints limit the applicability of the most advanced approaches. This creates a critical need for frameworks that balance theoretical rigor with operational practicality.

The specialization evident in current frameworks suggests that operational network security requires multiple complementary approaches rather than single comprehensive solutions. Security architects must consider the integration challenges and compatibility requirements associated with deploying multiple specialized frameworks in coordinated defensive strategies.

The computational and complexity costs associated with advanced adversarial robustness must be carefully balanced against security benefits and operational requirements. Organizations must develop risk-based approaches to determining appropriate levels of adversarial robustness based on threat models, asset criticality, and available resources.

Finally, the rapid evolution of adversarial techniques and defensive frameworks requires sustained investment in research and development capabilities. Organizations relying on network intrusion detection systems must maintain awareness of emerging threats and defensive capabilities to ensure continued effectiveness against sophisticated adversaries.

The state of the art in adversarial machine learning for network intrusion detection demonstrates significant progress in developing sophisticated frameworks for addressing complex security challenges, yet substantial work remains in creating comprehensive, deployable solutions for operational environments. The framework-centric analysis reveals both the achievements and limitations of current approaches while identifying critical research directions for advancing the field toward practical security improvements. Success in addressing these challenges will require continued interdisciplinary collaboration, sustained theoretical development, and careful attention to the operational requirements of modern network security environments.

4 Methodology

This chapter presents the methodological framework developed to generate functionality-preserving adversarial traffic capable of evading modern NIDS. The proposed approach extends an existing GAN-PSO-based adversarial traffic manipulator by introducing **FRANTIC** (FRAgmeNTation adversarial Component), a stochastic structural transformation layer operating at the IPv4 fragmentation level.

4.1 Motivations

The original manipulator, as discussed in Section 2.6.2, represents a foundational approach to adversarial traffic generation in raw packet space. By combining GAN-guided boundary exploration with PSO-based mutation operators, it enables realistic evasion of ML-based NIDS while maintaining protocol validity and attack semantics. However, the analysis in Section ?? revealed a fundamental limitation: the framework operates predominantly in feature space, manipulating header-level and temporal characteristics (e.g., inter-arrival times), while leaving the structural aspect of traffic largely untouched.

In particular, transformations at the IP layer, such as packet fragmentation, are not considered. Yet fragmentation significantly influences packet-level and flow-level statistical descriptors, burst structures, and fixed-size tensor representations commonly exploited by modern ML-NIDS. Despite its historical relevance in intrusion evasion, this structural dimension remains absent from the GAN-PSO design, thereby leaving a substantial portion of the adversarial search space unexplored.

To address this gap, FRANTIC is introduced as a post-manipulation transformation stage. Rather than replacing the existing optimization strategy, it augments it by enabling adversarial exploration along two complementary axes:

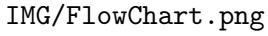
- **Temporal/Statistical Axis:** Optimization of inter-arrival times and payload-level perturbations via GAN-guided PSO.
- **Structural Axis:** Stochastic IPv4-layer fragmentation to alter packet size distributions, flow geometry, and structural entropy.

By jointly operating across these axes, the resulting framework enables wider adversarial exploration in feature space. This significantly broadens the reachable adversarial manifold while preserving strict protocol correctness and end-to-end attack functionality. The following sections detail the architectural design, formalization, and implementation of the proposed fragmentation-aware manipulator.

4.2 System Architecture

The proposed adversarial generation pipeline operates in two distinct and sequential phases: (i) a feature-space manipulation phase, and (ii) a structural fragmentation phase.

As illustrated in Figure 4.1, the original CIC-IDS2019 dataset is separated into benign and malicious traffic. Malicious flows undergo traffic manipulation followed by stochastic IP fragmentation, while benign traffic remains unchanged. The processed malicious samples are then recombined with benign traffic to construct the final adversarial dataset.



IMG/FlowChart.png

Figure 4.1: Overview of the adversarial dataset generation pipeline.

4.2.1 Phase 1: Feature-Space Optimization (GAN + PSO)

This phase implements the original manipulator logic:

- A GAN is trained to approximate the benign feature distribution.
- A PSO algorithm iteratively perturbs malicious flows, primarily via temporal adjustments and payload-level modifications, minimizing detection confidence.

The output consists of optimized malicious PCAPs that remain structurally unchanged at the IP layer but exhibit adversarial statistical properties.

4.2.2 Phase 2: Stochastic Fragmentation

The fragmentation stage applies probabilistic, controlled, RFC-compliant IPv4 fragmentation as a post-processing transformation. By decomposing packets into randomized yet protocol-valid fragments, the system induces measurable shifts in packet- and flow-level statistical features while preserving end-to-end attack functionality.

This structural perturbation challenges flow-based ML-NIDS that rely on packet size distributions, burst characteristics, and fixed-length tensor representations, thereby expanding the adversarial traffic space without compromising protocol correctness.

4.3 FRANTIC

The fragmentation module addresses the structural rigidity of adversarial samples generated by purely feature-based manipulation.

FRANTIC operates at the IPv4 layer on packets generated by the manipulation stage. Fragmentation is applied selectively and under strict protocol constraints to ensure semantic transparency and transport-layer stability.

Network traffic features are abstractions derived from packet streams. By altering stream structure via fragmentation, we effectively perturb the mapping:

$$\text{Traffic Space} \longrightarrow \text{Feature Space}.$$

4.3.1 Design Principles

The module is governed by four core design principles:

1. **Strict RFC Compliance:** All fragments conform to RFC 791, including 8-byte alignment, correct flag handling, and consistent IP Identification values.
2. **Configurability:** Parameters controlling fragmentation probability and fragment size bounds enable systematic experimentation.
3. **Semantic Transparency:** Fragmentation must remain invisible to transport and application layers; reassembly at the destination must fully restore the malicious payload.
4. **Stochastic Randomization:** Randomized selection and fragment sizes prevent deterministic evasion signatures.

4.3.2 Architecture

FRANTIC is divided into two logical layers: the Orchestration Layer and the Reconstruction Layer.

The first phase is implemented in `manipulator.py`. This component iterates over the packet stream, maintains global configuration parameters, and determines packet eligibility for subsequent processing. The complete source code is provided in Appendix B.1.

The configuration parameters include:

- ρ (`fragment_prob`): Fragmentation probability, $0 \leq \rho \leq 1$.
- S_{min} : Minimum fragment size (default: 64 bytes).
- S_{max} : Maximum fragment size (default: 500 bytes).

The reconstruction layer is implemented in `rebuilder.py` (Appendix B.2). This component performs byte-level payload decomposition and header reconstruction, ensuring syntactic correctness, accurate fragment offset computation, preservation of protocol metadata, and full reassembly consistency at the destination.

4.3.3 Algorithmic Formulation

The procedure operates on a packet stream P and produces a transformed stream P' in which eligible IPv4 packets may be decomposed into multiple protocol-valid fragments while preserving semantic correctness.

Let the original packet stream be:

$$P = \{p_1, p_2, \dots, p_n\}.$$

After fragmentation, the resulting stream becomes:

$$P' = \{f_{1,1}, f_{1,2}, \dots, f_{n,k}\}.$$

where each packet p_i may correspond to one or multiple fragments $f_{i,j}$.

Packet eligibility requires:

$$L_{payload} > S_{min} \quad \text{and} \quad r \sim \mathcal{U}[0, 1], \quad r \leq \rho.$$

Only IPv4 packets whose payload exceeds a configurable minimum threshold are considered. Small control packets, such as ACK-only TCP segments, are excluded in order to preserve connection stability and avoid unintended side effects on transport-layer behavior. Let r be a realization of a continuous uniform random variable $r \sim \mathcal{U}(0, 1)$. Fragmentation is applied if $r \leq \rho$, thereby implementing Bernoulli selection with parameter ρ . Only packets satisfying all conditions enter the fragmentation routine.

For eligible packets, fragmentation is applied probabilistically with $P_{frag} \in [0, 1]$, iteratively over the remaining payload length L_{rem} . This stochastic selection prevents deterministic transformation patterns and introduces variability across executions, thereby increasing structural entropy within the traffic stream.

When a packet is selected, fragment sizes are sampled from a bounded interval $[S_{min}, S_{max}]$. Intermediate fragments are aligned to 8-byte boundaries to comply with IPv4 offset encoding requirements, while the final fragment absorbs the remaining payload.

Intermediate fragment size is sampled as:

$$s_{frag} \sim \mathcal{U}(S_{min}, s_{limit}), \quad s_{limit} = \min(S_{max}, L_{rem} - S_{min}).$$

This ensures that a valid final fragment of at least S_{min} bytes can still be generated.

For non-final fragments:

$$s_{frag} \leftarrow \left\lfloor \frac{s_{frag}}{8} \right\rfloor \times 8.$$

To formalize the structural transformation process, Algorithm 1 presents the stochastic RFC-compliant fragmentation procedure. The algorithm abstracts from implementation-specific details and describes the high-level control flow governing packet eligibility, probabilistic selection, fragment size generation, and RFC-compliant header construction.

During reconstruction, each fragment inherits routing-relevant metadata from the original packet. The Identification field is consistently assigned across fragments derived from the same parent packet, the Fragment Offset is computed based on cumulative payload length, and the More Fragments flag is set appropriately. Header checksums are recalculated to ensure strict RFC 791 compliance.

To maintain temporal realism, the first fragment retains the original timestamp, while subsequent fragments are assigned minimal incremental delays that emulate realistic transmission bursts without significantly altering overall flow timing characteristics.

4.4 Protocol Compliance Constraints

Strict adherence to IPv4 fragmentation rules is required to ensure that generated traffic traverses intermediate routers, firewalls, and operating system stacks without being discarded.

Algorithm 1 Stochastic RFC-Compliant IP Fragmentation

Require: Packet stream P , probability ρ , S_{min} , S_{max}

Ensure: Fragmented packet stream P'

```
1:  $P' \leftarrow \emptyset$ 
2: for each packet  $p \in P$  do
3:   if  $p$  is IPv4 and  $\text{payload}(p) > S_{min}$  and  $\text{rand}(0, 1) \leq \rho$  then
4:      $ID \leftarrow \text{random 16-bit value}$ 
5:      $offset \leftarrow 0$ 
6:      $L \leftarrow \text{payload length}$ 
7:     while  $offset < L$  do
8:       Determine  $s_{frag}$ 
9:       Align to 8-byte boundary (if non-final)
10:      Set  $IP.id \leftarrow ID$ 
11:      Set  $IP.frag \leftarrow offset/8$ 
12:      Set MF flag appropriately
13:      Append fragment to  $P'$ 
14:       $offset \leftarrow offset + s_{frag}$ 
15:    end while
16:   else
17:     Append  $p$  to  $P'$ 
18:   end if
19: end for
20: return  $P'$ 
```

8-Byte Alignment

The IPv4 **Fragment Offset** field encodes offsets in units of 8 bytes. Consequently, all fragment payloads, except the final one, must be multiples of 8 bytes.

To ensure this, the algorithm enforces:

$$s_{frag} \leftarrow \left\lfloor \frac{s_{frag}}{8} \right\rfloor \times 8$$

for intermediate fragments.

Header Field Management

For each fragment f_j derived from parent packet p :

- **Identification (IP.id):** A unique 16-bit identifier

$$ID \sim \mathcal{U}[1, 65535]$$

is generated per parent packet and assigned to all its fragments.

- **Flags:** The MF (More Fragments) bit is set to 1 for all but the last fragment.
- **Fragment Offset:** Computed as cumulative payload bytes divided by 8:

$$\text{frag} = \frac{\text{payload_offset}}{8}.$$

- **Metadata Preservation:** Fields such as source address, destination address, protocol identifier, TTL, and TOS are copied unchanged.

4.4.1 Stochastic Properties and Adversarial Implications

The effectiveness of this mechanism derives from its non-deterministic nature. Identical executions on the same input trace produce distinct traffic realizations.

Three primary entropy sources are introduced:

1. **Selection Entropy:** A packet may be fragmented in one execution but not in another, governed by ρ .
2. **Structural Entropy:** The number and sizes of fragments vary due to uniform random sampling.
3. **Identifier Entropy:** Randomized IP identification values modify header checksums and signature-level patterns.

These stochastic variations induce measurable distribution shifts in features such as packet count per flow, mean and variance of packet size, burst density and fixed-length tensor zero-padding ratios.

As a result, machine learning-based NIDS must generalize to a substantially expanded traffic manifold. Models overfitting to non-fragmented statistical patterns are particularly susceptible to misclassification under this transformation.

4.4.2 Impact on NIDS Evasion

The introduction of controlled fragmentation enhances evasion effectiveness by inducing structural distribution shift while preserving semantic equivalence at higher protocol layers.

At the packet and flow level, fragmentation modifies size distributions, increases packet counts per flow, and alters burst density patterns. These transformations directly affect statistical descriptors commonly exploited by flow-based ML-NIDS, including mean packet size, variance, and inter-arrival dynamics.

In addition, fragmentation perturbs feature extraction pipelines that rely on fixed-length windows or tensor-based representations. By increasing packet granularity and redistributing payload across multiple fragments, the method modifies zero-padding ratios, temporal ordering patterns, and aggregate flow statistics. Such perturbations can shift samples beyond previously learned decision boundaries without altering the underlying malicious functionality.

As a result, the adversarial manipulator operates not only in feature space but also in structural space. This dual-layer transformation expands the reachable adversarial manifold and exposes overfitting tendencies in ML-based NIDS models, thereby supporting more robust adversarial training and systematic evaluation.

4.5 Preprocessing Pipeline

A robust preprocessing pipeline is essential for preparing network traffic data for adversarial manipulation and subsequent evaluation. In this thesis, preprocessing is performed using the `lucid_dataset_parser.py` script from the Flatten Lucid framework, which transforms raw packet capture (PCAP) files into structured datasets suitable for machine learning and adversarial analysis.

Array Dimensions: For the flattened model, each flow is represented as a 1D array of 21 features. For matrix input, each flow is a 2D array of shape $(\text{max_flow_len}, \text{num_features})$, e.g., $(100, 21)$ for 100 packets per flow and 21 features per packet. The HDF5 files store train/val/test datasets as arrays of shape $(\text{num_flows}, \text{max_flow_len}, \text{num_features})$ for matrix input, or $(\text{num_flows}, 21)$ for flattened input.

The preprocessing flow consists of the following steps:

1. **Feature Extraction:** Each packet is parsed to extract a comprehensive set of features, including timestamps, packet lengths, IP fragmentation flags, protocol identifiers, and transport-layer flags.
2. **Flow Construction:** Packets are grouped into flows based on 5-tuple identifiers and time windows.
3. **Labeling:** Flows are labeled as benign or attack using dataset-specific rules and encoding.
4. **Balancing and Splitting:** The dataset is balanced and split into training, validation, and test sets.

5. **Normalization and Flattening:** Feature vectors are normalized and optionally flattened to 1D arrays.
6. **Export to HDF5:** The processed data is exported as HDF5 files, with separate files for training, validation, and test sets.

Required Input for TrafficManipulator: The TrafficManipulator framework requires as input the HDF5 datasets generated by this preprocessing pipeline. Additionally, a target benign feature set to mimic must be prepared and saved as a `.npy` file. This file should contain feature arrays classified as benign by the targeted system.

Parameter Tuning for TrafficManipulator: The algorithm exposes nine key parameters, divided into three groups:

- **PSO velocity update:** `w` (inertia), `c1` (cognitive force), `c2` (social force)
- **PSO search configuration:** `max_iter` (iterations), `particle_num` (population), `grp_size` (neighborhood size)
- **Manipulator parameters:** `grp_size` (packets mutated per processing), `min_time_extend/max_time_ext` (max interarrival time extension), `max_cft_pkt` (max crafted packets per original), `max_crafted_pkt_prob` (probability of crafting)

Example parameter settings:

```
m.change_particle_params(w=0.6, c1=0.7, c2=1.4)
m.change_pso_params(max_iter=5, particle_num=10, grp_size=5)
m.change_manipulator_params(grp_size=5, min_time_extend=0., max_time_extend=5., max_cft_pkt=
```

This preprocessing pipeline ensures that all subsequent adversarial experiments are conducted on standardized, well-structured datasets, enabling reproducible and meaningful evaluation of evasion techniques.

5 Experimental Evaluation

In this chapter, the efficacy of packet fragmentation as an evasion technique against NIDS is assessed.

To evaluate the individual contribution of packet fragmentation, the detection performance is compared across the following three cases: the original traffic, the traffic manipulated by the original framework, and the traffic manipulated by packet fragmentation in combination with the existing transformations. The results of the study reveal that packet fragmentation indeed contributes to the overall performance degradation. This is expected, as the feature-level manipulation affects the statistical properties of the traffic, while the packet fragmentation affects the actual packet structure, thereby leading to compounded evasion when the two transformations are combined. This further confirms that the performance improvements obtained by the proposed extension are not incremental but, rather, arise from the exploitation of a different vulnerability in the NIDS processing pipeline.

5.1 Experimental Setup

All experiments are based on network traffic data from the CICIDS2019 dataset, which represents a wide range of benign and malicious network traffic.

Three detection models are considered to assess their performance against packet fragmentation, each based on different machine learning paradigms: a Random Forest (RF), a Multi-Layer Perceptron (MLP), and LUCID. The choice of these models enables a comprehensive evaluation.

Detection performance is evaluated using standard classification metrics, including Accuracy, F1-score, and FNR. Given the adversarial setting, particular emphasis is placed on the latter, as it directly quantifies the success of evasion attacks by measuring the proportion of malicious traffic incorrectly classified as benign. All models are evaluated on identical test sets to ensure a fair comparison between baseline traffic, manipulated traffic, and fragmentation-enhanced traffic.

The evaluation involves three primary dataset configurations:

- **Baseline:** Represents the standard performance of the NIDS on the original, unmodified dataset. It serves as the ground truth for evaluating the effectiveness of the attacks.
- **Original Manipulator:** Contains traffic generated using the original traffic manipulation strategy, which employs a combination of GAN and PSO to perturb traffic features.
- **Fragmentation Enhanced:** Contains traffic generated with the improved manipulator that integrates random packet fragmentation after the manipulation.

5.2 Baseline Performance

The baseline evaluation investigates the detection performance of each model under normal traffic conditions, for which no adversarial manipulation is applied. This will provide a standard operational behavior setting and can be used for comparison in further adversarial experiments.

Model	F1 Score	FNR	Accuracy
MLP	0.9987	0.0014	0.9987
RF	0.9999	0.0001	0.9999
LUCID	0.9757	0.0466	0.9763

Table 5.1: Performance on the Baseline Dataset

The results in Table 5.1 point towards very good detection performance, expressed by high accuracy and F1-scores-along with low numbers of false negatives in each model tested. MLP and RF’s

performance is near perfect, indicating that the models perform well at identifying known attack patterns when the distribution of input data is unchanged.

But most of the time, LUCID demonstrates slightly worse absolute values of performance metrics, since its results remain stable and consistent, due to capturing temporal dependencies between packets. This temporal modeling capability is of particular significance in network traffic analysis, whereby sequential patterns form a keystone. Overall, the baseline results confirm that all selected models were well-trained and reliably operating under benign conditions. Importantly, this test sets a very strong point of reference: any subsequent decrease in performance due to the applied adversarial attacks can be attributed to the attack itself rather than a failure in model training or the quality of the dataset.

5.3 Performance Under the Original Manipulator

This section discusses the evaluation of the impact of the original traffic manipulation framework on NIDS performance.

Table 5.2 summarizes the overall performance on the manipulated dataset. A consistent degradation is observed across all models, characterized by increased False Negative Rates (FNR) and reduced F1-scores and accuracy. The elevated FNR values indicate a higher tendency to misclassify malicious traffic as benign, confirming the effectiveness of the manipulation strategy in inducing evasion.

The Random Forest and MLP classifiers exhibit substantial performance deterioration. Since both models rely primarily on aggregated statistical flow features, their decision boundaries are directly influenced by feature-space perturbations. The observed degradation suggests that controlled modifications of high-level descriptors, such as packet counts, byte distributions, and flow timing statistics, can significantly alter the feature representation in ways that reduce separability between benign and malicious samples.

Model	F1 Score	FNR	Accuracy
MLP	0.8949	0.1417	0.8996
RF	0.8849	0.1780	0.8935
LUCID	0.8536	0.2103	0.8650

Table 5.2: Performance on the Manipulated Dataset

LUCID shows the largest overall degradation, with the highest FNR and the lowest F1-score among the evaluated models. Although LUCID incorporates temporal modeling of packet sequences, the results indicate that temporal awareness alone does not guarantee robustness against structured manipulation. The applied transformations appear sufficient to distort both statistical summaries and temporal patterns, thereby impairing the model’s ability to capture discriminative traffic dynamics.

A more granular view is provided in Table 5.3, which compares per-attack FNR values under baseline and manipulated conditions. The impact of the manipulator is highly uneven across attack classes. Several attacks, including WebDDoS, DNS, NTP, SYN, and TFTP, exhibit dramatic increases in FNR across all models. In extreme cases (e.g., SYN and TFTP), FNR values approach or reach 1.0, indicating near-complete evasion. These results suggest that detection of such attacks depends heavily on structural and statistical properties that can be effectively altered through feature-space manipulation.

In contrast, other attack types, such as SSDP, SNMP, and MSSQL, demonstrate comparatively greater resilience. Although some degradation is present, their FNR increases are more moderate and detection performance remains relatively stable. This behavior implies that these attacks possess stronger or more invariant characteristics, potentially including protocol-specific semantics, distinctive traffic signatures, or persistent volumetric patterns that are less sensitive to moderate feature perturbations.

Certain protocol-constrained attacks (e.g., LDAP and PortMap) show partial but not catastrophic degradation. This indicates that while feature-level manipulation can obscure statistical regularities, it may be insufficient to fully mask attacks whose detection implicitly relies on protocol behavior or flow-level structural consistency.

Attack	Random Forest		MLP		Lucid	
	Base	Manip	Base	Manip	Base	Manip
WebDDoS	0.0000	0.4024	0.0000	0.5091	0.5185	0.9091
LDAP	0.0348	0.0912	0.0127	0.0052	0.3165	0.1675
PortMap	0.0000	0.0346	0.0000	0.0020	0.0833	0.1657
DNS	0.0822	0.3494	0.1622	0.1022	0.4595	0.2883
UDPLag	0.0000	0.0554	0.0000	0.0252	0.0000	0.0190
UDP	0.0000	0.0554	0.0000	0.0009	0.0006	0.0005
NTP	0.0000	0.5669	0.0000	0.5256	0.1453	0.5897
SNMP	0.0000	0.0357	0.0000	0.0054	0.0152	0.1597
SSDP	0.0000	0.0955	0.0000	0.0111	0.0023	0.0047
SYN	0.0036	1.0000	0.0000	1.0000	0.0066	1.0000
TFTP	0.0000	0.8543	0.0000	0.8594	0.0000	0.8603
NetBIOS	0.0001	0.0295	0.0030	0.0000	0.1714	0.1562
MSSQL	0.0000	0.0246	0.0001	0.0000	0.0001	0.0000

Table 5.3: False Negative Rate comparison across models under Baseline and Manipulated conditions.

Overall, these findings reveal a fundamental limitation of the original evasion framework. By operating exclusively at the feature level, the manipulator targets post-processed statistical representations rather than earlier stages of the NIDS pipeline, such as packet parsing, state tracking, or flow reconstruction. Consequently, its evasion capability remains constrained and uneven across attack types and model architectures.

5.4 Impact of Fragmentation-Enhanced Attacks

Unlike purely feature-level perturbations, fragmentation introduces structural modifications that directly affect how traffic is processed during packet reassembly and flow construction. This additional evasion layer significantly increases the complexity of the detection task.

Model	F1 Score	FNR	Accuracy
MLP	0.1826	0.8922	0.5551
RF	0.0306	0.9837	0.5237
LUCID	0.1695	0.9005	0.5504

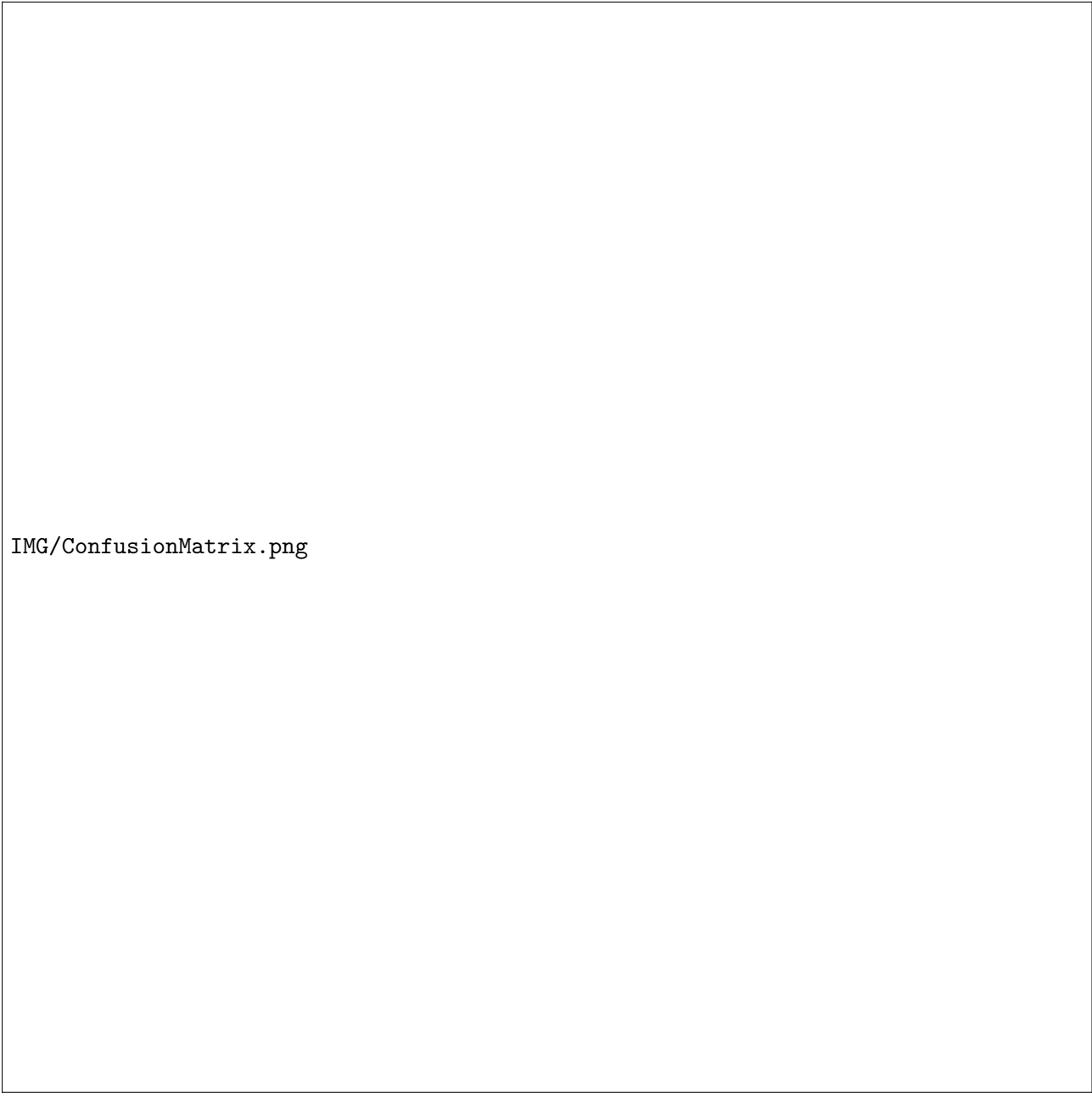
Table 5.4: Performance on the Fragmented Dataset

TODO: da giustificare la differenza, potrebbe servire la confusion matrix:

Table 5.4 summarizes the overall performance on the fragmented dataset. The degradation is substantial across all models. F1-scores collapse to below 0.20 for every architecture, while FNR surge above 0.89 and reach 0.9837 for the Random Forest model. Overall accuracy drops to nearly random-classification levels (approximately 52–56%), indicating that fragmentation severely undermines the discriminative capability of all evaluated detectors.

The fragmentation-enhanced manipulator preserves protocol compliance and attack semantics while subdividing packets into multiple fragments. This transformation alters packet counts, payload distributions, inter-arrival times, and flow lengths, thereby distorting both statistical summaries and temporal structure. Because most NIDS pipelines rely on assumptions about packet continuity and coherent flow construction, fragmentation disrupts intermediate processing stages before classification even occurs.

Compared to the original manipulator, fragmentation consistently produces markedly higher FNR values across nearly all attack categories (Table 5.5). For many attacks, including WebDDoS, NTP, UDPLag, and DNS, the shift from manipulated to fragmented traffic results in dramatic additional increases in FNR. In extreme cases such as SYN and TFTP floods, detection performance effectively collapses across all models, with FNR values approaching one. These results demonstrate that struc-



IMG/ConfusionMatrix.png

Figure 5.1: Confusion matrix

tural perturbations amplify evasion effectiveness far beyond what is achievable through feature-level manipulation alone.

Attack	Random Forest		MLP		Lucid	
	Manip	Frag	Manip	Frag	Manip	Frag
WebDDoS	0.4024	0.6656	0.5091	0.6791	0.9091	0.9933
LDAP	0.0912	0.2809	0.0052	0.2945	0.1675	0.0798
PortMap	0.0346	0.3277	0.0020	0.1404	0.1657	0.1356
DNS	0.3494	0.4309	0.1022	0.3077	0.2883	0.5538
UDPLag	0.0554	0.6557	0.0252	0.6351	0.0190	0.5961
UDP	0.0554	0.6557	0.0009	0.3227	0.0005	0.3386
NTP	0.5669	0.8736	0.5256	0.8996	0.5897	0.9627
SNMP	0.0357	0.3786	0.0054	0.1925	0.1597	0.1616
SSDP	0.0955	0.1976	0.0111	0.0370	0.0047	0.0556
SYN	1.0000	0.9999	1.0000	0.9999	1.0000	0.9999
TFTP	0.8543	0.9988	0.8594	0.9969	0.8603	0.9983
NetBIOS	0.0295	0.3492	0.0000	0.1774	0.1562	0.1669
MSSQL	0.0246	0.3825	0.0000	0.1462	0.0000	0.1085

Table 5.5: False Negative Rate across models under Manipulated and Fragmented conditions.

Models relying on aggregated or flattened statistical representations are particularly vulnerable. For Random Forest and MLP, fragmentation drastically inflates flow-level features (e.g., packet counts and timing dispersion), thereby shifting samples outside the learned decision boundaries. However, even LUCID, despite its sequence-aware design, experiences severe degradation. Fragmentation breaks packet-sequence continuity and distorts temporal dependencies, weakening the coherence of learned attack signatures. Consequently, fragmented malicious traffic increasingly resembles benign communication patterns in both statistical and temporal dimensions.

Although the degradation is widespread, variability across attack types remains observable. Attacks characterized by high-volume flooding behavior (e.g., NTP and WebDDoS) exhibit near complete evasion under fragmentation. Others, such as SNMP or MSSQL, still show degradation but to a comparatively lesser extent, suggesting partial robustness due to stronger protocol-specific characteristics. Nonetheless, no model demonstrates meaningful resilience under fragmented conditions.

Overall, these findings highlight the critical role of structural assumptions within NIDS pipelines. Packet fragmentation operates at an earlier stage than feature manipulation, targeting flow reconstruction and temporal coherence before classification occurs. By disrupting these foundational processing steps, fragmentation acts as a highly effective evasion mechanism that generalizes across heterogeneous detection architectures.

5.5 Robustness Analysis

This section analyzes the robustness of the evaluated architectures against fragmentation-based evasion, focusing on how different modeling assumptions interact with disrupted traffic representations.

The experimental results reveal a clear stratification among model families. Tree-based models and fully connected neural networks exhibit the highest sensitivity to fragmentation-based manipulation. These architectures primarily rely on aggregated, flow-level statistical features, such as packet counts, size distributions, and inter-arrival times, all of which are significantly distorted when packet fragmentation is introduced. As a result, their decision boundaries are strongly affected by structural perturbations that do not alter the underlying malicious semantics of the traffic.

Sequence-aware models, such as LUCID, demonstrate a higher degree of robustness under fragmentation. By incorporating temporal ordering and local dependencies, these models retain partial discriminative capability even when packet continuity is disrupted. Nevertheless, the results show that fragmentation remains an effective evasion mechanism even against sequence-based architectures, particularly when temporal coherence is sufficiently degraded. This indicates that sequence awareness

alone is insufficient to guarantee robustness when the assumptions about packet continuity and flow reconstruction are violated.

A key observation emerging from this analysis is that architectural complexity does not inherently imply robustness. Instead, robustness depends on the alignment between the model’s inductive biases and the actual traffic representation processed by the NIDS. Fragmentation-based evasion exploits a fundamental discrepancy between packet-level network reality and the feature-level abstractions used by learning-based detectors. This discrepancy persists even in advanced deep learning models, underscoring that robustness is primarily a systems-level property rather than a function of model depth or capacity.

From a defensive perspective, these findings highlight the necessity of detection strategies that explicitly address this representation gap. Approaches such as robust and protocol-aware packet reassembly, multi-resolution feature extraction, and adversarially informed training regimes emerge as essential components for improving resilience against fragmentation-based evasion. Model Robustness Comparison

While Random Forest generally achieved the strongest baseline performance, it exhibited significant vulnerability under manipulated and fragmented conditions. The MLP model showed similar fragility, particularly under fragmentation, where it failed entirely to detect certain attack types. Lucid, although exhibiting lower baseline performance overall, demonstrated marginally improved robustness for a small subset of attacks under fragmented conditions. Nevertheless, it also failed catastrophically for SYN, TFTP, and NTP attacks.

These observations indicate that architectural differences alone are insufficient to ensure robustness against traffic transformation. None of the evaluated models consistently maintained acceptable detection performance under fragmentation, underscoring a systemic limitation rather than a model-specific weakness.

From a practical security perspective, the observed increase in FNR is particularly concerning. High FNR values imply that attacks remain undetected, posing a significant risk in real-world deployments. The results demonstrate that high accuracy on clean or baseline datasets can provide a false sense of security, as attackers can exploit simple and well-known traffic manipulation techniques to evade detection.

The findings emphasize the need for intrusion detection systems that are resilient to realistic network behaviors, including packet fragmentation and adversarial traffic transformations. Without such robustness, ML-based IDS solutions remain vulnerable to evasion, limiting their effectiveness in operational environments.

5.6 Statistical Considerations

All experimental evaluations are conducted using a consistent dataset, preprocessing pipeline, and evaluation protocol to ensure fair comparison across models and manipulation scenarios. This controlled setup isolates the impact of fragmentation and other adversarial transformations, allowing meaningful comparative analysis.

Nevertheless, several factors may influence the observed performance trends. The CICIDS2019 dataset exhibits inherent class imbalance, and different attack categories vary substantially in traffic volume and behavioral complexity. Additionally, network traffic is inherently non-stationary, and models trained on static datasets may not fully capture evolving attack strategies or background traffic patterns.

In this work, the FNR is prioritized as the primary evaluation metric due to its critical importance in adversarial and operational security contexts, where missed detections carry substantially higher risk than false alarms. While an increase in evasion success may be associated with changes in the false positive rate, the primary objective of fragmentation-based evaluation is to assess a model’s susceptibility to attack concealment rather than overall classification balance.

Future work may strengthen the statistical validity of these findings through repeated experimental runs, confidence interval estimation, and cross-dataset validation. Despite these limitations, the consistency of the observed trends across architectures and manipulation scenarios provides compelling

evidence that packet fragmentation significantly increases evasion success and constitutes a critical challenge for contemporary NIDS.

5.7 Ablation Study and Design Validation

The ablation study further validates the design choice of integrating packet fragmentation as a first-class manipulation primitive rather than treating it as a secondary or optional transformation.

A direct comparison among the **BASELINE**, **MANIPULATED**, and **FRAGMENTED** scenarios reveals a clear and progressive degradation in NIDS robustness. While statistical manipulation alone already reduces detection performance, the introduction of fragmentation consistently amplifies evasion success across models and attack categories.

IMG/performance_comparison.png

Figure 5.2: Comparison of detection performance across Baseline, Manipulated, and Fragmented datasets for MLP, RF, and LUCID. The grouped bar charts report F1-score, FNR, and Accuracy.

Notably, the **FRAGMENTED** scenario consistently outperforms the **MANIPULATED** scenario in terms of evasion effectiveness. This indicates that modern NIDS, which often rely on features derived from packet headers and flow-level statistics, are particularly vulnerable to adversarial strategies that alter

packet structure while preserving higher-level flow semantics. Fragmentation disrupts size-based, timing-based, and aggregation-dependent features without invalidating the underlying attack behavior, thereby exposing a fundamental weakness in feature extraction pipelines.

These results confirm that fragmentation is not merely an auxiliary perturbation but a structurally impactful evasion mechanism. Its inclusion as a core manipulation primitive is therefore essential for realistic adversarial evaluation and for the development of more robust intrusion detection systems. that the model relies on for classification.

5.8 Single Flow Evolution Analysis

To provide a concrete and interpretable view of how the proposed evasion framework alters traffic behavior, we analyze the evolution of a representative (“centroid”) flow across the three experimental phases: **Original**, **Manipulated**, and **Fragmented**. Because fragmentation may split packets and the generation process can replicate or restructure flows, a strict one-to-one mapping of flow identifiers is not feasible. Instead, we compare statistically representative flows extracted from each phase, enabling a meaningful characterization of feature-level transformations.

Table 5.6 reports the complete feature vector for the selected flows. The comparison offers a granular perspective on how statistical, structural, and temporal properties evolve under adversarial pressure.

Table 5.6: Complete Feature Vector Evolution for Representative Flows

ID	Feature Name	Original	Manipulated	Fragmented
0	Flow Duration	0.0417	0.0028	0.0055
1	Fwd Pkt Len Mean	0.0046	0.0011	0.0031
2	Bwd Pkt Len Mean	0.1200	0.0600	0.0400
3	Tot Len Fwd Pkts	0.0000	0.0000	0.0000
4	Tot Len Bwd Pkts	0.0000	0.0000	0.0000
5	Tot Fwd Pkts	0.0000	0.0000	0.0000
6	Tot Bwd Pkts	0.1413	0.1406	0.1417
7	Flow Bytes/s	0.0038	0.0004	0.0025
8	Flow Pkts/s	0.1200	0.1000	0.0600
9	Flow IAT Std	0.0000	0.0000	0.0000
10	Flow IAT Max	0.0000	0.0000	0.0000
11	Fwd IAT Std	0.0400	0.0000	0.0000
12	Fwd IAT Max	0.0400	0.0600	0.0200
13	Bwd IAT Std	0.0000	0.0000	0.0000
14	Bwd IAT Max	0.0000	0.0000	0.0000
15	Flow IAT Mean	0.0000	0.0000	0.0200
16	Fwd IAT Mean	0.0000	0.0000	0.0000
17	Bwd IAT Mean	0.0023	0.1509	0.1135
18	Fwd Pkt Len Std	0.0000	0.0000	0.0000
19	Bwd Pkt Len Std	0.0000	0.0000	0.0000
20	Pkt Len Var	0.1200	0.1000	0.0700

Note: Zero values indicate features that were originally absent or not activated for this specific attack flow.

5.8.1 Observed Transformation Patterns

The feature evolution reveals three distinct categories of impact:

- **Suppression Effects.** Certain timing features are explicitly neutralized. For example, **Fwd IAT Std** is reduced to zero after manipulation, removing variability cues that may signal bursty or anomalous behavior.

- **Progressive Structural Degradation.** Key discriminative features exhibit stepwise reduction from Original to Manipulated to Fragmented phases. Notable examples include **Bwd Pkt Len Mean** and **Flow Pkts/s**. This gradual attenuation suggests deliberate weakening of statistical signatures commonly associated with high-intensity or amplification-based attacks.
- **Adversarial Artifacts.** Some features increase rather than decrease. The most prominent case is **Bwd IAT Mean**, which rises sharply after manipulation. This reflects intentional injection of inter-packet delays, altering temporal spacing to avoid high-frequency detection rules.

The observed shifts are consistent with the design principles of the evasion framework and provide mechanistic validation of its operation:

1. **Fragmentation-Induced Size Dilution.** The systematic reduction in packet length metrics (e.g., **Bwd Pkt Len Mean**: $0.120 \rightarrow 0.040$) confirms that fragmentation redistributes payload data across multiple smaller packets. This dilutes size-based signatures frequently exploited by NIDS for detecting amplification or data-exfiltration patterns.
2. **Rate-Based Camouflage.** High-rate flow characteristics are substantially attenuated. **Flow Duration** decreases by approximately 86%, while **Flow Pkts/s** drops by roughly 50%. Since many detection rules rely on sustained volumetric anomalies, this suppression effectively camouflages the attack within normal traffic variability.
3. **Temporal Reconfiguration.** The pronounced increase in **Bwd IAT Mean** indicates deliberate spacing of packets. A low inter-arrival time is typically indicative of rapid flooding behavior; increasing this value reduces temporal burstiness and weakens high-frequency attack signatures. Fragmentation further redistributes timing patterns, introducing additional irregularity that disrupts sequential modeling.

5.8.2 Implications

This single-flow analysis demonstrates that the evasion process does not merely induce statistical noise but systematically reshapes the defining characteristics of the attack signature. Structural (packet size), temporal (inter-arrival time), and rate-based (packets per second) attributes are jointly manipulated to weaken discriminative signals.

Importantly, fragmentation amplifies these effects by targeting earlier stages of the traffic processing pipeline. By altering packet granularity and temporal continuity, it compounds the feature-level modifications introduced by the optimizer.

Overall, the analysis confirms that the degradation observed at the model level is grounded in tangible and interpretable traffic transformations. The attack signature is not hidden through random perturbation, but through coordinated structural and temporal reconfiguration designed to exploit inherent assumptions in NIDS feature extraction and flow modeling.

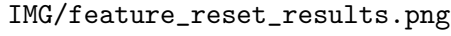
5.9 Vulnerability Analysis: Feature Reset Test

To investigate the structural fragility observed under fragmentation, a progressive feature reset experiment was first conducted. In this setting, an increasing percentage of input features is set to zero at inference time while keeping the remaining features unchanged. This allows to evaluate how resilient the learned representation is to partial information loss.

Figure 5.3 reports the resulting performance trends in terms of absolute accuracy, accuracy degradation, and relative performance drop.

Under baseline conditions, the model remains stable even when up to 50% of features are reset. Significant degradation occurs only at extreme reset levels (75%), suggesting that detection under clean traffic relies on a distributed and redundant set of features.

In contrast, the fragmented scenario exhibits early and severe degradation. Even modest feature removal leads to substantial accuracy loss, and performance collapses rapidly as the reset percentage increases. The manipulated scenario lies between these two extremes.



IMG/feature_reset_results.png

Figure 5.3: Model performance under increasing percentages of feature reset for baseline, manipulated, and fragmented traffic.

The relative degradation curves further confirm that fragmentation amplifies model sensitivity to feature perturbation, indicating reduced redundancy and increased dependency on a limited subset of discriminative statistics.

5.9.1 Single-Feature Dependency Analysis

To identify which specific features drive this vulnerability, each feature was individually reset while keeping all others unchanged. The resulting accuracy drops quantify the model’s reliance on individual variables.

Table 5.7 summarizes the most critical features under fragmented conditions.

Table 5.7: Top Critical Features for Fragmented Traffic Detection

Feature	Accuracy Drop when Reset
Flow IAT Mean (Feat 15)	-41.4%
Flow Duration (Feat 0)	-12.3%
Fwd Pkt Len Mean (Feat 2)	-8.5%

Among all features, **Flow IAT Mean** emerges as the dominant dependency, causing a 41.4% accuracy reduction when reset. This confirms that fragmentation primarily disrupts temporal flow regularity, concentrating discriminative power into timing-based statistics.

The comparatively smaller impact of **Flow Duration** and **Fwd Pkt Len Mean** indicates that while volumetric and size-based features contribute to detection, they are secondary to inter-arrival timing under fragmented conditions.

Overall, the combined analysis demonstrates that fragmentation induces a structural shift in feature reliance, reducing representational redundancy and increasing sensitivity to timing perturbations.

5.10 Strategic Insights

The combined experimental evidence provides several important insights into the security posture of ML-based NIDS:

1. **Timing-Centric Vulnerability.** The strong dependence on inter-arrival time (IAT) statistics represents a structural weakness. Because these features can be modified without altering attack semantics, adversaries can manipulate packet timing, or exploit fragmentation, to reshape flow-level temporal patterns and evade detection.
2. **Processing-Stage Exploitation.** Fragmentation operates upstream of feature extraction, altering packet granularity before statistical aggregation occurs. This reveals a blind spot in pipelines that assume coherent, contiguous flows and do not explicitly account for fragment-aware reconstruction.
3. **Multi-Vector Synergy.** The combination of feature-space optimization (e.g., PSO-driven statistical adjustment) and structural manipulation (fragmentation) yields significantly stronger evasion than either technique independently. This demonstrates that layered adversarial strategies can exploit dependencies across multiple stages of the detection pipeline.
4. **Over-Reliance on Low-Dimensional Signals.** The disproportionate influence of a single timing feature suggests limited redundancy in the learned decision boundaries. When detection hinges on a narrow subset of correlated features, adversarial perturbation of those features can induce cascading failure.

Overall, the Feature Reset analysis confirms that the effectiveness of fragmentation is not incidental but strategically aligned with the models' learned dependencies. These findings highlight the necessity for next-generation NIDS architectures to (i) incorporate fragmentation-aware preprocessing, (ii) diversify feature representations beyond easily manipulable timing statistics, and (iii) adopt robustness-oriented training methodologies capable of resisting coordinated structural and temporal perturbations.

5.11 Overhead Analysis of the Extended Manipulator

The extended version of the *Manipulator* introduces architectural improvements and optional IP fragmentation capabilities. While the core adversarial optimization mechanism remains unchanged, these additions introduce additional computational and traffic overhead. This section analyzes the overhead from three perspectives: computational cost, memory consumption, and network traffic expansion.

5.11.1 Computational Overhead

PSO Optimization Cost

Both the original and extended implementations rely on Particle Swarm Optimization (PSO) for packet manipulation. The computational complexity of PSO per packet group is given by:

$$\mathcal{O}(\text{pso_iter} \times \text{pso_num} \times \text{grp_size}) \quad (5.1)$$

Since the PSO configuration and execution logic remain unchanged, the base optimization cost is identical in both versions.

Feature Extraction Cost

After each optimization step, both implementations:

- Reinitialize the feature extractor,
- Copy the internal network statistics (`nstat`),
- Recompute features using `RunFE()`.

Therefore, the feature extraction overhead remains equivalent. The extended version replaces deprecated timing functions with `time.perf_counter()`, improving measurement precision without affecting computational complexity.

Fragmentation Processing Cost

When packet fragmentation is enabled, additional per-packet operations are introduced:

- IP layer inspection,
- Header length computation,
- Payload size verification,
- Probabilistic fragmentation decision.

These operations introduce a linear overhead:

$$\mathcal{O}(N) \tag{5.2}$$

where N denotes the number of processed packets.

If fragmentation occurs, payload splitting and fragment construction further increase computational cost. Let f denote the average number of fragments per fragmented packet and N_f the number of fragmented packets. The resulting packet count becomes:

$$N' = N + (f - 1) \cdot N_f \tag{5.3}$$

In the worst case, fragmentation introduces an additional complexity of:

$$\mathcal{O}(N \cdot f) \tag{5.4}$$

However, since fragmentation is probabilistic and parameter-controlled, the practical overhead remains bounded.

5.11.2 Memory Overhead

Statistics Storage

The original implementation stores statistics in global variables, whereas the extended version uses instance-level attributes. Although this improves modularity and safety, memory usage remains approximately identical.

Fragmentation-Induced Memory Growth

Fragmentation increases the total number of packet objects stored in memory. If the fragmentation ratio is denoted by r , memory usage scales approximately as:

$$\text{Memory}_{new} \approx (1 + r) \cdot \text{Memory}_{original} \tag{5.5}$$

For example, if 30% of packets are fragmented into three fragments on average, the total packet count increases by approximately 60%, resulting in proportional memory growth.

5.11.3 Network Traffic Overhead

The most significant overhead arises from traffic expansion. Each fragment contains its own IP header, and possibly transport-layer headers, resulting in header duplication.

Let f denote the number of fragments generated per packet. The additional transmitted bytes per fragmented packet can be approximated as:

$$\text{Extra Bytes} \approx (f - 1) \times \text{Header Size} \quad (5.6)$$

Consequently, total bandwidth consumption increases with fragmentation intensity. While this may improve evasion effectiveness, it also increases detectability risk and processing cost for intrusion detection systems.

5.11.4 Summary of Overhead Impact

Table ?? summarizes the comparative overhead characteristics.

Table 5.8: Overhead comparison between original and extended versions			
Component	Original	Extended (No Frag.)	Extended (Frag.)
PSO Cost	Same	Same	Same
Feature Extraction	Same	Same	Same
Packet Inspection	Minimal	Minimal	$+\mathcal{O}(N)$
Packet Creation	Baseline	Baseline	$+\mathcal{O}(N \cdot f)$
Memory Usage	Baseline	Same	Increased
PCAP Size	Baseline	Same	Increased

5.11.5 Conclusion

The architectural refactoring of the *Manipulator* introduces negligible computational overhead when fragmentation is disabled. The primary additional cost stems from optional packet fragmentation, which increases computational complexity linearly, expands memory usage proportionally to the fragmentation ratio, and increases network traffic volume due to header duplication.

Importantly, the overhead is controllable through configurable fragmentation parameters, allowing a trade-off between adversarial effectiveness and resource consumption.

6 Applications of AEGIS

In the remainder of this thesis, the proposed system is referred to as **AEGIS** (Adversarial Evaluation and Generation framework for Intrusion detection Systems), a unified framework integrating the Traffic Manipulator [24] and the FRANTIC fragmentation module into a comprehensive robustness assessment and adversarial training pipeline.

By combining feature-level optimization with controlled structural perturbations, AEGIS generates realistic adversarial traffic that preserves protocol semantics while systematically targeting statistical and architectural vulnerabilities in ML-based NIDS. As such, it constitutes a versatile tool for both offensive and defensive security research.

The framework supports four primary application domains:

1. Dataset Augmentation
2. Penetration Testing and Red Teaming
3. NIDS Robustness Evaluation
4. Adversarial Training and Robust Model Development

6.1 Dataset Augmentation and Penetration Testing

One of the primary challenges in training robust ML-based NIDS is the limited availability of diverse and adversarially informed malicious traffic. Although ML-based NIDS frequently report near-perfect benchmark performance, a growing body of literature highlights systemic weaknesses that undermine their real-world reliability.

Several studies emphasize the over-reliance on outdated or static benchmark datasets. For instance, [27] note that outdated datasets may fail to capture representative modern network patterns. Similarly, [5] argue that heavy dependence on legacy benchmarks significantly reduces relevance to sophisticated threats.

Static datasets also fail to reflect evolving adversarial behavior. As observed in [42], static test corpora do not adequately represent the dynamic nature of adversarial tactics. Moreover, many publicly available datasets lack realistic traffic diversity and concept-drift evaluation capability [8].

AEGIS addresses this limitation by serving as a high-fidelity generator of synthetic adversarial traffic. As illustrated by the data presented in the confusion matrix (Figure 5.1), the progressive increase in sample counts across augmentation stages confirms the effective expansion of the training dataset achieved.

Through its GAN-PSO optimization pipeline, AEGIS actively explores the decision boundary of the target NIDS, generating polymorphic variants of known attacks that retain malicious intent while exhibiting altered statistical properties.

Injecting these adversarially optimized samples into training datasets expands feature space coverage, particularly in underrepresented regions. This reduces reliance on fragile statistical shortcuts and promotes more invariant representations.

The FRANTIC module further enhances diversity by introducing controlled packet fragmentation. By varying parameters such as `fragment_probability` and fragment size, AEGIS produces a continuum of structurally perturbed traffic instances.

The discrepancy between laboratory evaluation and real-world deployment has been widely acknowledged. Public datasets, while useful for benchmarking, are often unsuitable for deployable systems [40]. The gap between academic experimentation and operational practicality remains significant.

As demonstrated in Chapter 5, models trained solely with the original public dataset (CIC-IDS2019) exhibit severe degradation under such conditions. The inclusion of fragmented samples is therefore essential for realistic robustness assessment.

AEGIS serves as a practical tool for systematic adversarial testing of deployed NIDS. Unlike traditional traffic replay or fuzzing approaches, it preserves protocol compliance and attack semantics and manipulates statistical flow features.

This enables security analysts to probe model-specific weaknesses, including timing dependencies and fragmentation blind spots. Organizations can employ AEGIS as a red-teaming instrument to evaluate detection resilience before real adversaries exploit these vulnerabilities.

6.2 NIDS Robustness Evaluation

A major limitation of standard benchmarking practices is the reliance on static datasets that do not incorporate adaptive adversarial behavior.

Fragmentation-based evasion has been documented extensively in prior literature. Attackers have successfully split malicious payloads across multiple packets to hinder signature reconstruction and evade detection [37]. Earlier work demonstrated that inconsistencies in fragment reassembly policies between sensors and hosts enable evasion through overlapping or reordered fragments [15].

Flow-based NIDS introduce additional constraints. As discussed in [28], flow-level features often require connection termination before computation, limiting real-time detection and increasing vulnerability to structurally manipulated traffic.

These documented weaknesses motivate the design of AEGIS as a systematic robustness evaluation framework rather than a purely generative adversarial tool.

To emphasize the general applicability of AEGIS, a consolidated representation of the previously discussed experiments is presented across multiple NIDS architectures.

Figure 6.1 summarizes the performance degradation observed when models trained on baseline traffic are evaluated on manipulated and fragmentation-enhanced traffic generated by AEGIS.

Under baseline conditions, all models achieve near-perfect F1 scores (above 0.97). However, when evaluated against fragmented traffic, performance collapses dramatically. For example:

- MLP F1 decreases from 0.9987 to 0.1826,
- RF F1 decreases from 0.9999 to 0.0306,
- LUCID F1 decreases from 0.9757 to 0.1695.

Correspondingly, FNR increases to values exceeding 0.89 across architectures.

These results demonstrate that fragmentation-based structural obfuscation represents a general vulnerability rather than a model-specific weakness.

6.3 Adversarial Training

Beyond vulnerability discovery, AEGIS enables defensive hardening through adversarial training. However, adversarial training offers a promising defensive strategy but introduces non-trivial trade-offs.

Indeed, it does not impose any inference-time overhead, unlike detection-based methods. However, it often results in suboptimal clean-data performance, leading to the trade-off between robustness and accuracy. This is caused by shifts in the decision boundaries of the models due to min-max optimization.

Robust training may also require substantially increased model capacity and significantly more training data. Empirical evidence shows that modest robustness improvements can demand hundreds of thousands of additional samples.

Furthermore, robustness gains may be uneven across classes, leading to class-wise robustness imbalance and potential blind spots.

Transferability Considerations

Transfer-based attacks leverage the phenomenon whereby adversarial examples crafted for one model remain effective against another [27]. However, transferability in network security contexts is less consistent than in vision domains. Architectural variance and feature constraints reduce cross-model generalization of adversarial samples.

These findings justify AEGIS’s surrogate-based optimization approach while acknowledging inherent uncertainty.

Surrogate Model Limitations

The effectiveness of surrogate-based optimization depends on accurate approximation of the target model’s decision boundary. Surrogate models may fail to capture complex boundary geometry, reducing transfer success [20].

Searching for evasion points that are only marginally misclassified by the surrogate may rarely evade the true classifier [60]. Additionally, unrealistic assumptions about attacker data access undermine practical applicability in some prior work [50].

AEGIS mitigates these issues by constraining perturbations to realistic feature manipulations and protocol-compliant structural transformations.

6.3.1 Training Strategy Comparison

To evaluate this capability, the MLP model was trained under three regimes:

1. OG: Original dataset only,
2. MANIPULATED: Dataset augmented using the Traffic Manipulator,
3. ADV: Dataset augmented using the full AEGIS pipeline.

Figure 6.2 reports the resulting performance across baseline, manipulated, and fragmented test conditions.

Training exclusively on clean data leads to catastrophic failure under fragmentation ($\text{FNR} = 0.8922$). In contrast, adversarial training significantly reduces FNR under evasion scenarios while maintaining competitive baseline performance.

These findings confirm that AEGIS functions not only as an attack generator but also as a robustness enhancement mechanism.

Moreover, the configurable nature of AEGIS enables staged training strategies in which perturbation intensity is progressively increased. Such approach may further mitigate the typical robustness–accuracy trade-off observed in adversarial training.

6.4 Limitations

Despite its capabilities, the improved manipulator has several limitations that warrant discussion.

Protocol Implementation Constraints

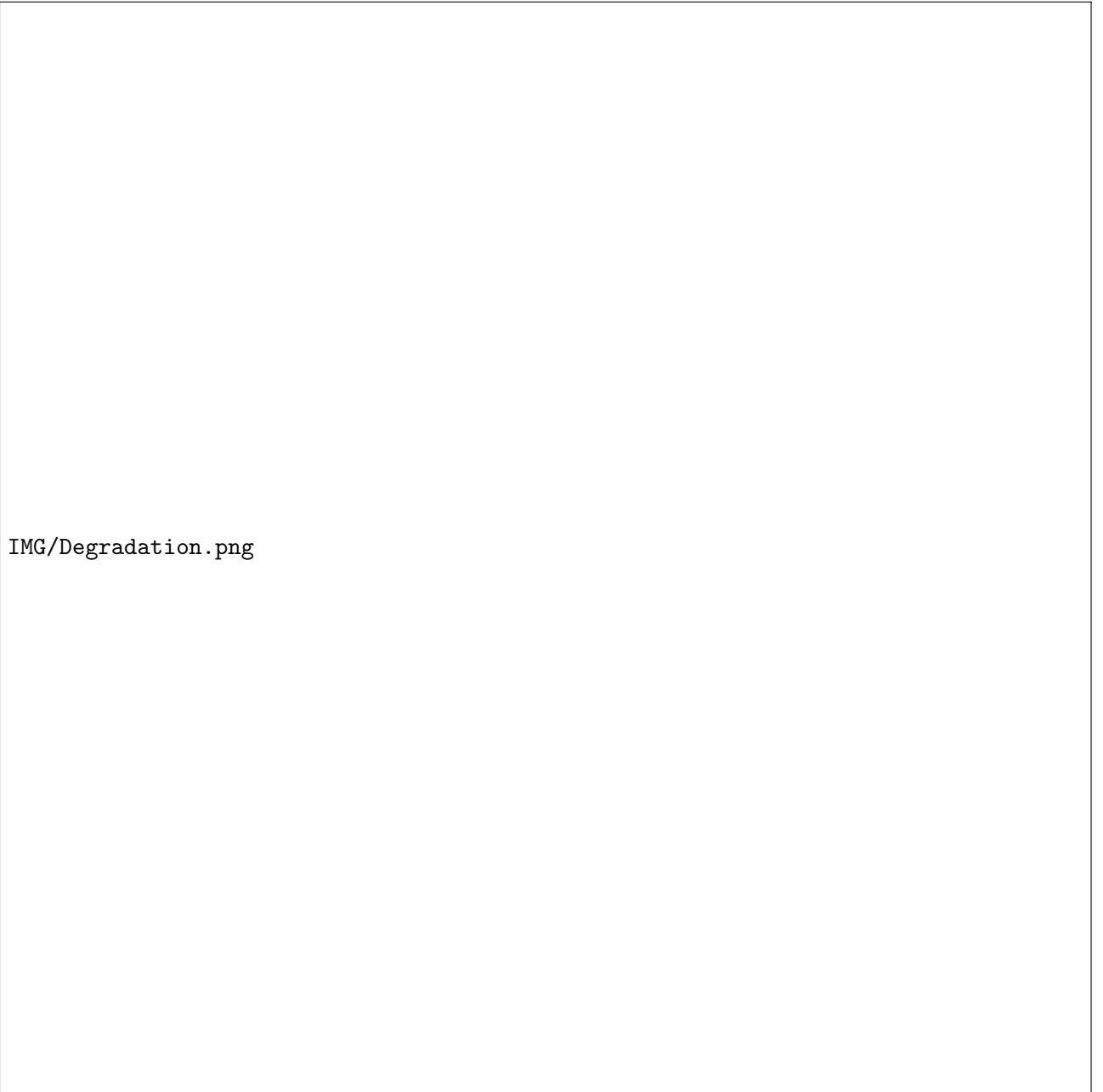
The current fragmentation implementation relies on standard IP header manipulation and remains compliant with relevant RFCs. However, it does not fully emulate complex fragmentation-based exploits such as *Teardrop* attacks or OS-specific overlapping fragment strategies. Additionally, aggressive perturbations of stateful protocols (e.g., TCP handshake manipulation) may inadvertently invalidate connections, leading to traffic that is evasive due to non-functionality rather than true adversarial behavior.

Dependency on Surrogate Models

The effectiveness of the GAN-PSO adversarial search depends on the fidelity of the surrogate model used during optimization. If the target NIDS employs substantially different features or architectures, attack transferability may be reduced, limiting the effectiveness of generated adversarial samples.

Real-Time Generation Latency

The iterative nature of PSO introduces non-negligible computational overhead, making the framework more suitable for offline dataset generation and evaluation than for real-time deployment as an evasion proxy. Fragmentation itself is computationally lightweight and can be applied near real-time; however, the full adversarial optimization pipeline is not designed for high-throughput live attack scenarios.



IMG/Degradation.png

Figure 6.1: Performance degradation across multiple NIDS architectures under baseline, manipulated, and fragmented traffic conditions generated by AEGIS.



IMG/ADV_TRAINING_COMPARISON.png

Figure 6.2: Model performance under different training strategies: original training (OG), feature-level augmentation (MANIPULATED), and full AEGIS adversarial training (ADV).

7 Conclusions and Future Work

This thesis has examined the structural and statistical vulnerabilities of ML-NIDS when confronted with adversarially manipulated network traffic. While recent literature frequently reports near-perfect detection rates on benchmark datasets, the results presented in this work demonstrate that such performance metrics may provide a misleading sense of security when adversarial conditions are introduced.

The central objective of this research was to investigate whether flow-based ML-NIDS remain reliable when malicious traffic is deliberately altered in a manner that preserves its semantic intent while perturbing its statistical and structural representation. To this end, an existing GAN-PSO adversarial manipulation framework was extended with a protocol-compliant packet fragmentation engine FRANTIC, enabling the generation of traffic samples that simultaneously modify feature-space statistics and packet-level structure.

The experimental findings confirm that current flow-based detection models are highly sensitive to such perturbations. Even moderate manipulations significantly degrade classification performance, and when fragmentation is introduced, detection accuracy collapses in several scenarios. These results highlight a fundamental fragility: models trained on static datasets learn decision boundaries that are not invariant to realistic traffic transformations. Consequently, robustness cannot be assumed from high baseline accuracy alone.

Beyond demonstrating an evasion mechanism, this thesis contributes to a broader understanding of how feature extraction pipelines, rather than classifier architectures alone, constitute critical attack surfaces in ML-based security systems. The fragmentation strategy does not directly attack the model weights; instead, it alters the statistical representation of traffic prior to classification. This distinction underscores the importance of considering the entire detection pipeline, including preprocessing and feature aggregation, when evaluating adversarial resilience.

7.1 Summary of Contributions

The primary contributions of this thesis can be articulated along methodological, experimental, and analytical dimensions.

- **Extension of an Adversarial Manipulation Framework.** This work enhances an existing GAN-PSO traffic manipulation framework by integrating a structural perturbation module capable of performing controlled IP fragmentation. Unlike purely statistical manipulation approaches, the extended framework generates adversarial traffic that modifies both high-level flow features and low-level packet composition. This dual manipulation significantly increases the attack surface and more closely approximates realistic adversarial behavior.
- **Design and Implementation of an RFC-Compliant Fragmentation Engine.** A key contribution lies in the implementation of a fragmentation mechanism that adheres strictly to IPv4 standards (RFC 791). Ensuring protocol compliance guarantees that generated adversarial samples are not artificial artifacts but valid, routable traffic instances capable of traversing network infrastructure. This design choice strengthens the realism and practical relevance of the experimental evaluation.
- **Comprehensive Experimental Evaluation Across Manipulation Scenarios.** Through systematic experimentation, this thesis quantified the impact of statistical manipulation alone and in combination with packet fragmentation. The results demonstrate that while feature-space perturbations already degrade detection performance, structural manipulation amplifies this effect considerably. In several attack categories, evasion rates approach complete bypass, revealing a critical gap between benchmark accuracy and adversarial robustness.

- **Feature-Level Vulnerability Analysis.** A feature reset analysis was conducted to identify which flow statistics contribute most strongly to model decisions under fragmentation. The results indicate heavy reliance on temporal aggregation metrics such as inter-arrival time statistics. This finding provides insight into the specific mechanisms through which fragmentation disrupts detection and highlights the need for more invariant feature representations.
- **Empirical Evidence of Structural Fragility in Flow-Based NIDS.** The overall findings support the hypothesis that flow-based ML-NIDS are inherently brittle when feature extraction depends on aggregated packet statistics. Fragmentation alters packet counts, timing distributions, and byte-level aggregates, effectively corrupting the feature vector before classification occurs. This reveals a structural weakness in detection pipelines that rely exclusively on summarized flow descriptors.

7.2 Implications for NIDS Research

The implications of these findings extend beyond the specific framework implemented in this work and raise broader considerations for the design and evaluation of ML-based intrusion detection systems.

7.2.1 The Critical Role of Traffic Reassembly

The high success rate of fragmentation-based evasion strongly suggests that any NIDS operating on fragmented flows without prior reassembly is inherently vulnerable. Feature extraction performed on incomplete or artificially segmented flows produces distorted representations that undermine classifier reliability. Therefore, robust defragmentation mechanisms must be considered a foundational requirement rather than an optional preprocessing step. Increasing model complexity or adopting deeper architectures cannot compensate for fundamentally corrupted input representations.

7.2.2 Rethinking Feature Engineering in ML-NIDS

The demonstrated vulnerability to PSO-driven manipulation indicates that models heavily dependent on low-level statistical features, such as packet length distributions, flow duration, and inter-arrival times, are particularly susceptible to adversarial perturbation. These features, while computationally efficient, lack invariance to structural transformations of traffic.

Future research should therefore explore alternative representations, including:

- Stateful behavioral modeling across extended time windows,
- Graph-based representations of host communication patterns,
- Payload-aware or hybrid Deep Packet Inspection (DPI) assisted approaches,
- Temporal sequence modeling using recurrent or attention-based architectures.

Such approaches may reduce sensitivity to superficial statistical manipulation while preserving detection capability.

7.2.3 Adversarial Robustness as a Standard Evaluation Criterion

A central insight of this thesis is that high performance on static benchmark datasets does not guarantee operational security. The observed discrepancy between baseline accuracy and adversarial performance underscores the necessity of incorporating robustness evaluation into standard NIDS assessment protocols. Just as accuracy, precision, and recall are routinely reported, robustness under active adversarial manipulation should become a standard metric in future research.

Without adversarial stress-testing, published performance figures risk overstating the true defensive capability of ML-based systems.

7.3 Limitations

While the results provide compelling evidence of structural vulnerability, certain limitations must be acknowledged. The experimental evaluation was conducted within a controlled environment and fo-

cused primarily on flow-based detection architectures. Although the fragmentation engine is protocol-compliant, real-world network conditions, such as middlebox behavior, stateful firewalls, and traffic normalization mechanisms, may influence attack feasibility.

Furthermore, the surrogate-based optimization approach assumes partial knowledge of the target system. Although such assumptions are common in adversarial ML research, they may not always reflect practical deployment scenarios. These limitations do not invalidate the findings but rather contextualize their scope and motivate further empirical validation.

7.4 Future Work

Several promising directions emerge from this study, aiming both to enhance adversarial evaluation methodologies and to develop more resilient detection systems.

7.4.1 Adversarial Training and Defensive Hardening

One immediate extension involves systematically incorporating adversarial samples generated by the framework into the training process. Evaluating robustness–accuracy trade-offs, class-wise robustness balance, and potential overfitting to specific perturbation patterns would provide deeper insight into the defensive potential of adversarial training within NIDS contexts.

7.4.2 Extension to IPv6 and Advanced Transport-Layer Manipulation

The current implementation focuses on IPv4 fragmentation. Extending support to IPv6 fragmentation via Extension Headers represents a necessary step to maintain relevance as network infrastructure evolves. Additionally, incorporating more advanced transport-layer manipulations, such as overlapping fragments, TCP segmentation anomalies, or adaptive window manipulation, would further expand the threat model and improve realism.

7.4.3 Hybrid and Multi-Layer Detection Architectures

Future research should explore hybrid detection architectures that integrate:

- Signature-based detection of known fragmentation anomalies,
- Stateful traffic reassembly engines,
- Machine learning-based anomaly detection.

Investigating methods for fusing these components, either through ensemble strategies or hierarchical decision pipelines, may yield architectures that balance robustness and computational efficiency.

7.4.4 Explainability under Adversarial Conditions

Understanding why specific fragmented samples evade detection remains a critical research question. Applying Explainable AI (XAI) techniques to visualize feature importance shifts and decision-boundary distortions induced by fragmentation could reveal systematic blind spots. Such insights would not only improve interpretability but also guide the development of more invariant feature representations.

7.4.5 Real-World Deployment Validation

Finally, validating the framework and corresponding defenses in realistic network environments represents a crucial step toward operational relevance. Evaluating performance against production-grade NIDS systems, measuring throughput constraints, and assessing latency overhead would bridge the gap between academic experimentation and deployable security solutions.

In conclusion, this thesis demonstrates that adversarial robustness remains an under-addressed challenge in ML-based network intrusion detection. By exposing structural weaknesses in flow-based detection pipelines and proposing a realistic adversarial evaluation framework, this work contributes to a more security-oriented understanding of machine learning deployment in network defense. Ensuring robustness against adaptive adversaries must become a foundational objective in the next generation of intelligent intrusion detection systems.

Bibliography

- [1] Maged Abdelaty, Sandra Scott-Hayward, Roberto Doriguzzi-Corin, and Domenico Siracusa. Gadot: Gan-based adversarial training for robust ddos attack detection. In *2021 IEEE Conference on Communications and Network Security (CNS)*, pages 119–127. IEEE, 2021.
- [2] Abdulruhman Abomakhelb, Kamarularifin Abd Jalil, Alya Geogiana Buja, Abdulraqueb Alhammadi, and Abdulmajeed M Alenezi. A comprehensive review of adversarial attacks and defense strategies in deep neural networks. *Technologies*, 13(5):202, 2025.
- [3] Rana Abou Khamis and Ashraf Matrawy. Evaluation of adversarial training on different types of neural networks in deep learning-based idss. In *2020 international symposium on networks, computers and communications (ISNCC)*, pages 1–6. IEEE, 2020.
- [4] Rana Abou Khamis, M Omair Shafiq, and Ashraf Matrawy. Investigating resistance of deep learning-based ids against adversaries using min-max optimization. In *ICC 2020-2020 IEEE international conference on communications (ICC)*, pages 1–7. IEEE, 2020.
- [5] Meaad Ahmed and Qutaiba Abdullah. Network intrusion detection systems: Machine learning-based attack and remedy strategies—a review. *Al-Salam Journal for Engineering and Technology*, 4(2):11–29, 2025.
- [6] Usama Ahmed, Mohammad Nazir, Amna Sarwar, Tariq Ali, El-Hadi M Aggoune, Tariq Shahzad, and Muhammad Adnan Khan. Signature-based intrusion detection using machine learning and deep learning approaches empowered with fuzzy clustering. *Scientific Reports*, 15(1):1726, 2025.
- [7] Qutaiba Alasad, Meaad Ahmed, Shahad Alahmed, Omer T Khattab, Saba Alaa Abdulwahhab, and Jiann-Shuin Yuan. A comprehensive review: The evolving cat-and-mouse game in network intrusion detection systems leveraging machine learning. *Journal of Cybersecurity and Privacy*, 6(1):13, 2026.
- [8] Afnan Alotaibi and Murad A Rassam. Adversarial machine learning attacks against intrusion detection systems: A survey on strategies and defense. *Future Internet*, 15(2):62, 2023.
- [9] Giovanni Apruzzese, Aurore Fass, and Fabio Pierazzi. When adversarial perturbations meet concept drift: an exploratory analysis on ml-nids. In *Proceedings of the 2024 Workshop on Artificial Intelligence and Security*, pages 149–160, 2024.
- [10] Md Ahsan Ayub, William A Johnson, Douglas A Talbert, and Ambareen Siraj. Model evasion attack on intrusion detection systems using adversarial machine learning. In *2020 54th annual conference on information sciences and systems (CISS)*, pages 1–6. IEEE, 2020.
- [11] Ziyi Bao and Luis Muñoz-González. Mitigating evasion attacks against machine learning systems through dimensionality reduction and denoising. 2018.
- [12] Andrea Bittau. The fragmentation attack in practice. In *Ieee symposium on security and privacy, ieee computer society*, 2005.
- [13] Panagiotis Bountakas, Apostolis Zarras, Alexios Lekidis, and Christos Xenakis. Defense strategies for adversarial machine learning: A survey. *Computer Science Review*, 49:100573, 2023.
- [14] Daniela Brauckhoff, Kave Salamatian, and Martin May. Applying pca for traffic anomaly detection: Problems and solutions. In *IEEE INFOCOM 2009*, pages 2866–2870. IEEE, 2009.

- [15] Igino Corona, Giorgio Giacinto, and Fabio Roli. Adversarial attacks against intrusion detection systems: Taxonomy, solutions and open issues. *Information sciences*, 239:201–225, 2013.
- [16] Susu Cui, Cong Dong, Meng Shen, Yuling Liu, Bo Jiang, and Zhigang Lu. Cbseq: A channel-level behavior sequence for encrypted malware traffic detection. *IEEE Transactions on Information Forensics and Security*, 18:5011–5025, 2023.
- [17] Islam Debicha, Benjamin Cochez, Tayeb Kenaza, Thibault Debatty, Jean-Michel Dricot, and Wim Mees. Adv-bot: Realistic adversarial botnet attacks against network intrusion detection systems. *Computers & Security*, 129:103176, 2023.
- [18] Archan Dhadhanania, Jitendra Bhatia, Rachana Mehta, Sudeep Tanwar, Ravi Sharma, and Amit Verma. Unleashing the power of sdn and gnn for network anomaly detection: State-of-the-art, challenges, and future directions. *Security and Privacy*, 7(1):e337, 2024.
- [19] Roberto Doriguzzi-Corin, Stuart Millar, Sandra Scott-Hayward, Jesus Martinez-del Rincon, and Domenico Siracusa. Lucid: A practical, lightweight deep learning solution for ddos attack detection. *IEEE Transactions on Network and Service Management*, 17(2):876–889, 2020.
- [20] Sabrine Ennaji, Fabio De Gaspari, Dorjan Hitaj, Alicia Kbidi, and Luigi Vincenzo Mancini. Adversarial challenges in network intrusion detection systems: Research insights and future prospects. *IEEE Access*, 2025.
- [21] Bardia Esmaili, Amin Azmoodeh, Ali Dehghantanha, Gautam Srivastava, Hadis Karimipour, and Jerry Chun-Wei Lin. A gnn-based adversarial internet of things malware detection framework for critical infrastructure: Studying gafgyt, mirai and tsunami campaigns. *IEEE Internet of Things Journal*, 2023.
- [22] Xin Fan Guo, Albert Merono Penuela, Sergio Maffei, and Fabio Pierazzi. Knowml: Improving generalization of ml-nids with attack knowledge graphs. *arXiv preprint arXiv:2506.19802*, 2025.
- [23] Dongqi Han, Zhiliang Wang, Wenqi Chen, Ying Zhong, Su Wang, Han Zhang, Jiahai Yang, Xingang Shi, and Xia Yin. Deepaid: Interpreting and improving deep learning-based anomaly detection in security applications. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 3197–3217, 2021.
- [24] Dongqi Han, Zhiliang Wang, Ying Zhong, Wenqi Chen, Jiahai Yang, Shuqiang Lu, Xingang Shi, and Xia Yin. Evaluating and improving adversarial robustness of machine learning-based network intrusion detectors. *IEEE Journal on Selected Areas in Communications*, 39(8):2632–2647, 2021.
- [25] Alexander Hartl, Maximilian Bachl, Joachim Fabini, and Tanja Zseby. Explainability and adversarial robustness for rnns. In *2020 IEEE Sixth International Conference on Big Data Computing Service and Applications (BigDataService)*, pages 148–156. IEEE, 2020.
- [26] Mohammad J Hashemi and Eric Keller. Enhancing robustness against adversarial examples in network intrusion detection systems. In *2020 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)*, pages 37–43. IEEE, 2020.
- [27] Ke He, Dan Dongseong Kim, and Muhammad Rizwan Asghar. Adversarial machine learning for network intrusion detection systems: A comprehensive survey. *IEEE Communications Surveys & Tutorials*, 25(1):538–566, 2023.
- [28] Ke He, Dan Dongseong Kim, and Muhammad Rizwan Asghar. Adversarial machine learning for network intrusion detection systems: A comprehensive survey. *IEEE Communications Surveys & Tutorials*, 25(1):538–566, 2023.
- [29] Ling Huang, Anthony D Joseph, Blaine Nelson, Benjamin IP Rubinstein, and J Doug Tygar. Adversarial machine learning. In *Proceedings of the 4th ACM workshop on Security and artificial intelligence*, pages 43–58, 2011.

- [30] Olakunle Ibitoye, Rana Abou-Khamis, Ashraf Matrawy, M Omair Shafiq, et al. The threat of adversarial attacks against machine learning in network security: a survey. *Journal of Electronics and Electrical Engineering*, pages 16–59, 2025.
- [31] Hongling Jiang, Jinzhi Lin, and Haiyan Kang. Fgmd: A robust detector against adversarial attacks in the iot network. *Future Generation Computer Systems*, 132:194–210, 2022.
- [32] Zeinab Khorshidpour, Jafar Tahmoresnezhad, Sattar Hashemi, and Ali Hamzeh. Domain invariant feature extraction against evasion attack. *International Journal of Machine Learning and Cybernetics*, 9(12):2093–2104, 2018.
- [33] Deqiang Li and Qianmu Li. Adversarial deep ensemble: Evasion attacks and defenses for malware detection. *IEEE Transactions on Information Forensics and Security*, 15:3886–3900, 2020.
- [34] Zixuan Liu, Yi Zhao, Zhuotao Liu, Qi Li, Chuanpu Fu, Guangmeng Zhou, and Ke Xu. A hard-label black-box evasion attack against ml-based malicious traffic detection systems. *arXiv preprint arXiv:2510.14906*, 2025.
- [35] Pooria Madani and Natalija Vlajic. Robustness of deep autoencoder in intrusion detection under adversarial contamination. In *Proceedings of the 5th Annual Symposium and Bootcamp on Hot Topics in the Science of Security*, pages 1–8, 2018.
- [36] Andrew McCarthy, Panagiotis Andriotis, Essam Ghadafi, and Phil Legg. Feature vulnerability and robustness assessment against adversarial machine learning attacks. In *2021 international conference on cyber situational awareness, data analytics and assessment (CyberSA)*, pages 1–8. IEEE, 2021.
- [37] Khandakar Md Shafin, GM Abdullah Al Kafi, and Saha Reno. Guardians of the network: An ensemble learning framework with adversarial alignment for evasive cyber threat detection. *Engineering Reports*, 7(10):e70419, 2025.
- [38] Mohamed Amine Merzouk, Frédéric Cuppens, Nora Boulahia-Cuppens, and Reda Yaich. Investigating the practicality of adversarial evasion attacks on network intrusion detection. *Annals of Telecommunications*, 77(11):763–775, 2022.
- [39] David J Miller, Zhen Xiang, and George Kesidis. Adversarial learning targeting deep neural network classification: A comprehensive review of defenses against attacks. *Proceedings of the IEEE*, 108(3):402–433, 2020.
- [40] Subash Neupane, Jesse Ables, William Anderson, Sudip Mittal, Shahram Rahimi, Ioana Banicescu, and Maria Seale. Explainable intrusion detection systems (x-ids): A survey of current methods, challenges, and opportunities. *IEEE Access*, 10:112392–112415, 2022.
- [41] Khushnaseeb Roshan and Aasim Zafar. Black-box adversarial transferability: An empirical study in cybersecurity perspective. *Computers & Security*, 141:103853, 2024.
- [42] Ms Khushnaseeb Roshan and Aasim Zafar. Boosting robustness of network intrusion detection systems: A novel two phase defense strategy against untargeted white-box optimization adversarial attack. *Expert Systems with Applications*, 249:123567, 2024.
- [43] Benjamin IP Rubinstein, Blaine Nelson, Ling Huang, Anthony D Joseph, Shing-hon Lau, Satish Rao, Nina Taft, and J Doug Tygar. Antidote: understanding and defending against poisoning of anomaly detectors. In *Proceedings of the 9th ACM SIGCOMM Conference on Internet Measurement*, pages 1–14, 2009.
- [44] Alper Sarıkaya, Banu Günel Kılıç, and Mehmet Demirci. Raids: Robust autoencoder-based intrusion detection system model against adversarial attacks. *Computers & Security*, 135:103483, 2023.

- [45] Kudzai Sauka, Gun-Yoo Shin, Dong-Wook Kim, and Myung-Mook Han. Adversarial robust and explainable network intrusion detection systems based on deep learning. *Applied Sciences*, 12(13):6451, 2022.
- [46] Sanidhya Sharma and Zesheng Chen. A systematic study of adversarial attacks against network intrusion detection systems. *Electronics*, 13(24):5030, 2024.
- [47] Kamran Shaukat, Suhuai Luo, and Vijay Varadharajan. A novel method for improving the robustness of deep learning-based malware detectors against adversarial attacks. *Engineering Applications of Artificial Intelligence*, 116:105461, 2022.
- [48] Zhenlu Sun, André MH Teixeira, and Salman Toor. Gnn-ids: Graph neural network based intrusion detection system. In *Proceedings of the 19th international conference on availability, reliability and security*, pages 1–12, 2024.
- [49] Qi Tian, Kun Kuang, Kelu Jiang, Fei Wu, and Yisen Wang. Analysis and applications of class-wise robustness in adversarial training. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, pages 1561–1570, 2021.
- [50] João Vitorino, Isabel Praça, and Eva Maia. Sok: Realistic adversarial attacks and defenses for intelligent network intrusion detection. *Computers & Security*, 134:103433, 2023.
- [51] João Vitorino, Miguel Silva, Eva Maia, and Isabel Praça. Reliable feature selection for adversarially robust cyber-attack detection. *Annals of Telecommunications*, 80(3):341–355, 2025.
- [52] Minxiao Wang, Ning Yang, Nicolas J Forcade-Perkins, and Ning Weng. Progen: Projection-based adversarial attack generation against network intrusion detection. *IEEE Transactions on Information Forensics and Security*, 19:5476–5491, 2024.
- [53] Shunyao Wang, Ryan KL Ko, Guangdong Bai, Naipeng Dong, Taejun Choi, and Yanjun Zhang. Evasion attack and defense on machine learning models in cyber-physical systems: A survey. *IEEE communications surveys & tutorials*, 26(2):930–966, 2023.
- [54] Feng Wei, Hongda Li, Ziming Zhao, and Hongxin Hu. {xNIDS}: Explaining deep learning-based network intrusion detection systems for active intrusion responses. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 4337–4354, 2023.
- [55] Luming Yang, Lin Liu, Jun-jie Huang, Jiangyong Shi, Shaojing Fu, Yongjun Wang, and Jinshu Su. Robustness matters: Pre-training can enhance the performance of encrypted traffic analysis. *IEEE Transactions on Information Forensics and Security*, 2025.
- [56] Shuo Yang, Xinran Zheng, Jinze Li, Jinfeng Xu, Xinchun Zhang, and Edith CH Ngai. Self-supervised adaptation method to concept drift for network intrusion detection. *IEEE Transactions on Dependable and Secure Computing*, 2025.
- [57] Shaokai Ye, Kaidi Xu, Sijia Liu, Hao Cheng, Jan-Henrik Lambrechts, Huan Zhang, Aojun Zhou, Kaisheng Ma, Yanzhi Wang, and Xue Lin. Adversarial robustness vs. model compression, or both? In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 111–120, 2019.
- [58] Xinwei Yuan, Shu Han, Wei Huang, Hongliang Ye, Xianglong Kong, and Fan Zhang. A simple framework to enhance the adversarial robustness of deep learning-based intrusion detection system. *Computers & Security*, 137:103644, 2024.
- [59] Chaoyun Zhang, Xavier Costa-Perez, and Paul Patras. Adversarial attacks against deep learning-based network intrusion detection systems and defense mechanisms. *IEEE/ACM Transactions on Networking*, 30(3):1294–1311, 2022.

- [60] Fei Zhang, Patrick PK Chan, Battista Biggio, Daniel S Yeung, and Fabio Roli. Adversarial feature selection against evasion attacks. *IEEE transactions on cybernetics*, 46(3):766–777, 2015.
- [61] Jianjin Zhao, Zhiwei Cui, Junsong Fu, Meng Shen, and Qi Li. A unified framework for robust encrypted malicious traffic detection in adverse environments via graph structure learning. *IEEE Transactions on Network Science and Engineering*, 2025.

[?]

Appendix A Complete tables

A.1 F1 score tables attack per attack

Dataset	WebDDoS	LDAP	PortMap	DNS	UDP	NTP	SNMP	SSDP	SYN	TFTP	NetBIOS	MSSQL
BASILINE	0.9880	0.9817	1.0000	0.9563	1.0000	0.9999	0.9993	0.9998	0.9982	1.0000	0.9999	1.0000
MANIPULATED	0.7317	0.9383	0.9676	0.7338	0.9114	0.5753	0.9251	0.8742	0.0000	0.2225	0.9732	0.9093
FRAGMENTED	0.4565	0.7940	0.7807	0.6393	0.4284	0.2048	0.7261	0.8174	0.0001	0.0020	0.7435	0.6998

Table A.1: Random Forest F1-score across datasets and attack types

Dataset	WebDDoS	LDAP	Portmap	DNS	UDPLag	NTP	SNMP	SSDP	SYN	TFTP	UDP	NetBIOS	MSSQL
Baseline	0.6500	0.8120	0.9167	0.7018	0.9635	0.9217	0.9594	0.9955	0.9967	1.0000	0.9997	0.9063	0.9999
Manipulated	0.1639	0.8689	0.8792	0.8008	0.9665	0.5289	0.8854	0.9783	0.0000	0.2343	0.9803	0.7714	0.9790
Fragmented	0.0129	0.9346	0.9096	0.5859	0.5617	0.0708	0.8903	0.9444	0.0002	0.0032	0.7688	0.8849	0.9186

Table A.2: Lucid F1-score across dataset conditions and attack types

Dataset	WebDDoS	LDAP	Portmap	DNS	UDPLag	NTP	SNMP	SSDP	SYN	TFTP	UDP	NetBIOS	MSSQL
Baseline	1.0000	0.9936	1.0000	0.9118	0.9925	0.9972	1.0000	0.9992	1.0000	0.9989	0.9996	0.9977	0.9995
Manipulated	0.6429	0.9500	0.9741	0.9094	0.9640	0.5873	0.9678	0.9683	0.0000	0.2341	0.9721	0.8562	0.9774
Fragmented	0.4762	0.8185	0.9022	0.8036	0.5173	0.1775	0.8688	0.9630	0.0002	0.0057	0.7842	0.8720	0.8983

Table A.3: MLP F1-score across dataset conditions and attack types

A.2 FNR tables attack per attack

Dataset	WebDDoS	LDAP	PortMap	DNS	UDP	NTP	SNMP	SSDP	SYN	TFTP	NetBIOS	MSSQL
BASILINE	0.0000	0.0348	0.0000	0.0822	0.0000	0.0000	0.0000	0.0000	0.0036	0.0000	0.0001	0.0000
MANIPULATED	0.4024	0.0912	0.0346	0.3494	0.0554	0.5669	0.0357	0.0955	1.0000	0.8543	0.0295	0.0246
FRAGMENTED	0.6656	0.2809	0.3277	0.4309	0.6557	0.8736	0.3786	0.1976	0.9999	0.9988	0.3492	0.3825

Table A.4: Random Forest FNR across datasets and attack types

Dataset	WebDDoS	LDAP	Portmap	DNS	UDPLag	NTP	SNMP	SSDP	SYN	TFTP	UDP	NetBIOS	MSSQL
Baseline	0.0000	0.0127	0.0000	0.1622	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0030	0.0001
Manipulated	0.5091	0.0052	0.0020	0.1022	0.0252	0.5256	0.0054	0.0111	1.0000	0.8594	0.0009	0.0000	0.0000
Fragmented	0.6791	0.2945	0.1404	0.3077	0.6351	0.8996	0.1925	0.0370	0.9999	0.9969	0.3227	0.1774	0.1462

Table A.5: MLP FNR across dataset conditions and attack types

Dataset	WebDDoS	LDAP	Portmap	DNS	UDPLag	NTP	SNMP	SSDP	SYN	TFTP	UDP	NetBIOS	MSSQL
Baseline	0.5185	0.3165	0.0833	0.4595	0.0000	0.1453	0.0152	0.0023	0.0066	0.0000	0.0006	0.1714	0.0001
Manipulated	0.9091	0.1675	0.1657	0.2883	0.0190	0.5897	0.1597	0.0047	1.0000	0.8603	0.0005	0.1562	0.0000
Fragmented	0.9933	0.0798	0.1356	0.5538	0.5961	0.9627	0.1616	0.0556	0.9999	0.9983	0.3386	0.1669	0.1085

Table A.6: Lucid FNR across dataset conditions and attack types

Appendix B Implementation of the Fragmentation Module

This appendix provides the implementation details of the proposed fragmentation module described in Section ?? . The code listings are included to ensure full reproducibility of the experimental setup.

B.1 Orchestration Logic (manipulator.py)

The following method applies probabilistic fragmentation to a stream of packets after PSO-based optimization.

Listing B.1: Probabilistic fragmentation orchestration.

```
1 def _apply_fragmentation(self, packets):
2     """Apply random fragmentation to packets with configurable probability
3         """
4     from rebuilder import fragment_packet
5     fragmented_packets = []
6
7     for i, pkt in enumerate(packets):
8         if pkt.haslayer(IP):
9             ip_layer = pkt[IP]
10            ip_header_len = 20 if ip_layer.ihl is None else ip_layer.ihl * 4
11            payload_size = len(pkt) - ip_header_len
12
13            if payload_size > self.min_fragment_size:
14                if np.random.random() <= self.fragment_prob:
15                    fragments = fragment_packet(
16                        pkt,
17                        self.min_fragment_size,
18                        self.max_fragment_size
19                    )
20                    fragmented_packets.extend(fragments)
21                else:
22                    fragmented_packets.append(pkt)
23            else:
24                fragmented_packets.append(pkt)
25            else:
26                fragmented_packets.append(pkt)
27
28    return fragmented_packets
```

B.2 Fragment Construction Logic (rebuilder.py)

The following function performs RFC 791-compliant IP fragmentation with randomized fragment sizes.

Listing B.2: RFC-compliant stochastic IP fragmentation.

```
1 def fragment_packet(pkt, min_fragment_size=64, max_fragment_size=1200):
2     payload = bytes(pkt)[ip_header_len:]
3     fragment_id = random.randint(1, 65535)
4     payload_offset = 0
5     fragments = []
6
7     while payload_offset < len(payload):
8         remaining_payload = len(payload) - payload_offset
9         can_create_another = remaining_payload > (2 * min_fragment_size)
10
11         if not can_create_another:
12             fragment_payload_size = remaining_payload
13             more_fragments = False
14         else:
15             max_allowed_size = min(max_fragment_size,
16                                     remaining_payload - min_fragment_size)
17             fragment_payload_size = random.randint(
18                 min_fragment_size, max_allowed_size
19             )
20             more_fragments = True
21
22         # Enforce 8-byte alignment (except final fragment)
23         if more_fragments and fragment_payload_size % 8 != 0:
24             fragment_payload_size = (fragment_payload_size // 8) * 8
25
26         fragment_pkt = IP()
27         fragment_pkt.id = fragment_id
28         fragment_pkt.flags = 'MF' if more_fragments else 0
29         fragment_pkt.frag = payload_offset // 8
30
31         fragments.append(fragment_pkt)
32         payload_offset += fragment_payload_size
33
34     return fragments
```

Appendix C Complete comparison tables

C.1 F1-score comparison

Attack	Random Forest			Lucid			MLP		
	Base	Manip	Frag	Base	Manip	Frag	Base	Manip	Frag
WebDDoS	0.9880	0.7317	0.4565	0.6500	0.1639	0.0129	1.0000	0.6429	0.4762
LDAP	0.9817	0.9383	0.7940	0.8120	0.8689	0.9346	0.9936	0.9500	0.8185
PortMap	1.0000	0.9676	0.7807	0.9167	0.8792	0.9096	1.0000	0.9741	0.9022
DNS	0.9563	0.7338	0.6393	0.7018	0.8008	0.5859	0.9118	0.9094	0.8036
UDPLag	1.0000	0.9115	0.4284	0.9635	0.9665	0.5617	0.9925	0.9640	0.5173
UDP	1.0000	0.9114	0.4284	0.9997	0.9803	0.7688	0.9996	0.9721	0.7842
NTP	0.9999	0.5753	0.2048	0.9217	0.5289	0.0708	0.9972	0.5873	0.1775
SNMP	0.9993	0.9251	0.7261	0.9594	0.8854	0.8903	1.0000	0.9678	0.8688
SSDP	0.9998	0.8742	0.8174	0.9955	0.9783	0.9444	0.9992	0.9683	0.9630
SYN	0.9982	0.0000	0.0001	0.9967	0.0000	0.0002	1.0000	0.0000	0.0002
TFTP	1.0000	0.2225	0.0020	1.0000	0.2343	0.0032	0.9989	0.2341	0.0057
NetBIOS	0.9999	0.9732	0.7435	0.9063	0.7714	0.8849	0.9977	0.8562	0.8720
MSSQL	1.0000	0.9093	0.6998	0.9999	0.9790	0.9186	0.9995	0.9774	0.8983

Table C.1: F1-score comparison across models under Baseline, Manipulated, and Fragmented conditions.

C.2 FNR comparison

Attack	Random Forest			MLP			Lucid		
	Base	Manip	Frag	Base	Manip	Frag	Base	Manip	Frag
WebDDoS	0.0000	0.4024	0.6656	0.0000	0.5091	0.6791	0.5185	0.9091	0.9933
LDAP	0.0348	0.0912	0.2809	0.0127	0.0052	0.2945	0.3165	0.1675	0.0798
PortMap	0.0000	0.0346	0.3277	0.0000	0.0020	0.1404	0.0833	0.1657	0.1356
DNS	0.0822	0.3494	0.4309	0.1622	0.1022	0.3077	0.4595	0.2883	0.5538
UDPLag	0.0000	0.0554	0.6557	0.0000	0.0252	0.6351	0.0000	0.0190	0.5961
UDP	0.0000	0.0554	0.6557	0.0000	0.0009	0.3227	0.0006	0.0005	0.3386
NTP	0.0000	0.5669	0.8736	0.0000	0.5256	0.8996	0.1453	0.5897	0.9627
SNMP	0.0000	0.0357	0.3786	0.0000	0.0054	0.1925	0.0152	0.1597	0.1616
SSDP	0.0000	0.0955	0.1976	0.0000	0.0111	0.0370	0.0023	0.0047	0.0556
SYN	0.0036	1.0000	0.9999	0.0000	1.0000	0.9999	0.0066	1.0000	0.9999
TFTP	0.0000	0.8543	0.9988	0.0000	0.8594	0.9969	0.0000	0.8603	0.9983
NetBIOS	0.0001	0.0295	0.3492	0.0030	0.0000	0.1774	0.1714	0.1562	0.1669
MSSQL	0.0000	0.0246	0.3825	0.0001	0.0000	0.1462	0.0001	0.0000	0.1085

Table C.2: False Negative Rate across models under Baseline, Manipulated, and Fragmented conditions.

Table C.3: FNR Comparison Table (Exact Values per Attack) after adversarial training

Attack	Baseline	Manipulated	Fragmented
WebDDoS	0.0000	0.1455	0.0080
LDAP	0.0127	0.0052	0.0061
Portmap	0.0000	0.0031	0.0024
DNS	0.2432	0.1387	0.2615
UDPLag	0.0000	0.0068	0.0244
NTP	0.0028	0.1410	0.0198
SNMP	0.0000	0.0046	0.0204
SSDP	0.0015	0.0000	0.0000
Syn	0.0000	0.0000	0.0000
TFTP	0.0000	0.0005	0.0000
UDP	0.0000	0.0000	0.0955
NetBIOS	0.0038	0.0000	0.0183
MSSQL	0.0001	0.0000	0.0000

Table C.4: FNR Comparison Table (adv training with only manip and baseline)

Attack	Baseline	Manipulated	Fragmented
WebDDoS	0.1111	0.1273	0.6150
LDAP	0.0000	0.0052	0.0368
Portmap	0.0000	0.0006	0.1356
DNS	0.0000	0.0547	0.3077
UDPLag	0.0000	0.0075	0.4568
NTP	0.0000	0.1667	0.7130
SNMP	0.0000	0.0037	0.1517
SSDP	0.0000	0.0032	0.0000
Syn	0.0000	0.0000	0.0001
TFTP	0.0000	0.0005	0.0007
UDP	0.0000	0.0000	0.3273
NetBIOS	0.0010	0.0000	0.1597
MSSQL	0.0001	0.0000	0.1439